📂 Caminho: /Users/darwin/Projetos/Enki-mvp/BACKLOG.md

📋 Backlog Enki - SCRUM Control

Este arquivo rastreia o desenvolvimento do Enki seguindo as regras de versionamento `Vx.y.z.k`.

🟢 Backlog 0 a 7 - [ENTREGUE]

Status: Concluído | Versão: V7.5.0.1

(Veja o histórico completo nas versões anteriores)

🟢 Backlog 8 (Command Center - Side Panels) - [ENTREGUE]

Status: Concluído | Versão: V8.0.0.0

1. **Feature 8.1.0.0:** [x] Criar `CommandCenter.tsx` com Tabs (Atividades/Notificações/Mensagens).
2. **Feature 8.2.0.0:** [x] Implementar Backdrop Blur e Animação Lateral Premium.
3. **Feature 8.3.0.0:** [x] Sincronizar o Sino e o Histórico para abrirem o mesmo painel.

🟢 Backlog 9 (Gestão de Itens - Mover/Copiar) - [ENTREGUE]

Status: Concluído | Versão: V9.0.0.0

1. **Feature 9.1.0.0:** [x] Criar interface de "Mover" e "Copiar" no `CardModal`.
2. **Feature 9.2.0.0:** [x] Implementar lógica de transposição entre quadros no `cardService`.
3. **Feature 9.3.0.0:** [x] Suporte a seleção de Quadro -> Lista -> Posição.

🟢 Backlog 10 (Privacidade) - [ENTREGUE]

Status: Concluído | Versão: V10.1.0.0

1. **Feature 10.1.0.0:** [x] Implementar níveis de permissão (owner/admin/editor/viewer).
2. **Feature 10.2.0.0:** [x] Adicionar campo permissions ao Board com mapeamento userId -> nível.
3. **Feature 10.3.0.0:** [x] Atualizar createBoard para definir owner como 'owner'.

4. **Feature 10.4.0.0:** [x] Atualizar inviteUserToBoard para definir convidado como 'editor'.
5. **Feature 10.5.0.0:** [x] Implementar interface de gerenciamento de permissões no SettingsModal (agora exibindo o identifier/displayName do usuário).
6. **Feature 10.6.0.0:** [x] Adicionar campo permissions ao Card para permissões específicas por item.

🟢 Backlog 11 (Automação) - [ENTREGUE]

Status: Concluído | Versão: V11.3.0.0

1. **Feature 11.1.0.0:** [x] Criar sistema Butler simplificado com regras automáticas (`AutomationModal.tsx`).
2. **Feature 11.2.0.0:** [x] Implementar triggers: card_created, card_moved, card_archived.
3. **Feature 11.3.0.0:** [x] Implementar actions: move, add_label, set_due_date.

🟢 Backlog 12 (Hub Social - O Grande Zigurate) - [ENTREGUE]

Status: Concluído | Versão: V12.4.0.0

1. **Feature 12.1.0.0:** [x] Modelo de Perfil Expandido (`UserProfile` com bio, title, etc).
2. **Feature 12.2.0.0:** [x] Página de Perfil personalizada com métricas de usuário.
3. **Feature 12.3.0.0:** [x] Motor de Presença Real-Time via heartbeat e Firestore.
4. **Feature 12.4.0.0:** [x] Indicadores visuais de presença (Navbar, Board, CommandCenter).

🟢 Backlog 13 (Visões Supremos - Dashboards & Armazenamento) - [ENTREGUE]

Status: Concluído | Versão: V13.1.0.0

1. **Feature 13.1.0.0:** [x] Atualização de Attachment, Board, List, Card e UserProfile para suportar `storageMetrics`.
2. **Feature 13.2.0.0:** [x] Motor de armazenamento em tempo real usando batches do Firestore.
3. **Feature 13.3.0.0:** [x] Novo `StorageDashboard` detalhando consumo por Quadro, Lista e Cartão.
4. **Feature 13.4.0.0:** [x] Painel Administrativo (`/admin`) para gestão de cotas e usuários fantasmas.
5. **Feature 13.5.0.0:** [x] Propriedade de cartões automática e mensagens de erro

explícitas.

🟢 Backlog 14 (Campos Ancestrais - Custom Fields) - [ENTREGUE]

Status: Concluído | Versão: V14.0.0.0

- [x] Criação de campos personalizados (Texto, Número, Data, Checkbox, Seleção) por Quadro.
- [x] Suporte a Sufixos/Unidades (ex: h, pts, R$).
- [x] Exibição dos campos na capa do cartão (Custom Badges).

🟢 Backlog 15 (Visões de Oráculo - Calendar & Table) - [ENTREGUE]

Status: Concluído | Versão: V15.0.0.0

- [x] Alternador de Oráculo (View Switcher) na barra superior.
- [x] Visão de Tabela (Grid) com colunas dinâmicas para Custom Fields.
- [x] Visão de Calendário (Grade Mensal) com suporte a Scribe Mode.

🟢 Backlog 16.1 (Sincronização Ancestral - iCal) - [ENTREGUE]

Status: Concluído | Versão: V16.1.0.0

- [x] Exportação manual de arquivo iCal (.ics) seguindo RFC 5545.
- [x] Nova interface de abas no Modal de Configurações para organização de recursos.

🟢 Backlog 17.1 (Caminho da Liberdade - Portabilidade & UI) - [ENTREGUE]

Status: Concluído | Versão: V17.1.0.0

- [x] Exportação de dados do quadro em JSON (Backup Completo).
- [x] Exportação de cartões em CSV (Compatível com Excel/Sheets, incluindo Custom Fields).
- [x] Personalização visual: Cores para listas (Sinetes de Cor).
- [x] Otimização de Espaço: Opção de colapsar colunas no Kanban.

🟢 Backlog 17.3 (Hardening de Segurança & Markdown) - [ENTREGUE]

Status: Concluído | Versão: V17.3.0.0

- [x] **Arquitetura 3-Tier de Dados**: Migração de dados sensíveis para

`userPrivate` e lookup indexado em `userLookup`.
- [x] **Zero Data Leakage**: Blindagem de payloads de rede em membros do board (Remoção de e-mails/métricas da coleção pública).
- [x] **Markdown Seguro**: Suporte a Rich Text em Descrições e Comentários com sanitização via `DOMPurify` e `react-markdown`.
- [x] **Migração Atômica**: Motor de migração automática no login para converter usuários antigos para o novo schema seguro.
- [x] **Hardening de Regras**: Refatoração total do `firestore.rules` para eliminar erros de recursividade e vazamento de atributos.

🔴 Próximos Backlogs (Planejados)

1. **Backlog 16.2+ (Conexão com o Mundo Exterior):** V16.2.0.0
   - Implementação de iCal Feed (Link Dinâmico).
   - Integração com Google Calendar API (Visualização de eventos externos).

2. **Backlog 17.2 (Caminho da Liberdade - Filtros):** V17.2.0.0
   - Filtros persistentes (Saved

Searches).

3. **Backlog 18 (Mobile Mastery - PWA):** V18.0.0.0
   - Refinamento total para uso em dispositivos móveis (gestos de swipe).
   - Configuração de Manifesto PWA para instalação como App nativo.

4. **Backlog ∞ (Lojas Nativas - App Stores):** V∞
   - Integração do Capacitor para empacotar o código web.
   - Compilação do binário `.ipa` para Apple App Store e `.aab` para Google Play Store.

5. **Backlog (Painel Administrativo - Segurança):** `[PRIORIDADE: SEGURANÇA]`
   - Implementar exclusão completa de usuário (Banimento). A lógica no código deverá chamar duas funções simultâneas:
     - Uma para excluir a conta no Auth (`deleteUser(uid)`).
     - Outra para deletar o documento no Firestore (`deleteDoc(...)`), garantindo o acionamento da trava de segurança global no mesmo milissegundo.

📂 Caminho: /Users/darwin/Projetos/

Enki-mvp/cursorrules.md

---
documento: cursorrules
versão: 1.1
revisão: 2026-10-05
hash-pendente: <a calcular após
aprovação>
autoridade: constitucional
projeto: Enki
sintaxe-templates: mustache.js v4.x

📜 REGRAS DE OURO — PROTOCOLO DE
BLINDAGEM (Projeto Enki)

> Documento **constitucional** do projeto
Enki. Toda Ordem de Serviço
> (`prompt.md` + templates de `docs/
templates/`) é interpretada SOB
> estas regras. Em qualquer conflito,
este documento prevalece.
>
> **Glossário:**
> - **Antigravity** = agente de
codificação assistido por IA utilizado
>   como desenvolvedor neste projeto.
> - **Oráculo** = LLM externa de
validação que gera Ordens de Serviço
>   parametrizadas a partir de briefings
do Product Owner.
> - **Product Owner** = Aristóteles
Soares Benício.

---

§0. MANIFESTO (4 PILARES)

§0.1. Privacidade por Padrão (ISO 27001)
- Toda interface é construída assumindo **ambiente do cliente hostil**.
- **Isolamento de Contexto:** temas, permissões e dados sensíveis são
  injetados via *Scoped Variables*. O visual de um Board "Escriba"
  nunca vaza para o contexto global.

§0.2. Transparência e Ética Algorítmica (ISO 42001)
- Interfaces que usam IA devem expor **estado de processamento** e
  origem dos dados.
- Proibidas "alucinações de UI" e Layout Shifts não documentados.

§0.3. Arquitetura "Rigid Shell"
- **Proibido transbordamento de viewport.** Layout = grid fixo e
  previsível.
- Scroll é recurso **local e controlado**. Eliminam-se vulnerabilidades
  de overlap e bugs de "tela branca".

§0.4. Acessibilidade por Padrão

- **WCAG 2.2 AA** é o piso mínimo, validado em pipeline de CI.

---

§1. 🏗️ ARQUITETURA DE INTERFACE (RIGID SHELL)

§1.1. Morte ao `100vh` / `100vw` em containers internos
- Usar `dvh` (Dynamic Viewport Height) para compatibilidade Safari/iOS.
- Usar `svh` para telas fixas.
- Usar `lvh` apenas com justificativa documentada via `[PROVENANCE]`.

§1.2. Viewport Lockdown
- `root` e `body` com `overflow: hidden`.
- Scroll permitido APENAS em containers `.pt-scroll-area` via CSS Grid.

§1.3. PWA Mobile-First (macOS/iOS)
- `padding: env(safe-area-inset-*)` obrigatório (notches).
- `touch-action: pan-x pan-y` em áreas de scroll (evita "bounce"
  elástico que quebra Drag-and-Drop).

§1.4. Isolamento de Temas (Scoped CSS)
- **Proibidas** classes globais de tema no `body`.
- Usar `data-theme="scribe"` ou

`style={{ '--accent': ... }}` no
  container raiz do Board.

§1.5. Skeletons (Layout Shift)
- Reservar espaço exato da UI final em
Kanban / Tabela / Calendário.
- **Métrica:** CLS < 0.05 nas três
visões.

§1.6. Estética "Precision Tech"
- Densidade de informação alta.
- **Tipografia mono canônica:**
`JetBrains Mono` como família
  primária; fallback: `ui-monospace,
SFMono-Regular, "SF Mono",
  Menlo, Consolas, monospace`.
- Feedback visual imediato em escritas no
Firestore.

> 📌 **Nota constitucional:** o token
canônico foi fixado nesta
> revisão (v1.1) para remover débito
anterior. Mudanças exigem ADR
> em `docs/governance/`.

---

§2. 🔐 SEGURANÇA DE APLICAÇÃO

§2.1. Shadow Logic (Espelhamento
Frontend ↔ Firestore Rules)
- **Anti-Frontend-Security:** proibido
confiar apenas em `if(user)`

  no React.
- Toda lógica de permissão **DEVE** ter regra equivalente em
  `firestore.rules`.
- **Database Rejection:** o banco rejeita leituras quando o UID não
  está em `board.members[]` (ou campo canônico equivalente no schema
  Enki).

§2.2. Sanitização & Anti-XSS
- Todo input do usuário é tratado como **dado radioativo**.
- Sanitização **obrigatória via `DOMPurify`** (ou biblioteca auditada
  equivalente) antes de renderização no DOM.
- **Proibida** sanitização via regex manual.
- Aplicação sistemática em títulos, descrições e qualquer campo livre.

§2.3. Data Leakage (Selectors Mínimos)
- Auditar selectors: buscar APENAS o campo necessário.
  - ✅ `useUserPhotoURL(uid)`
  - ❌ `useUser(uid).photoURL`
- E-mails e metadados internos **não podem** vazar ao frontend.

§2.4. Política de Segredos
- **Proibido** segredo no client bundle

(`process.env.*` exposto,
  chaves em código, tokens em
`localStorage`).
- Usar **Firebase App Check** + variáveis
server-side / Cloud Functions.

---

§3. 🏗️ ZERO-TRUST TYPESCRIPT

§3.1. Uso de `any` = falha de
segurança **Crítica** (dívida técnica).
§3.2. Uso de `unknown` sem narrowing
explícito = severidade **Alta**.
§3.3. Proibidos `@ts-ignore` e `@ts-
expect-error` sem justificativa em
comentário e ticket no `BACKLOG.md`.
§3.4. Tipos genéricos/inconsistentes
herdados → **refatoração imediata** para
interfaces estritas.
§3.5. Funções exportadas devem ter
tipo de retorno explícito.

---

§4. 🚮 HIGIENE DE AMBIENTE (Migração
Windows → macOS)

§4.1. Anti-Metadata
- Ignorar/remover `Thumbs.db`,
`desktop.ini`, `.DS_Store`,
  `.AppleDouble`, `._*`, `Icon?` e
caminhos `C:\`.

§4.2. Line Endings
- Forçar `LF` (Unix) em todos os novos arquivos
  (`.gitattributes` com `* text=auto eol=lf`).

§4.3. Git Identity
- Novos commits usam apenas configuração `--local` do repositório
  (nunca `--global`).

---

§5. 🧠 COMANDO DE DESCONFIANÇA (Security Engineer Mode)

Antes de QUALQUER tarefa de backlog, o agente assume a persona de **Security Engineer** e executa varredura prévia em:

- `src/services/` (em especial `cardService`, `boardService`, `userService`)
- `src/hooks/`
- `src/context/`
- `src/pages/`

Vetores obrigatórios de verificação:
1. Shadow Logic (§2.1)
2. XSS / Sanitização (§2.2)
3. Data Leakage (§2.3)

4. Tipagem tênue / `any` / `unknown` (§3)
5. Vazamento de segredos (§2.4)

A varredura é **recursiva** e cobre todos os subdiretórios.

---

§6. 📊 OBSERVABILIDADE (ISO 42001)

§6.1. Decisões mediadas por IA devem gerar **log estruturado não-PII**.
§6.2. Telemetria de UI (cliques, navegação) **nunca** captura conteúdo livre digitado pelo usuário.
§6.3. Logs de erro do client são scrubados antes do envio ao backend.

---

§7. 🚨 GOVERNANÇA DE BACKLOG

§7.1. Recomendações de segurança em runtime
Toda nova recomendação de segurança identificada em runtime **DEVE** ser registrada em `BACKLOG.md` no mesmo ciclo de trabalho, com a tag `[PRIORIDADE: SEGURANÇA]`.

§7.2. Itens Críticos bloqueiam release
Itens de severidade **Crítica**

identificados em auditorias
(ex.: `AUDIT-V17.x.md`) bloqueiam
fechamento de release.

§7.3. Tags anotadas obrigatórias
Tags de release
(`vMAJOR.MINOR.PATCH.BUILD`) são
anotadas
(`git tag -a`). Tags leves são proibidas.

---

§8. 🧠 PROTOCOLO ANTI-VIBE-CODING (ISO
42001)

> "Vibe coding" = aceitar sugestão da LLM
porque "soa certo", sem
> verificação contra a stack atual,
documentação oficial e contexto
> do projeto. É considerado **falha de
governança de IA**.

§8.1. Princípio da Suspeita Reversa
A IA é tratada como **estagiário
talentoso porém desatualizado**.
Toda sugestão é considerada
"potencialmente herdada de corpus
obsoleto" até prova em contrário.

§8.2. Proibições Explícitas (Padrões
Mortos)
Proibida a introdução de:
- `var` (usar `const`/`let`)

- Class components React em código novo (preferir function + hooks)
- `componentWillMount`, `componentWillReceiveProps`, e outros lifecycles legados
- Callbacks aninhados quando `async/await` resolve
- `moment.js` (usar `date-fns` ou `Temporal` API)
- `request`, `request-promise` (usar `fetch` nativo ou `undici`)
- `node-sass` (usar `sass` / `dart-sass`)
- `enzyme` em testes novos (usar `@testing-library`)
- jQuery em qualquer hipótese

§8.3. Verificação de Atualidade da Stack

Antes de sugerir qualquer biblioteca, o agente DEVE:

1. Verificar `package.json` para versões reais instaladas.
2. Confirmar que a API sugerida existe na versão instalada
   (não em versão imaginada).
3. Em caso de dúvida sobre API: declarar a dúvida em vez de
   gerar código "plausível" (ver §9).

§8.4. Anti-Slopsquatting (Dependências Fantasma)

- **Proibido** sugerir pacote npm sem confirmação de existência real.

- Toda nova dependência exige:
  - Link para o registro npm/GitHub oficial.
  - Verificação de manutenção ativa (último commit < 12 meses).
  - Análise de superfície de ataque (downloads semanais > 10k OU
    auditoria manual documentada).

§8.5. Detecção de Antipadrões Memorizados
Sinalizar e refatorar quando encontrar:
- `try/catch` que engole erro silenciosamente
- `JSON.parse` sem `try/catch` ou schema (Zod/Valibot)
- Regex caseiro para e-mail, URL, CPF (usar libs auditadas)
- `innerHTML` (usar `textContent` ou framework)
- `eval`, `Function()` constructor, `setTimeout(string, ...)`
- JWT/tokens em `localStorage` (usar httpOnly cookies)
- `Math.random()` para qualquer uso security-sensitive
- Comparação de string com timing attack (`===` em senha/token)

§8.6. Cargo Cult Hooks (React-específico)
Sinalizar uso desnecessário de:
- `useEffect` para sincronizar estado

derivado (preferir cálculo direto)
- `useMemo`/`useCallback` sem profiling que justifique
- `useState` para valor que poderia ser ref ou constante
- `useRef` para valor que poderia ser `useState`

§8.7. Provenance Obrigatória
Toda decisão arquitetural não-óbvia introduzida pelo Antigravity
DEVE vir com comentário no formato:

```ts
// [PROVENANCE] <razão> | <fonte> | <data>
// ex.: [PROVENANCE] React 19 use() preferred over useEffect for data fetching
//      | https://react.dev/reference/react/use | 2026-06
```

§8.8. "Idiomatic Check" Antes do Commit
Para a stack declarada (Tailwind v4 / React 19 / TS 5.x / Vite atual):
- O código gerado segue a documentação OFICIAL da versão instalada?
- Há padrão mais idiomático introduzido na major recente?
- Em caso de dúvida: marcar com `// TODO: idiomatic-review` em vez de

commitar "no feeling".

---

§9. 🤔 PRINCÍPIO DO "NÃO SEI" HONESTO

> Promovido de §8.9 da v1.0 para regra estrutural na v1.1.

§9.1. Declaração de incerteza é preferível a fabricação
A IA DEVE preferir declarar incerteza a fabricar plausibilidade.

Frase canônica aceita:
> *"Não tenho certeza se esta API existe nesta versão — favor
> confirmar na documentação oficial."*

Saída proibida: gerar código "plausível" sem ancoragem verificável.

§9.2. Canal de captura de incerteza
Quando o agente declara incerteza durante uma OS, ele DEVE:
1. Marcar o ponto com `// TODO: provenance-pending` no código.
2. Registrar a pergunta em aberto no Gate humano da OS.
3. Aguardar decisão do Product Owner antes de prosseguir.

§9.3. "Hipóteses Assumidas" são

primeira-classe
Toda OS pode (e deve, quando aplicável) declarar bloco
`LISTA_HIPOTESES` no briefing. Hipóteses não declaradas são
consideradas inexistentes — Antigravity NÃO assume tacitamente.

---

§10. 🏛️ HIERARQUIA DE AUTORIDADE

Em qualquer conflito entre artefatos do projeto, esta é a ordem de
precedência inviolável:

1. **`cursorrules.md`** (este documento) — autoridade constitucional.
2. **`prompt.md`** — protocolo de OS parametrizadas.
3. **OS materializada** em `docs/orders/OS-*.md` — instrução pontual.
4. **Instrução verbal** do Product Owner em chat.

> ⚠️ Em qualquer conflito detectado, o Antigravity DEVE **abortar e
> reportar** ao Product Owner antes de executar.

---

§11. ✅ CHECKLIST DE CONFORMIDADE

(auto-verificação por commit)

- [ ] Nenhum `100vh` / `100vw` introduzido em containers internos (§1.1)
- [ ] Nenhum `any` introduzido sem ticket de débito (§3.1)
- [ ] Toda permissão de UI tem espelho em `firestore.rules` (§2.1)
- [ ] Inputs livres passam por `DOMPurify` (§2.2)
- [ ] Selectors retornam apenas campos necessários (§2.3)
- [ ] Nenhum segredo no bundle final (§2.4)
- [ ] Arquivos novos com EOL = LF (§4.2)
- [ ] WCAG 2.2 AA validado em componentes alterados (§0.4)
- [ ] `[PROVENANCE]` presente em decisões não-óbvias (§8.7)
- [ ] Nenhum padrão morto introduzido (§8.2)
- [ ] Nenhum antipadrão memorizado introduzido (§8.5)
- [ ] Tag de release é anotada, nunca leve (§7.3)

📎 RASTREABILIDADE

Campo	Valor
Versão deste documento	1.1

Revisão	2026-10-05
Documento substituído	cursorrules.md v1.0
Próxima revisão prevista	trimestral ou após ADR
Local oficial	raiz do repositório

📂 Caminho: /Users/darwin/Projetos/Enki-mvp/PROMPT.md

---
documento: prompt
versão: 1.1
revisão: 2026-10-05
hash-pendente: <a calcular após aprovação>
autoridade: protocolo
projeto: Enki
documento-superior: cursorrules.md v1.1
sintaxe-templates: mustache.js v4.x

📐 PROTOCOLO DE ORDENS DE SERVIÇO PARAMETRIZADAS — Projeto Enki

> Este documento define o **protocolo permanente** de geração,
> materialização e execução de Ordens de Serviço (OSs) no projeto
> Enki. Ele é executado SOB as Regras de Ouro do `cursorrules.md`

> (v1.1 ou superior).
>
> Em qualquer conflito, vale a Hierarquia de Autoridade definida em
> `cursorrules.md §10`.

---

1. 🎯 PROPÓSITO

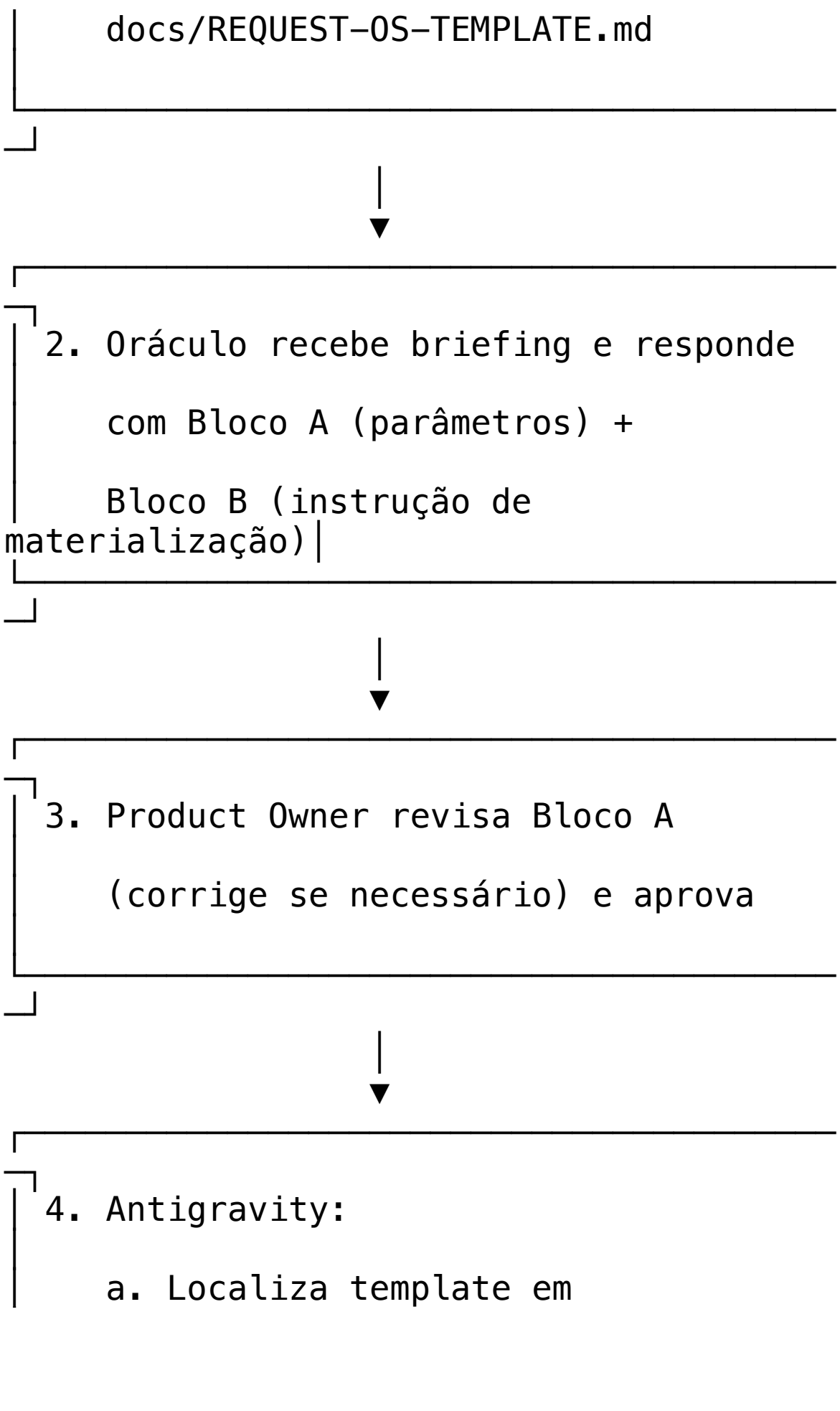
Padronizar o ciclo completo de uma OS no Enki, de modo que:

- Todo pedido do Product Owner gere uma OS **rastreável e auditável**.
- O Antigravity execute apenas instruções **resolvidas, validadas e
  ancoradas** no `cursorrules.md`.
- Nenhum placeholder Mustache vaze para o repositório.
- Nenhuma decisão "plausível" sem `[PROVENANCE]` seja aceita
  (cursorrules §8.7 + §9).

---

2. 🧭 FLUXO CANÔNICO DA OS

```
┌──────────────────────────────────────────────┐
│ 1. Product Owner preenche briefing
│
```


docs/REQUEST-OS-TEMPLATE.md
2. Oráculo recebe briefing e responde
com Bloco A (parâmetros) +
Bloco B (instrução de
materialização)
3. Product Owner revisa Bloco A
(corrige se necessário) e aprova
4. Antigravity:
a. Localiza template em

```
│
│       docs/templates/
│
│    b. Renderiza via mustache.js v4.x
│
│    c. Valida (regex + engine)
│
│    d. Salva em docs/orders/
│
│    e. Executa, parando nos Gates
│
└──────────────────────────────────────────────────────────────────────
┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────
┐
│ 5. DoD verificada → OS [EXECUTADA]
│
│    + commit/tag conforme aplicável
│
└──────────────────────────────────────────────────────────────────────
┘
```

---

3. 🔧 ENGINE MUSTACHE OFICIAL

3.1. Engine canônico
- **Implementação:** [`mustache.js`](https://github.com/janl/mustache.js)
  versão `^4.2.0`.

- **Justificativa:** padrão de mercado, manutenção ativa, sem
  dependências, especificação Mustache 1.4 completa.
- **Alternativa permitida:** apenas se documentada via ADR em
  `docs/governance/`.

3.2. Sintaxe oficial de placeholders

Tipo	Sintaxe	Renderização
Simples	`{{NOME_PARAMETRO}}`	substituído por valor único
Lista (seção positiva)	`{{#LISTA_NOME}}...{{/LISTA_NOME}}`	repetido por item
Condicional positiva	`{{#IF_X}}...{{/IF_X}}`	incluído se truthy
Condicional negativa	`{{^IF_X}}...{{/IF_X}}`	incluído se falsy
Comentário	`{{! texto }}`	renderizado como vazio

3.3. Convenção de nomenclatura
- **Variáveis simples:** `UPPER_SNAKE_CASE` (ex.: `TITULO_OS`).
- **Listas:** prefixo `LISTA_` (ex.: `LISTA_INCLUI`).
- **Flags booleanas:** prefixo `IF_` (ex.: `IF_TOCA_FIRESTORE_RULES`).

---

4. 🛡️ VALIDAÇÃO DE PLACEHOLDERS (REGRA INVIOLÁVEL)

4.1. Após renderização pelo engine

Antes de salvar QUALQUER OS materializada em `docs/orders/`, o
Antigravity DEVE executar a sequência abaixo **na ordem**:

Passo 1 — Validação de parâmetros não-resolvidos

Aplicar a regex:

```regex
\{\{(?![!#/^])[^}]+\}\}
```

> 📌 **Por que esta regex e não a anterior:**
> - `(?![!#/^])` = lookahead negativo que ignora **comentários**
> (`{{!}}`), **aberturas** (`{{#}}`), **fechamentos** (`{{/}}`) e
> **inversões** (`{{^}}`).
> - Captura **apenas** placeholders simples não-resolvidos
> (`{{NOME}}`).
> - Resolve o falso-positivo que existia na regex `\{\{[^}]+\}\}`
> da v1.0.

Se houver QUALQUER ocorrência:
- **ABORTAR** imediatamente.
- **NÃO SALVAR** o arquivo.
- Reportar no chat:
  - Quantidade de placeholders pendentes.
  - Lista exata de cada um.
  - Linha onde aparecem.
- Aguardar nova OS corrigida.

Passo 2 — Validação de seções remanescentes

Aplicar a regex:

```regex
\{\{[#/^][^}]+\}\}
```

Se houver QUALQUER ocorrência:
- **ABORTAR** com mensagem específica:
  > *"Falha de engine Mustache: aberturas/fechamentos de seção
  > remanescentes. Verificar se o engine `mustache.js` foi invocado
  > corretamente, ou se a OS forneceu o array/flag esperado."*
- Esta falha é **diferente** da anterior: indica que o **engine**
  falhou, não que o **parâmetro** está faltando.

Passo 3 — Confirmação positiva

Se ambas as regex retornarem zero ocorrências:
- Salvar o arquivo materializado.
- Prosseguir com a execução da OS.

4.2. Auto-validação adicional

Além das regex acima, o Antigravity DEVE verificar:

- **Consistência de listas:** se um template usa `{{#LISTA_X}}` mas
  o parâmetro `LISTA_X` não foi fornecido, abortar.
- **Tipos esperados:** se um parâmetro deveria ser data UTC e veio em
  outro formato, abortar.
- **Caminhos de arquivo:** se a OS referencia caminhos que não existem
  e não estão na lista de `LISTA_ARQUIVOS_CRIAR`, abortar.
- **Coerência de versão:** `CURSORRULES_VERSAO_LIDA` deve ser
  compatível com a versão do `cursorrules.md` no momento da execução.

> Esta camada impede que arquivos com `{{XYZ}}` literal vazem para o
> repositório, o que constituiria falha de governança ISO 27001/42001.

---

5. 🤔 CAPTURA DE INCERTEZA (espelha cursorrules §9)

5.1. Quando o Antigravity declarar `Não Sei` honesto

Conforme `cursorrules §9.1`, o Antigravity deve declarar incerteza em vez de fabricar plausibilidade. Quando isso ocorrer durante uma OS:

1. Marcar o ponto no código com `// TODO: provenance-pending`.
2. Registrar a pergunta em aberto no Gate humano mais próximo.
3. **Não prosseguir** sem decisão do Product Owner.

5.2. `LISTA_HIPOTESES` no briefing

Toda OS pode declarar `LISTA_HIPOTESES` no briefing (`docs/REQUEST-OS-TEMPLATE.md`). Hipóteses não declaradas são consideradas inexistentes — o Antigravity **não assume tacitamente**.

5.3. Frase canônica de incerteza

> *"Não tenho certeza se esta API existe nesta versão — favor
> confirmar na documentação oficial."*

Saída proibida: gerar código "plausível" sem ancoragem verificável.

---

6. 🏛️ HIERARQUIA EM CASO DE CONFLITO

Esta seção **delega** para `cursorrules.md §10`, que é a fonte única da hierarquia de autoridade no projeto.

Em qualquer conflito detectado entre OS, `prompt.md` e `cursorrules.md`,
o Antigravity DEVE:
1. Aplicar a precedência de `cursorrules §10`.
2. **Abortar e reportar** ao Product Owner antes de executar.

---

7. 📁 ARTEFATOS E SEUS LUGARES

Artefato	Local	Quem cria
`cursorrules.md`	raiz	constitucional, raramente alterado
`prompt.md`	raiz	este protocolo
`docs/REQUEST-OS-TEMPLATE.md`	docs/	preenchido pelo PO
`docs/templates/*.md`	docs/templates/	moldes Mustache

`docs/orders/OS-*.md`	docs/orders/	OS materializada (Antigravity)
`docs/orders-requests/REQUEST-*.md`	docs/orders-requests/	briefings arquivados (opcional)
`docs/audits/AUDIT-*.md`	docs/audits/	auditorias geradas por OSs 📕
`docs/governance/ADR-*.md`	docs/governance/	decisões arquiteturais
`docs/plans/*.md`	docs/plans/	planos macro (ex.: PLAN-V18)
`docs/diagnostics/*.md`	docs/diagnostics/	diagnósticos pontuais

---

8. 🚫 EXCEÇÕES PERMITIDAS

A única exceção ao fluxo canônico é o template `prompt-retomada.md` (perfil 📐 Molde Livre), usado para casos atípicos onde nenhum dos templates estruturados se aplica.

Mesmo nesse caso:
- A OS DEVE ser arquivada em `docs/orders/`.
- A validação de placeholders permanece obrigatória.
- `CURSORRULES_REFS` permanece obrigatório (decisão da v1.1).

---

9. 📎 RASTREABILIDADE

Campo	Valor
Versão deste protocolo	1.1
Revisão	2026-10-05
Documento superior	`cursorrules.md` v1.1
Engine Mustache oficial	`mustache.js` ^4.2.0
Próxima revisão prevista	trimestral ou após ADR

📜 HISTÓRICO DE MUDANÇAS

v1.1 — 2026-10-05
- Engine Mustache declarado oficialmente.
- Regex de validação corrigida (ignora `{{!}}`, `{{#}}`, `{{/}}`, `{{^}}`).
- Hierarquia de conflito delegada para `cursorrules §10`.
- OS V17.3→18 extraída para `docs/orders/` (não-inline).
- Captura de incerteza formalizada (espelha cursorrules §9).

v1.0
- Versão inicial com regex `\{\{[^}]+\}\}` (substituída por falsa-positiva).

📂 Caminho: /Users/darwin/Projetos/Enki-mvp/README.md

Enki - Trello Clone Premium

Desenvolvimento de um clone do Trello usando as melhores práticas de **ACID, SOLID, SCRUM, UI/UX e Secure by Design (SbD)**.

🚀 Tecnologias

- **Frontend:** React + TypeScript + Vite
- **Estilização:** Tailwind CSS v4 + Vanilla CSS
- **Backend:** Firebase (Auth, Firestore, Storage)
- **Animações:** Framer Motion / CSS Transitions

📅 Status do Projeto

Atualmente na **V17.3.0.0** – Todos os backlogs até **Segurança & Markdown (Backlog 17.3)** estão entregues.

Backlogs Entregues

| Backlog | Descrição
Versão	

```
---------------------------------- |
--------- |
| 0–7      | Fundação, CRUD, DnD, Busca
| V7.5.0.1  |
| 8        | Command Center (Side Panels)
| V8.0.0.0  |
| 9        | Gestão de Itens (Mover/
Copiar)    | V9.0.0.0  |
| 10       | Privacidade (Permissões)
| V10.1.0.0 |
| 11       | Automação (Butler
simplificado)  | V11.3.0.0 |
| 12       | Hub Social (Presença Real-
time)  | V12.4.0.0 |
| 13       | Dashboards & Métricas
| V13.1.0.0 |
| 14       | Custom Fields (Campos)
| V14.0.0.0 |
| 15       | Views (Table & Calendar)
| V15.0.0.0 |
| 16       | Export iCal
| V16.1.0.0 |
| 17       | Export JSON/CSV & Kanban UX
| V17.1.0.0 |
| 17.3     | Hardening de Segurança &
Markdown | V17.3.0.0 |

Para ver o progresso detalhado de cada
Sprint e Feature, acesse:

👉 [BACKLOG.md](./BACKLOG.md)

## 🛠️ Como rodar o projeto
```

1. Instale as dependências: `npm install`
2. Inicie o Firebase Emulator: `firebase emulators:start`
3. Inicie o servidor de desenvolvimento: `npm run dev`

🏷️ Versionamento

O projeto segue o padrão `Vx.y.z.k`:

- **x:** Major version (0 para pre-MVP)
- **y:** Backlog (Sprint)
- **z:** Feature dentro do backlog
- **k:** Correções de erro

📊 Arquitetura

```text
src/
├── components/     # UI Components (CardModal, CommandCenter, SettingsModal, etc.)
├── contexts/       # React Contexts (Auth, Theme, Notification)
├── constants/      # Design tokens e constantes
├── lib/            # Firebase config
├── pages/          # Pages (Board, Dashboard, Login)
├── services/       # Firebase services (board, card, list, automation, etc.)
└── types/          # TypeScript
```

interfaces
```

## 📋 Relatórios

- [reportStatus.md](./reportStatus.md) — Análise completa do projeto vs backlog

---

### 📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/README.md

# Documentacao do Projeto Enki

Estrutura de governanca documental sob ISO 27001 / 42001.

## Diretorios

| Diretorio | Conteudo |
|---|---|
| templates/ | Moldes de Ordem de Servico (OS) |
| orders/ | OSs executadas, arquivadas |
| audits/ | Relatorios de auditoria |
| diagnostics/ | Diagnosticos tecnicos |
| plans/ | Planos de reconstrucao / refatoracao |
| governance/ | Politicas, ADRs, conformidade |
```

Hierarquia de Autoridade

1. cursorrules.md (raiz) - Regras de Ouro
2. prompt.md (raiz) - Diretrizes Gerais
3. docs/templates/ - Moldes de execucao
4. docs/orders/ - Instancias executadas

Em conflito, o item de menor numero vence.

📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/REQUEST-OS-TEMPLATE.md

---
documento: docs/REQUEST-OS-TEMPLATE
versão: 1.1
revisão: 2026-10-05
hash-pendente: <a calcular após aprovação>
autoridade: briefing
projeto: Enki
documento-superior: cursorrules.md v1.1 + prompt.md v1.1

📩 Briefing de Pedido de Ordem de Serviço

> **Destinatário:** Oráculo de Delfos (LLM externa de validação)

> **Remetente:** Product Owner do Projeto Enki
> **Propósito:** Solicitar geração de uma nova OS parametrizada para
> execução pelo Antigravity.

Como Usar Este Briefing

1. Copie este arquivo para uma mensagem nova ao Oráculo (ou para um
   arquivo temporário, ex.: `/tmp/briefing-hoje.md`).
2. Preencha cada campo abaixo (não deixe vazio — use `N/A` quando
   genuinamente não aplicável).
3. Cole as saídas de comando solicitadas no **Bloco 4**.
4. Envie ao Oráculo.
5. Aguarde resposta com **Bloco A** (parâmetros) + **Bloco B**
   (instrução ao Antigravity).
6. (Opcional, recomendado) Arquive este briefing preenchido em
   `docs/orders-requests/REQUEST-AAAA-MM-DD-slug.md`.

BLOCO 1 — Identificação do Pedido

| Campo | Valor |

```
|---|---|
| Data UTC do pedido | `AAAA-MM-DD HH:MM`
|
| Sessão de trabalho | ex.: "manhã do dia
X" / "sprint 18 dia 3" |
| Quem solicita | ARISTÓTELES SOARES
BENÍCIO |
| Ambiente atual | macOS / Windows /
Linux |

---

## BLOCO 2 — Estado Atual do Projeto

### 2.1. Versão e Branch

| Campo | Valor |
|---|---|
| Última tag aplicada | ex.: `V17.3.0.0`
|
| Branch atual de trabalho | ex.:
`main` / `feat/appshell` |
| Há trabalho não-commitado? | Sim / Não
|
| Se sim, descrever brevemente | ex.:
"ajustes no AppShell em rascunho" |

### 2.2. Backlog Vigente

| Campo | Valor |
|---|---|
| Épico/Backlog ativo | ex.: "Backlog 18
— UI/UX Tabula Rasa" |
```

```
| Sub-épico em curso | ex.: "Sub-épico 1 — AppShell" |
| Sub-épicos pendentes | ex.: "Tokens, Headless, Skeletons" |

### 2.3. Itens Críticos Abertos

Lista breve de itens `[PRIORIDADE: SEGURANÇA]` ou Críticos da última auditoria que ainda não foram endereçados:

- `<item 1, ou "nenhum">`
- `<item 2>`

---

## BLOCO 3 — Pedido em Si

### 3.1. Objetivo

| Campo | Valor |
|---|---|
| Objetivo em UMA frase | `<verbo no infinitivo + alvo + critério>` |
| Tipo presumido | 📘 Documentação / 📗 Implementação / 📕 Governança / 📐 Livre |
| Por que agora? | ex.: "Gate 2 do plano UI foi aprovado ontem" |

### 3.2. Origem da Demanda
```

De onde nasceu este pedido?

- [ ] Item de auditoria → referenciar arquivo: `<ex.: AUDIT-V17.3.md §2.4.b>`
- [ ] Plano aprovado → referenciar arquivo: `<ex.: PLAN-V18.md §2.1>`
- [ ] Backlog do épico atual
- [ ] Dívida técnica conhecida
- [ ] Iniciativa nova (sem origem documental ainda)
- [ ] Outro: `<descrever>`

3.3. Restrições e Limites

Campo	Valor
Prazo desejado	`<data ou "sem prazo rígido">`
Dependências externas	ex.: "depende do bump do Tailwind para v4"
Escopo proibido (NÃO INCLUI)	`<itens que NÃO devem entrar nesta OS>`
Branch alvo sugerida	ex.: `feat/appshell-rigid-shell`

3.4. Critérios de Sucesso

Como saberei que esta OS foi bem executada?

- `<critério 1, mensurável>`
- `<critério 2>`
- `<critério 3>`

3.5. Hipóteses Assumidas (LISTA_HIPOTESES) ⬅ NOVO v1.1

Conforme `cursorrules.md §9.3` e `prompt.md §5.2`, toda OS pode declarar hipóteses não-óbvias. Hipóteses **não declaradas** são consideradas inexistentes — o Antigravity **não assume tacitamente**.

Liste as hipóteses sob as quais esta OS faz sentido. Use `nenhuma` quando genuinamente não houver:

- ex.: "Assumo que `firestore.rules` está sob controle de versão."
- ex.: "Assumo que o usuário tem acesso de admin para deploy."
- ex.: "Assumo que CI está configurado para rodar `npm test` em PRs."
- `<sua hipótese 4, ou "nenhuma">`

3.6. Pré-DoR — Declaração de Pré-Condições ⬅ NOVO v1.1

Antes de pedir a OS, confirme que as pré-condições estão atendidas:

- [ ] O **escopo** desta OS está claro o suficiente para 1 frase
      verificável (item 3.1).
- [ ] O **NÃO INCLUI** (item 3.3) está

explicitamente preenchido.
- [ ] As **hipóteses não-óbvias** estão declaradas (item 3.5).
- [ ] Os **critérios de sucesso** são mensuráveis (item 3.4).
- [ ] As **dependências externas** foram listadas (item 3.3).
- [ ] Esta OS **não acumula múltiplos perfis** (📘/📗/📕/📐).
      Se acumular, fatiar antes de pedir.

> Se qualquer item da Pré-DoR não puder ser marcado, **reformule o
> pedido antes de enviar ao Oráculo**.

BLOCO 4 — Anexos de Contexto (OBRIGATÓRIOS)

4.1. Árvore de OSs Executadas

Cole abaixo a saída literal de:

```bash
tree docs/orders/ 2>/dev/null || find docs/orders/ -maxdepth 2 -type f
```

```text
<colar saída aqui>
```

4.2. Tags Recentes

Cole abaixo a saída literal de:

```bash
git tag --list --sort=-version:refname |
head -10
```

```text
<colar saída aqui>
```

4.3. Status do Git

Cole abaixo a saída literal de:

```bash
git status --short --branch
```

```text
<colar saída aqui>
```

4.4. Stack Real (apenas se a OS toca código)

Cole abaixo a saída literal de:

```bash
npm ls vite tailwindcss typescript react
2>/dev/null | head -20
```

```

```text
<colar saída aqui>
```

4.5. Trecho Relevante de AUDIT/PLAN (se aplicável)

Se a OS deriva de um item específico de auditoria ou plano, cole aqui
o trecho relevante (5–15 linhas) para o Oráculo não precisar adivinhar:

```text
<colar trecho aqui, ou "N/A">
```

4.6. Versões dos Artefatos Constitucionais ⬅ NOVO v1.1

Para garantir coerência entre OS gerada e regras vigentes:

Artefato	Versão lida	Arquivo
`cursorrules.md`	ex.: 1.1	raiz do repo
`prompt.md`	ex.: 1.1	raiz do repo
`docs/templates/README.md`	ex.: 1.1	docs/templates/

> Em caso de divergência entre versão
```

lida pelo PO e versão vigente
> no repositório no momento da execução, o Antigravity DEVE abortar
> e reportar (conforme `cursorrules.md §10`).

BLOCO 5 — Output Esperado do Oráculo

Marque o que você quer receber na resposta:

- [ ] Apenas os parâmetros calculados (Bloco A)
- [ ] Parâmetros + justificativa breve de cada um
- [ ] Parâmetros + matriz de riscos antes da OS final
- [ ] Parâmetros + sugestão de fatiamento se a OS estiver grande demais
- [ ] Parâmetros + alerta de itens potencialmente fora de escopo

5.1. Nível de Verbosidade

- [ ] Mínimo (só o essencial)
- [ ] Padrão (parâmetros + comentários inline críticos)
- [ ] Extenso (com racional de cada decisão)

5.2. Formato de Entrega

- [ ] Resposta única (um só bloco)
- [ ] Bloco A separado de Bloco B (recomendado para revisão)
- [ ] Bloco A em tabela + Bloco B em texto corrido

BLOCO 6 — Observações Livres

Espaço para qualquer contexto que não coube acima:

```text
<texto livre, ou deixar vazio>
```

✅ Checklist de Submissão

Antes de enviar este briefing ao Oráculo, confirme:

- [ ] Bloco 1 totalmente preenchido
- [ ] Bloco 2 totalmente preenchido (sem "N/A" em Versão/Branch)
- [ ] Bloco 3.1 com objetivo em UMA frase verificável
- [ ] Bloco 3.5 (`LISTA_HIPOTESES`) preenchido (ou "nenhuma")
- [ ] Bloco 3.6 (Pré-DoR) com todos os

checkboxes marcados
- [ ] Bloco 4.1 com saída de `tree docs/orders/` colada
- [ ] Bloco 4.2 com tags recentes coladas
- [ ] Bloco 4.3 com `git status` colado
- [ ] Bloco 4.4 preenchido se a OS toca código
- [ ] Bloco 4.6 com versões dos artefatos constitucionais
- [ ] Bloco 5 com pelo menos um checkbox marcado
- [ ] Briefing salvo em `docs/orders-requests/REQUEST-AAAA-MM-DD-slug.md`
      (opcional, mas recomendado para auditoria do próprio fluxo)

📌 Notas Importantes

1. **Não preencher os anexos do Bloco 4** = Oráculo opera no escuro
   e pode gerar OS inconsistente com o estado real. Resista à
   tentação de pular essa etapa.

2. **Briefing minimalista** é aceito apenas para OSs do tipo
   📐 Livre (molde `prompt-retomada.md`) ou para correções triviais
   (typos, pequenos fixes).

3. **O Oráculo NÃO executa nada** —

apenas gera a OS parametrizada.
   Quem executa é o Antigravity, e somente após sua aprovação humana
   da OS gerada.

4. **Histórico de briefings** pode ser opcionalmente arquivado em
   `docs/orders-requests/` para rastreabilidade do próprio processo
   decisório (útil em auditoria ISO 42001).

============== FIM DO BRIEFING ==============

🔍 Anatomia do Briefing — Por Que Cada Bloco Existe

> Seção de referência. Não precisa ser preenchida — explica o
> racional para revisores futuros.

Bloco 1 — Identificação
Resolve: "quando isto foi pedido e em que ambiente". Importante para reproduzibilidade e para detectar pedidos antigos sendo reaproveitados
sem revisão.

Bloco 2 — Estado Atual
Resolve: "em que ponto o projeto está agora". Sem isso, o Oráculo pode gerar uma OS baseada em estado imaginado (vibe coding aplicado a OS).

Bloco 3 — Pedido
Resolve: "o quê e por quê". Forçar objetivo em **uma frase** evita escopo inflado desde a origem.

- **Bloco 3.5 (Hipóteses):** captura pressupostos que, se falsos, invalidam a OS. Materializa `cursorrules §9.3`.
- **Bloco 3.6 (Pré-DoR):** garante que o briefing não chega ao Oráculo malformado. Espelha o checklist do `docs/templates/README.md`.

Bloco 4 — Anexos (CRÍTICO)
Resolve: "que evidências objetivas você tem". `tree`, `git status`, `npm ls` são **fatos** — não interpretação. Isso elimina a necessidade de adivinhação do Oráculo.

Bloco 5 — Output
Resolve: "o que você espera receber do Oráculo". Permite que o PO module a verbosidade da resposta conforme a complexidade da OS.

Bloco 6 — Observações
Válvula de escape para tudo que não cabe nos blocos rígidos.

💡 Sugestão de Diretório Adicional

O checklist menciona `docs/orders-requests/` como local opcional para arquivar briefings. **Não é obrigatório**, mas é fortemente recomendado porque:

- Cria histórico do próprio processo decisório.
- Em auditoria ISO 42001, é o tipo de evidência valorizada.
- Permite ver padrões de pedidos ao longo do tempo.

🎯 Fluxo de Uso na Prática

```
1. cp docs/REQUEST-OS-TEMPLATE.md /tmp/briefing-hoje.md
2. Preencher /tmp/briefing-hoje.md
3. Rodar os comandos do Bloco 4 e colar as saídas
4. Colar o conteúdo de /tmp/briefing-
```

hoje.md em mensagem ao Oráculo
5. Receber a OS parametrizada (Bloco A + Bloco B)
6. Revisar Bloco A (parâmetros) — corrigir se necessário
7. Aprovar e enviar ao Antigravity
8. Antigravity materializa o template + valida + executa
```

---

## ⚠️ Pontos de Atenção

### 1. Tamanho do briefing
- Briefing completo pode passar de 200 linhas com os anexos colados.
- Isso é **intencional** — densidade de contexto é o que evita
  vibe coding.
- Em compensação, a OS gerada pelo Oráculo pode ser muito mais
  enxuta.

### 2. Briefings recorrentes
Para tarefas similares (ex.: vários sub-épicos do Backlog 18), você
pode manter um **briefing-base** e ajustar só os blocos 3 e 4.

### 3. Sigilo de anexos
- Cuidado ao colar saída de `git status` se houver paths sensíveis.
```

- Cuidado ao colar `npm ls` em projetos com pacotes privados.
- Opção: redactar manualmente antes de colar.

📜 HISTÓRICO DE MUDANÇAS

v1.1 — 2026-10-05
- Frontmatter YAML adicionado.
- Artefatos de paste removidos (limpeza completa).
- **Bloco 3.5** (`LISTA_HIPOTESES`) adicionado (cursorrules §9.3).
- **Bloco 3.6** (Pré-DoR) adicionado.
- **Bloco 4.6** (versões dos artefatos constitucionais) adicionado.
- Blocos shell envelopados em ` ```bash `.
- Seção "Anatomia" reescrita e movida ao final.

v1.0
- Versão inicial.

📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/diagnostics/UI-DIAGNOSIS.md

Diagnóstico Clínico da Interface (UI/UX)

1.1 Viewport Toxicity
Foram encontrados usos de classes utilitárias e estilos que forçam o uso integral da viewport, o que é a principal causa do scroll lateral indesejado (especialmente no Safari/macOS devido à barra de rolagem sobreposta) e das quebras de layout em dispositivos móveis (por causa das barras de navegação).

Caminho	Linha	Token	Contexto	Impacto Safari/iOS
`src/pages/AdminPanel.tsx`	85, 90, 112	`min-h-screen`	Container principal do painel	Scroll indesejado no mobile devido à safe-area.
`src/pages/Profile.tsx`	68, 76, 88	`min-h-screen`	Layout principal do perfil	Extensão da altura além da viewport real em iOS.
`src/pages/Login.tsx`	24	`h-screen w-full`	Flex container de login	`w-full` junto de comportamentos padrão e scrollbars causam scroll horizontal (scroll lateral).
`src/pages/Dashboard.tsx`	234	`min-h-screen`	Container do Dashboard	Quebra da base de altura no Safari móvel.
`src/pages/Board.tsx`	329, 349	`h-screen`	Flex wrapper principal	Força

rolagem interna e colide com dinâmicas de 100vh em Safari. |
| `src/components/CardModal.tsx` | 403 | `max-h-[90vh]` | Altura de Modal | Comportamento não adaptativo ao teclado virtual (iOS). |
| `src/components/PrivateRoute.tsx`| 8 | `h-screen` | Wrapper de loading | Shift ao transitar para layout sem h-screen. |

1.2 Vazamento de Tema (Modo Escriba)
O "Modo Escriba" é um tema pensado para aplicar a paleta específica em *Boards* do tipo *Scribe*. No entanto, está sofrendo vazamento para a aplicação inteira.

- **Origem (Onde definido):** `src/contexts/ThemeContext.tsx`
- **Escopo Desejado:** Apenas dentro de instâncias de visualização de `Board` (Scribe).
- **Escopo Indevido:** Toda a aplicação (Dashboard, Profile, AdminPanel).
- **Causa Raiz:** O Provider de tema (`ThemeContext.tsx`) gerencia o estado `scribeMode` globalmente e realiza injeção direta no root via `document.body.setAttribute("data-scribe-mode", String(scribeMode))`.
Adicionalmente, dezenas de componentes (ex: `Profile.tsx`, `AdminPanel.tsx`) consultam ativamente este contexto global

e aplicam classes utilitárias condicionalmente (ex: `scribeMode ? 'text-theme-main' : 'text-gray-900'`), acoplando regras de negócio com estilo e quebrando a especificidade.

1.3 Layout Shift (CLS)
A aplicação sofre de Cumulative Layout Shift considerável durante as respostas da API do Firebase devido à renderização bloqueante sem reserva visual.

- **Visão Kanban (`Board.tsx` / `KanbanBoard.tsx`):** Dados de listas e cartões não possuem reserva de altura. O estado intermédio apenas contém um texto `Carregando...`, causando o surgimento em bloco dos itens que infla a tela, deslocando posições.
- **Visão Tabela (`TableView.tsx`):** A estrutura nasce sem *Skeletons* de linhas. A passagem de 0 linhas de Skeleton para `n` linhas geradas subitamente quebra a experiência.
- **Visão Calendário (`CalendarView.tsx`):** Células redimensionam abruptamente quando eventos são montados.
- **Estimativa de CLS Atual:** Elevada. A falta de Skeletons (nenhum encontrado no código) e da manutenção da UI Shell implica num redesenho inteiro da DOM.

1.4 Acessibilidade Baseline (WCAG 2.2 AA)
Foram identificadas violações flagrantes no que tange os guias essenciais.

- **Landmarks (`<main>`, `<nav>`, `<aside>`):** A maioria das páginas baseia-se unicamente em `<div>` genéricas no lugar de elementos HTML5 semânticos.
- **Atributos ARIA:** Quase nula utilização de atributos `aria-*` para componentes modais (`CardModal.tsx`), menús (`CommandCenter.tsx`) ou alertas. Não há gestão adequada do foco de acessibilidade.
- **Foco Visível e Navegação por Teclado:** Falta de anéis `:focus-visible` nas abstrações de botões customizados para manter consistência sem depender dos padrões imprevisíveis do navegador.

📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/governance/README.md

Governanca

Este diretorio abriga:

- ADRs (Architecture Decision Records):

ADR-NNNN-<slug>.md
- Politicas de conformidade ISO 27001 / 42001
- Registros de revisao dos templates e cursorrules
- Historico de incidentes e licoes aprendidas

Convencoes

- ADRs numerados sequencialmente: ADR-0001, ADR-0002, ...
- Toda mudanca no cursorrules.md deve ter ADR correspondente.

📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/plans/UI-RECONSTRUCTION-PLAN.md

Plano de Reconstrução: Tabula Rasa (UI/UX)

Versão da Stack Validada:
- Tailwind CSS: `v4.2.4`
- Vite: `v8.0.10`
- React: `v19.2.6`
- TypeScript: `v6.0.3`

2.1 App Shell (Rigid Shell)
O App Shell deve abandonar as diretivas tóxicas (`h-screen`, `100vh`) e passar a usar as medidas de *Dynamic/Small Viewport Height* atreladas às safe-areas nativas. O design utilizará **CSS Grid** para trancar as áreas externas e habilitar rolagem apenas nas áreas internas designadas (`.pt-scroll-area`).

[PROVENANCE] (Melhoria de estabilidade mobile | Spec CSS Values 4 & Apple WebKit Blog | Maio 2026)

Diagrama Mermaid da Rigid Shell
```mermaid
grid
  grid-template-areas:
    "nav   header"
    "nav   main"
  grid-template-columns: 260px 1fr
  grid-template-rows: 64px 1fr
  height: calc(100svh - env(safe-area-inset-bottom))

  "nav" {
    background: var(--color-surface);
    overflow-y: auto; /* .pt-scroll-area */
  }
  "header" {
    background: var(--color-bg);
  }
```

```
  "main" {
    background: var(--color-bg);
    overflow: hidden; /* O conteúdo
interno lidará com seu próprio scroll */
  }
```
Em dispositivos móveis, o layout da grid muda para 1 coluna e a navegação se torna um Drawer off-canvas.

2.2 Sistema de Tokens
Com a atualização para **Tailwind CSS v4.2.4**, abandonamos o `tailwind.config.ts` em favor da injeção direta de variáveis via diretiva `@theme` diretamente no arquivo `index.css`.

Estratégia do Modo Escriba:
Em vez de vazar classes arbitrárias pelo DOM com ternários, o tema "Scribe" será reduzido a variáveis CSS limitadas ao contexto do componente que hospeda o *Board*. A injeção será feita via `data-theme="scribe"` aplicado **estritamente** na `<div>` raiz do Board, nunca no `document.body`.

[PROVENANCE] (Isolamento de estilo e Tailwind v4 CSS-first engine | Tailwind CSS Official Docs | Fev 2026)

Exemplo de Tokens em CSS:
```css
```

```
@theme {
  --font-mono: 'Space Mono', 'Courier New', monospace; /* Definindo token tipográfico mono */

  /* Cores padrão (Base) */
  --color-board-bg: var(--color-slate-50);
  --color-board-text: var(--color-slate-900);
}

/* Scoped Theme: Modo Escriba */
[data-theme="scribe"] {
  --color-board-bg: var(--color-amber-50);
  --color-board-text: var(--color-amber-900);
}
```

2.3 Headless Library (Modais / Popovers / Menus)

Para corrigir de forma definitiva os modais sobrepostos e a falta de acessibilidade (Gate 1), precisamos de componentes base que controlem o `z-index`, `focus trap`, e `aria-attributes` de maneira não-opinativa no estilo.

Critério	Radix UI	Ariakit	Headless UI

WCAG 2.2	Excelente (Completo)	Excelente (Completo)	Bom
Bundle Size	Médio (Modular)	Muito Leve	Leve
Atividade do Mantenedor	Altíssima	Média	Altíssima (Tailwind Labs)
Compat. Tailwind v4	100%	100%	100% nativa
Gestão Portal/z-index	Portal Isolado Nativo	Suporte a Portals	Portal Isolado Nativo

Recomendação: **Radix UI Primitives** (ex: `@radix-ui/react-dialog`, `@radix-ui/react-dropdown-menu`).
[PROVENANCE] (Padrão ouro de acessibilidade no ecossistema React, arquitetura de portais modularizada que não interfere no Grid do Rigid Shell | Radix UI Docs & W3C ARIA APG | Abril 2026). Portals serão injetados numa div root específica no fim do body `<div id="radix-portal"></div>` para prevenir quebra do App Shell Grid.

2.4 Skeletons e Mitigação de CLS
O projeto não tem *Skeletons*. Isso precisa ser implementado para alcançar **CLS < 0.05**.
[PROVENANCE] (Core Web Vitals e UX fluída | Google Web.Dev | Março 2026)

- **Visão Kanban (`Board.tsx`):**

  Ao carregar, serão renderizadas 3 colunas "fantasmas" `w-72` com altura de `h-full` (usando background pulsante simulado em Tailwind). Dentro de cada coluna fantasma, 3 *cards* opacos `h-24` para reservar o pixel-equivalent.
- **Visão Tabela (`TableView.tsx`):**
  Renderização imediata dos *Headers* da tabela, seguidos de 10 linhas artificiais simulando a altura base `h-12` da `<tr>`.
- **Visão Calendário (`CalendarView.tsx`):**
  Grid visual estática instantânea, inflando "pastilhas" de Skeleton nos dias correntes.

2.5 Inventário de Componentes

Componente	Caminho	Decisão	Justificativa
ThemeContext	`src/contexts/ThemeContext.tsx`	**REFATORAR**	Remover injeção de `data-scribe-mode` no `document.body` e não poluir o global state.
CardModal	`src/components/CardModal.tsx`	**DESCARTAR**	Componente monstruoso e não acessível. Reconstruir usando `@radix-ui/react-dialog`.
KanbanBoard	`src/components/KanbanBoard.tsx`	**REFATORAR**

Implementar `Rigid Shell` e áreas de scroll isoladas, somado aos Skeletons de layout. |
| Profile / AdminPanel | `src/pages/Profile.tsx` (etc) | **REFATORAR** | Remover classes de ternários sujos (ex: `scribeMode ? ...`) e usar CSS Custom Properties padrão. Eliminar `h-screen`. |
| TableView / CalendarView | `src/components/*.tsx` | **REFATORAR** | Adicionar fallback de *Skeleton* para inibir o layout shift durante o load. |

[PROVENANCE] (Princípio de Single Responsibility e DRY | Clean Code Patterns em React | 2026)

2.6 Roteiro High-Level (Sub-Épicos)

1. **Sub-Épico 1: Fundação do App Shell & Tailwind v4.**
   - Migrar variáveis/cores hardcoded para CSS variables (`@theme` no `index.css`).
   - Construir o CSS Grid do Rigid Shell e substituir `h-screen/100vh` por `svh/dvh` com safe-areas globais.
2. **Sub-Épico 2: Confinamento do Modo Escriba.**
   - Isolar o provedor de tema para agir apenas em instâncias de `Board`.
   - Limpar ternários espalhados nos componentes não-Scribe.

3. **Sub-Épico 3: Infraestrutura de Acessibilidade (Radix UI).**
   - Instalar primitivas de Modals, Dropdowns e Popovers.
   - Refatorar a `<div className="modal">` nativa para `<Dialog.Root>`.
4. **Sub-Épico 4: Estabilidade de Layout (Zero CLS).**
   - Desenvolver o componente estático `<KanbanSkeleton />`, `<TableSkeleton />`.
   - Plugar os skeletons no fallback do *Firebase Query State*.

📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/templates/exemplo-doc-sync.md

---
documento: docs/templates/exemplo-doc-sync
versão: 1.1
revisão: 2026-10-05
hash-pendente: <a calcular após aprovação>
autoridade: template
perfil: 📘 documentação
projeto: Enki
documento-superior: cursorrules.md v1.1 +

```
prompt.md v1.1
sintaxe: mustache.js v4.x
risco: 🟢 baixo
---

{{!
============================================
==================== }}
{{! TEMPLATE: 📘 DOCUMENTAÇÃO (exemplo-
doc-sync)                    }}
{{! Versão: 1.1 | Engine: mustache.js
v4.x                          }}
{{! Uso: sync de BACKLOG/README/
reportStatus + commit + tag opt  }}
{{!
============================================
==================== }}

{{!
--------------------------------------------
------------------ }}
{{! INVENTÁRIO DE PARÂMETROS
ESPERADOS                          }}
{{!
--------------------------------------------
------------------ }}
{{!
                           }}
{{!
OBRIGATÓRIOS:
                     }}
{{!   TITULO_OS              string  -
título imperativo curto   }}
```

```
{{!   DATA_UTC                string  -
YYYY-MM-DD                 }}
{{!   SLUG                   string  -
kebab-case nome arquivo    }}
{{!   OBJETIVO               string  - 1
frase verificável        }}
{{!   ENTREGA_RECENTE        string  - o
que foi concluído antes }}
{{!   ORIGEM_OS              string  - de
onde nasceu o pedido    }}
{{!   VERSAO_ATUAL           string  -
versão antes desta OS      }}
{{!   VERSAO_NOVA            string  -
versão a ser registrada    }}
{{!   BRANCH_ALVO            string  -
ex.: docs/sync-V17.3.1     }}
{{!   LISTA_INCLUI           array   -
itens dentro do escopo     }}
{{!   LISTA_NAO_INCLUI       array   -
itens fora do escopo       }}
{{!   LISTA_ARQUIVOS_DOCS    array   -
arquivos a atualizar       }}
{{!   MENSAGEM_COMMIT        string  -
msg Conventional Commits   }}
{{!   CURSORRULES_REFS       array   -
seções aplicáveis          }}
{{!   DOD_ITEMS              array   -
critérios verificáveis     }}
{{!
                          }}
{{! OPCIONAIS (controlados por flags
IF_):                      }}
{{!   IF_TAG_ANOTADA         bool    -
```

```
inclui bloco de tag       }}
{{!   TAG_NOME                string  - se
IF_TAG_ANOTADA         }}
{{!   TAG_MENSAGEM            string  - se
IF_TAG_ANOTADA         }}
{{!   CHANGELOG_ENTRY         string  - se
IF_TAG_ANOTADA (NOVO   }}
{{!
v1.1: obrigatório quando  }}
{{!                                    há
tag)                   }}
{{!   IF_NOVO_EPICO           bool    - se
cria novo épico        }}
{{!   NOVO_EPICO_NOME         string  - se
IF_NOVO_EPICO          }}
{{!   NOVO_EPICO_DOR          array   -
itens da DoR do novo épico}}
{{!   IF_REGISTRA_AUDITORIA   bool    - se
referencia AUDIT-*.md  }}
{{!   AUDIT_REFERENCIADO      string  -
caminho do AUDIT          }}
{{!   LISTA_ITENS_SEGURANCA   array   -
itens [PRIORIDADE: SEG.]  }}
{{!   LISTA_HIPOTESES         array   -
hipóteses não-óbvias      }}
{{!
(cursorrules §9.3)        }}
{{!   OBSERVACOES_LIVRES      string  -
texto livre adicional     }}
{{!
------------------------------------------
------------------ }}
```

```
# ORDEM DE SERVIÇO: {{TITULO_OS}}

> **Perfil:** 📘 Documentação
> **Data UTC:** {{DATA_UTC}}
> **Template:** `exemplo-doc-sync.md`
v1.1
> **Risco:** 🟢 Baixo

---

## 🎯 OBJETIVO

{{OBJETIVO}}

---

## 📍 CONTEXTO MÍNIMO

| Campo | Valor |
|---|---|
| Versão atual (antes) |
`{{VERSAO_ATUAL}}` |
| Versão a registrar | `{{VERSAO_NOVA}}`
|
| Entrega recente | {{ENTREGA_RECENTE}} |
| Origem desta OS | {{ORIGEM_OS}} |
| Branch alvo | `{{BRANCH_ALVO}}` |

{{#IF_REGISTRA_AUDITORIA}}
**Auditoria referenciada:**
`{{AUDIT_REFERENCIADO}}`
{{/IF_REGISTRA_AUDITORIA}}
```

🧭 ESCOPO

✅ INCLUI

{{#LISTA_INCLUI}}
- {{ITEM}}
{{/LISTA_INCLUI}}

🚫 NÃO INCLUI (proibido nesta OS)

{{#LISTA_NAO_INCLUI}}
- {{ITEM}}
{{/LISTA_NAO_INCLUI}}

> Em caso de dúvida sobre escopo, pausar e consultar Product Owner.

✅ DoR — Pré-condições do Pedido

Conforme `docs/REQUEST-OS-TEMPLATE.md §3.6`, esta OS só é executável se:

- [ ] `OBJETIVO` cabe em UMA frase verificável.
- [ ] `LISTA_ARQUIVOS_DOCS` lista APENAS arquivos de documentação
      (não há `.tsx`, `.ts`, `.css` ou outro código de produção).

- [ ] `VERSAO_NOVA` segue convenção semver `vMAJOR.MINOR.PATCH.BUILD`.
- [ ] `LISTA_INCLUI` e `LISTA_NAO_INCLUI` sem sobreposição.
- [ ] `ENTREGA_RECENTE` referencia entregáveis reais (não promessas).
{{#IF_TAG_ANOTADA}}
- [ ] `CHANGELOG_ENTRY` preenchido (obrigatório quando há tag).
- [ ] `TAG_NOME` segue convenção `V<MAJOR>.<MINOR>.<PATCH>.<BUILD>`.
{{/IF_TAG_ANOTADA}}
{{#IF_NOVO_EPICO}}
- [ ] `NOVO_EPICO_DOR` tem ≥ 3 itens binários.
{{/IF_NOVO_EPICO}}

> Se qualquer item da DoR não puder ser marcado, **abortar a
> execução** e devolver a OS ao Oráculo para revisão.

🚨 Sinais de que NÃO é caso de doc-sync

Se QUALQUER destes for verdadeiro, use outro template:

- A OS modifica código `.ts`/`.tsx`/`.css` → usar `exemplo-implementacao.md` (📗).
- A OS faz auditoria sistêmica de múltiplos diretórios → usar

  `exemplo-governanca.md` (📕).
- A OS gera ou referencia `AUDIT-*.md` recém-criado E corrige código
  → usar `exemplo-governanca.md` (📕).

> Apenas a **referência** a um AUDIT existente (sem correções de
> código) cabe em 📘. Geração + correção exige 📕.

{{#LISTA_HIPOTESES}}
🤔 Hipóteses Assumidas (cursorrules §9.3)

Esta OS faz sentido sob as seguintes hipóteses. Se qualquer uma se revelar falsa, **abortar e reportar**:

{{#LISTA_HIPOTESES}}
- {{ITEM}}
{{/LISTA_HIPOTESES}}

{{/LISTA_HIPOTESES}}

🔍 ETAPA 0 — VERIFICAÇÃO PRÉVIA

0.1. Confirmar branch limpa

```bash
git status
```

```

### 0.2. Verificar `.gitignore` (cursorrules §4.1)

Regras macOS obrigatórias:
- `.DS_Store`, `.AppleDouble`, `.LSOverride`, `Icon?`, `._*`
- `.env`, `.env.local`, `node_modules`, `dist`, `.vite`, `coverage`

Se ausentes, ADICIONAR antes de qualquer `git add`.

### 0.3. Verificar `.gitattributes` (cursorrules §4.2)

```text
* text=auto eol=lf
```

0.4. Reler regras aplicáveis do `cursorrules.md`

{{#CURSORRULES_REFS}}
- §{{REF}}
{{/CURSORRULES_REFS}}

0.5. Sinalizar QUALQUER divergência antes de prosseguir.

```

🛠️ ETAPA 1 — ATUALIZAÇÃO DOCUMENTAL

1.1. Arquivos a atualizar

{{#LISTA_ARQUIVOS_DOCS}}
- `{{CAMINHO}}` — {{DESCRICAO}}
{{/LISTA_ARQUIVOS_DOCS}}

1.2. `BACKLOG.md`

- Marcar épico anterior como **[ENTREGUE]** com data UTC `{{DATA_UTC}}`.
- Registrar referência à `{{VERSAO_NOVA}}`.

{{#IF_NOVO_EPICO}}
1.3. Novo Épico no `BACKLOG.md`

Criar entrada **{{NOVO_EPICO_NOME}}** com Definition of Ready (DoR)
mínima preenchida:

{{#NOVO_EPICO_DOR}}
- [ ] {{ITEM}}
{{/NOVO_EPICO_DOR}}

> Não criar histórias/tarefas executáveis nesta OS — apenas o épico
> com DoR.
{{/IF_NOVO_EPICO}}

{{#LISTA_ITENS_SEGURANCA}}

1.4. Itens `[PRIORIDADE: SEGURANÇA]`

Registrar no `BACKLOG.md` (cursorrules §7.1):

{{#LISTA_ITENS_SEGURANCA}}
- `[PRIORIDADE: SEGURANÇA]` {{ITEM}}
{{/LISTA_ITENS_SEGURANCA}}
{{/LISTA_ITENS_SEGURANCA}}

1.5. `README.md`

- Atualizar versão atual para `{{VERSAO_NOVA}}`.
- Garantir que stack documentada reflete o `package.json` real.
- Em caso de dúvida sobre versões, executar:

```bash
npm ls vite tailwindcss typescript react
```

antes de gravar.

1.6. `reportStatus.md`

Adicionar novo bloco com data UTC `{{DATA_UTC}}` contendo:

a) O que foi entregue: {{ENTREGA_RECENTE}}
b) Riscos abertos (referenciar AUDIT/

diagnostics se houver).
c) Próximos passos (sem prometer datas sem base).

{{#IF_TAG_ANOTADA}}
1.7. Entry de CHANGELOG (obrigatório com tag — v1.1)

Adicionar ao `CHANGELOG.md` (criar se não existir) bloco com:

```markdown
## {{VERSAO_NOVA}} — {{DATA_UTC}}

{{CHANGELOG_ENTRY}}
```

> ⚠️ Conforme `docs/templates/README.md`, toda tag publicada exige
> entry de CHANGELOG rastreável. Mensagem de tag (`TAG_MENSAGEM`)
> não substitui o CHANGELOG.
{{/IF_TAG_ANOTADA}}

🚪 GATE 1 ⛔ — REVISÃO DE DIFF

Quando: Etapa 1 concluída.

Apresentar no chat:

1. Saída de:

```bash
git diff --stat
```

2. Resumo das mudanças por arquivo (1 linha cada).
3. Lista de itens `[PRIORIDADE: SEGURANÇA]` criados (se houver).
4. Confirmação de que NÃO foi alterado código de produção.
{{#IF_TAG_ANOTADA}}
5. Prévia do `CHANGELOG_ENTRY` que será gravado.
{{/IF_TAG_ANOTADA}}

Aguardar: aprovação explícita do Product Owner antes da Etapa 2.

🛠️ ETAPA 2 — REGISTRO GIT (após aprovação do Gate 1)

2.1. Stage seletivo (não usar `git add .` cego)

```bash
git add {{#LISTA_ARQUIVOS_DOCS}} {{CAMINHO}} {{/LISTA_ARQUIVOS_DOCS}}
```

{{#IF_TAG_ANOTADA}}

Adicionar também o `CHANGELOG.md`:

```bash
git add CHANGELOG.md
```
{{/IF_TAG_ANOTADA}}

2.2. Commit Conventional

```bash
git commit -m "{{MENSAGEM_COMMIT}}

OS: docs/orders/OS-{{DATA_UTC}}-
{{SLUG}}.md
Cursorrules: §{{#CURSORRULES_REFS}}
{{REF}}, {{/CURSORRULES_REFS}}
{{#IF_REGISTRA_AUDITORIA}}Refs:
{{AUDIT_REFERENCIADO}}{{/
IF_REGISTRA_AUDITORIA}}"
```

{{#IF_TAG_ANOTADA}}
2.3. Tag Anotada (cursorrules §7.3)

```bash
git tag -a {{TAG_NOME}} -m
"{{TAG_MENSAGEM}}"
git push --follow-tags
```

> Tag leve é proibida. Apenas tags anotadas são aceitas para releases.
{{/IF_TAG_ANOTADA}}

{{^IF_TAG_ANOTADA}}
2.3. Push (sem tag nesta OS)

```bash
git push -u origin {{BRANCH_ALVO}}
```
{{/IF_TAG_ANOTADA}}

🚪 GATE 2 ⛔ — CONFIRMAÇÃO DE PUSH

Quando: Etapa 2 concluída.

Apresentar no chat:

1. Saída de:

```bash
git log -1 --stat
```

{{#IF_TAG_ANOTADA}}
2. Saída de:

```bash
git tag -n {{TAG_NOME}}
```

3. Confirmação de:

```bash
```

````
git ls-remote origin {{TAG_NOME}}
```
{{/IF_TAG_ANOTADA}}

{{^IF_TAG_ANOTADA}}
2. Confirmação de:

```bash
git ls-remote origin {{BRANCH_ALVO}}
```
{{/IF_TAG_ANOTADA}}

**Aguardar:** confirmação humana de fechamento da OS.

---

## ✅ DEFINITION OF DONE

### Itens fixos (sempre verificáveis)

- [ ] DoR (Pré-condições) totalmente marcada
- [ ] `.gitignore` em conformidade com `cursorrules §4.1`
- [ ] `.gitattributes` em conformidade com `cursorrules §4.2`
- [ ] `BACKLOG.md`, `README.md`, `reportStatus.md` atualizados com
      dados REAIS (não fictícios)
- [ ] Itens `[PRIORIDADE: SEGURANÇA]` registrados (se aplicável)
- [ ] Gate 1 aprovado (revisão de diff)

- [ ] Commit Conventional realizado com referência à OS e cursorrules
{{#IF_TAG_ANOTADA}}
- [ ] `CHANGELOG.md` atualizado com `CHANGELOG_ENTRY`
- [ ] Tag anotada `{{TAG_NOME}}` criada (proibida tag leve)
- [ ] Push com `--follow-tags` aceito no remoto
{{/IF_TAG_ANOTADA}}
- [ ] Gate 2 aprovado (confirmação de push)
- [ ] Nenhuma linha de código de produção alterada
- [ ] Nenhuma dependência adicionada
- [ ] OS arquivada em `docs/orders/`

### Itens específicos desta OS

{{#DOD_ITEMS}}
- [ ] {{ITEM}}
{{/DOD_ITEMS}}

---

## 🔄 ROLLBACK

> Como reverter em caso de problema:

{{#IF_TAG_ANOTADA}}
### Se a tag já foi pushada

```bash
```

```
git tag -d {{TAG_NOME}}
git push --delete origin {{TAG_NOME}}
```
{{/IF_TAG_ANOTADA}}

### Reverter o commit de docs

```bash
git revert <hash-do-commit>
git push
```

### Impactos colaterais conhecidos

- Nenhum em produção (alteração documental apenas).
- Histórico git terá commit de revert visível (auditável, esperado).
{{#IF_TAG_ANOTADA}}
- Tag deletada do remoto fica visível no histórico de operações git
  do servidor (auditável, esperado).
- Consumidores que clonaram a tag mantêm cópia local (órfã).
{{/IF_TAG_ANOTADA}}

### Após qualquer rollback

- [ ] Abrir OS de incidente em `docs/orders/` documentando causa.
- [ ] Atualizar `reportStatus.md` com bloco "Rollback {{DATA_UTC}}".
{{#IF_TAG_ANOTADA}}

```
- [ ] Não reaplicar a mesma `TAG_NOME`
(use `{{TAG_NOME}}-rev1` ou
      similar).
{{/IF_TAG_ANOTADA}}

---

{{#OBSERVACOES_LIVRES}}
## 📌 OBSERVAÇÕES

{{OBSERVACOES_LIVRES}}

---
{{/OBSERVACOES_LIVRES}}

## 📎 RASTREABILIDADE

| Campo | Valor |
|---|---|
| OS arquivada em | `docs/orders/OS-
{{DATA_UTC}}-{{SLUG}}.md` |
| Template usado | `exemplo-doc-sync.md`
v1.1 |
| Versão antes | `{{VERSAO_ATUAL}}` |
| Versão depois | `{{VERSAO_NOVA}}` |
| Documento superior | `cursorrules.md`
v1.1 + `prompt.md` v1.1 |
| Engine de renderização | `mustache.js`
^4.2.0 |
| Briefing de pedido | `docs/orders-
requests/REQUEST-{{DATA_UTC}}-
{{SLUG}}.md` (se arquivado) |
```

---

## 📜 HISTÓRICO DE MUDANÇAS DESTE TEMPLATE

### v1.1 — 2026-10-05
- Frontmatter YAML adicionado.
- Bloco DoR (Pré-condições do Pedido) adicionado.
- `LISTA_HIPOTESES` adicionada como parâmetro opcional renderizável.
- `CHANGELOG_ENTRY` obrigatório quando `IF_TAG_ANOTADA=true`
  (resolve A-05).
- Blocos shell envelopados em ` ```bash `.
- Bloco `IF_TAG_ANOTADA`/ `^IF_TAG_ANOTADA` corrigido (era ambíguo).
- Notação `§` padronizada.
- Seção "Sinais de que NÃO é caso de doc-sync" adicionada.
- Rollback expandido com checklist pós-rollback.

### v1.0
- Versão inicial.

---

### 📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/templates/exemplo-governanca.md

```
---
documento: docs/templates/exemplo-
governanca
versão: 1.1
revisão: 2026-10-05
hash-pendente: <a calcular após
aprovação>
autoridade: template
perfil: 📕 governança / release
projeto: Enki
documento-superior: cursorrules.md v1.1 +
prompt.md v1.1
sintaxe: mustache.js v4.x
risco: 🔴 alto
---

{{!
==========================================
==================== }}
{{! TEMPLATE: 📕 GOVERNANÇA / RELEASE
(exemplo-governanca)       }}
{{! Versão: 1.1 | Engine: mustache.js
v4.x                       }}
{{! Uso: auditoria sistêmica + correções
+ release com tag          }}
{{!
==========================================
==================== }}

{{!
------------------------------------------
------------------ }}
```

```
{{! INVENTÁRIO DE PARÂMETROS
ESPERADOS                              }}
{{!
---------------------------------------
------------------ }}
{{!
                            }}
{{!
OBRIGATÓRIOS:
                         }}
{{!   TITULO_OS                  string
- título imperativo    }}
{{!   DATA_UTC                   string
- YYYY-MM-DD             }}
{{!   SLUG                       string
- kebab-case             }}
{{!   OBJETIVO                   string
- 1 frase verificável   }}
{{!   CONTEXTO_VERSAO            string
- estado de versão      }}
{{!   ORIGEM_OS                  string
- de onde nasceu        }}
{{!   VERSAO_ATUAL               string
- versão antes          }}
{{!   VERSAO_NOVA                string
- versão a tagear       }}
{{!   TIPO_BUMP                  string
- MAJOR|MINOR|PATCH|BUILD}}
{{!   BRANCH_ALVO                string
- ex: chore/release-V18 }}
{{!   BRANCH_BASE                string
- ex: V17.3.0.0         }}
{{!   LISTA_INCLUI               array
```

```
- itens no escopo       }}
{{!   LISTA_NAO_INCLUI          array
- itens fora            }}
{{!   LISTA_DIRETORIOS_AUDITAR   array
- para varredura        }}
{{!   LISTA_REGRAS_VARRER       array
- regras cursorrules    }}
{{!   META_AUDITORIA            string
- caminho do AUDIT      }}
{{!   LISTA_STACK_VALIDAR       string
- pacotes para npm ls   }}
{{!   CURSORRULES_REFS          array
- seções aplicáveis     }}
{{!   LISTA_PROVENANCE_FONTES    array
- fontes oficiais       }}
{{!   TAG_NOME                  string
- tag anotada           }}
{{!   TAG_MENSAGEM              string
- mensagem da tag       }}
{{!   MENSAGEM_COMMIT           string
- commit Conventional   }}
{{!   DOD_ITEMS                 array
- critérios extras      }}
{{!
                        }}
{{! OPCIONAIS (controlados por flags
IF_):                    }}
{{!   IF_BUMP_MAJOR             bool
- breaking changes      }}
{{!   IF_BUMP_MINOR             bool
- feature add (v1.1:    }}
{{!
agora consumido)       }}
```

```
{{!   IF_REQUER_CORRECOES        bool
- Etapa 2 será executada}}
{{!   IF_REQUER_VALIDACAO_VISUAL bool
- check Safari/iOS      }}
{{!   IF_TOCA_FIRESTORE_RULES    bool
- revisão de rules      }}
{{!   IF_REQUER_HOTFIX_PLAN      bool
- plano hotfix pré-deploy}}
{{!   IF_DEPLOY_FIREBASE         bool
- inclui passos deploy  }}
{{!   LISTA_RISCOS_DECLARADOS    array
- riscos aceitos        }}
{{!   PLANO_HOTFIX               string
- se IF_REQUER_HOTFIX   }}
{{!   LISTA_BREAKING_CHANGES     array
- se IF_BUMP_MAJOR      }}
{{!   LISTA_FEATURE_NOTES        array
- se IF_BUMP_MINOR (NOVO}}
{{!
v1.1)                  }}
{{!   LISTA_MIGRATIONS           array
- migrações de dados    }}
{{!   LISTA_HIPOTESES            array
- hipóteses (§9.3)      }}
{{!   OBSERVACOES_LIVRES         string
- texto livre           }}
{{!
-------------------------------------
----------------- }}

# ORDEM DE SERVIÇO: {{TITULO_OS}}

> **Perfil:** 📕 Governança / Release
```

```
> **Data UTC:** {{DATA_UTC}}
> **Template:** `exemplo-governanca.md` v1.1
> **Risco:** 🔴 Alto
> **Bump:** {{TIPO_BUMP}} (`{{VERSAO_ATUAL}}` → `{{VERSAO_NOVA}}`)

---

## 🎯 OBJETIVO

{{OBJETIVO}}

---

## 📍 CONTEXTO MÍNIMO

| Campo | Valor |
|---|---|
| Versão atual (antes) | `{{VERSAO_ATUAL}}` |
| Versão a tagear | `{{VERSAO_NOVA}}` |
| Tipo de bump | **{{TIPO_BUMP}}** |
| Branch base | `{{BRANCH_BASE}}` |
| Branch alvo | `{{BRANCH_ALVO}}` |
| Origem desta OS | {{ORIGEM_OS}} |
| Auditoria a gerar | `{{META_AUDITORIA}}` |
{{#IF_TOCA_FIRESTORE_RULES}}
| ⚠️ Toca `firestore.rules` | **SIM** — exige revisão de Shadow Logic + deploy ordenado |
{{/IF_TOCA_FIRESTORE_RULES}}
```

{{#IF_BUMP_MAJOR}}
| ⚠️ Bump MAJOR (breaking) | **SIM** — exige documentação de Breaking Changes |
{{/IF_BUMP_MAJOR}}
{{#IF_BUMP_MINOR}}
| ℹ️ Bump MINOR (feature) | **SIM** — exige Feature Notes documentadas |
{{/IF_BUMP_MINOR}}

{{#IF_BUMP_MAJOR}}
### Breaking Changes Declarados

{{#LISTA_BREAKING_CHANGES}}
- {{ITEM}}
{{/LISTA_BREAKING_CHANGES}}

> Bump MAJOR exige que cada breaking change esteja documentado neste
> bloco. Mudanças não-listadas detectadas durante a execução exigem
> abortar a OS e rever o escopo.
{{/IF_BUMP_MAJOR}}

{{#IF_BUMP_MINOR}}
### Feature Notes Declaradas (v1.1)

{{#LISTA_FEATURE_NOTES}}
- {{ITEM}}
{{/LISTA_FEATURE_NOTES}}

> Bump MINOR exige que cada feature adicionada esteja descrita neste
> bloco. Será registrada em

`CHANGELOG.md` na Etapa 3.
{{/IF_BUMP_MINOR}}

{{#LISTA_MIGRATIONS}}
### Migrações Necessárias

{{#LISTA_MIGRATIONS}}
- {{ITEM}}
{{/LISTA_MIGRATIONS}}

> Migrações DEVEM ser idempotentes e reversíveis. Se houver migração
> destrutiva, exigir aprovação explícita adicional do Product Owner.
{{/LISTA_MIGRATIONS}}

---

## 🧭 ESCOPO

### ✅ INCLUI

{{#LISTA_INCLUI}}
- {{ITEM}}
{{/LISTA_INCLUI}}

### 🚫 NÃO INCLUI (proibido nesta OS)

{{#LISTA_NAO_INCLUI}}
- {{ITEM}}
{{/LISTA_NAO_INCLUI}}

> ⛔ Em caso de descoberta de itens fora

```
do escopo durante a execução,
> abortar e abrir OS específica. NÃO
acumular escopo nesta OS.

{{#LISTA_RISCOS_DECLARADOS}}
### Riscos Aceitos para Esta Release

{{#LISTA_RISCOS_DECLARADOS}}
- {{ITEM}}
{{/LISTA_RISCOS_DECLARADOS}}

> Riscos listados acima foram aceitos
pelo Product Owner como dívida
> consciente. Devem ser registrados em
`BACKLOG.md` com tag
> `[PRIORIDADE: SEGURANÇA]` ou
equivalente para endereçamento futuro.
{{/LISTA_RISCOS_DECLARADOS}}

---

## ✅ DoR — Pré-condições do Pedido

Conforme `docs/REQUEST-OS-TEMPLATE.md
§3.6`, esta OS só é executável se:

- [ ] `OBJETIVO` cabe em UMA frase
verificável.
- [ ] `TIPO_BUMP` está coerente com
`VERSAO_ATUAL → VERSAO_NOVA`
      (semver válido).
- [ ] `BRANCH_BASE` é tag ou branch
protegida.
```

- [ ] `META_AUDITORIA` aponta para caminho em `docs/audits/`
      seguindo nomenclatura `AUDIT-<VERSAO>.md`.
- [ ] `LISTA_DIRETORIOS_AUDITAR` contém apenas diretórios existentes
      no repo.
- [ ] `LISTA_REGRAS_VARRER` referencia seções reais do
      `cursorrules.md`.
- [ ] `LISTA_INCLUI` e `LISTA_NAO_INCLUI` sem sobreposição.
- [ ] Auditoria anterior (se existir) está em `docs/audits/` e
      acessível.
{{#IF_BUMP_MAJOR}}
- [ ] `LISTA_BREAKING_CHANGES` tem ≥ 1 item (bump MAJOR sem
      breaking change é contradição).
{{/IF_BUMP_MAJOR}}
{{#IF_BUMP_MINOR}}
- [ ] `LISTA_FEATURE_NOTES` tem ≥ 1 item (bump MINOR sem feature
      é contradição).
{{/IF_BUMP_MINOR}}
{{#IF_DEPLOY_FIREBASE}}
- [ ] PO tem credenciais para `firebase deploy` no projeto.
- [ ] Janela de manutenção comunicada (se aplicável).
{{/IF_DEPLOY_FIREBASE}}
{{#IF_REQUER_HOTFIX_PLAN}}
- [ ] `PLANO_HOTFIX` preenchido com

passos executáveis.
{{/IF_REQUER_HOTFIX_PLAN}}

> Se qualquer item da DoR não puder ser marcado, **abortar a
> execução** e devolver a OS ao Oráculo para revisão.

### 🚨 Sinais de que NÃO é caso de governanca

Se QUALQUER destes for verdadeiro, use outro template:

- A OS apenas atualiza docs/BACKLOG/README sem auditoria nem bump → `exemplo-doc-sync.md` (📘).
- A OS cria/altera código `.ts`/`.tsx` em escopo restrito SEM auditoria sistêmica → `exemplo-implementacao.md` (📗).
- A OS é uma **resposta a incidente em produção** com janela estreita → considerar criar template `exemplo-hotfix.md` (ainda não disponível — usar `prompt-retomada.md` 📐 com risco explicitamente declarado).

---

{{#LISTA_HIPOTESES}}
## 🤔 Hipóteses Assumidas (cursorrules

§9.3)

Esta OS faz sentido sob as seguintes hipóteses. Se qualquer uma se revelar falsa, **abortar e reportar**:

{{#LISTA_HIPOTESES}}
- {{ITEM}}
{{/LISTA_HIPOTESES}}

---
{{/LISTA_HIPOTESES}}

## 🔍 ETAPA 0 — VERIFICAÇÃO PRÉVIA

### 0.1. Branch e estado git

```bash
git checkout -b {{BRANCH_ALVO}}
{{BRANCH_BASE}}
git status
git log --oneline -5
```

Confirmar:
- Branch criada exatamente a partir de `{{BRANCH_BASE}}`.
- Diretório de trabalho limpo.
- Últimos 5 commits coerentes com o estado declarado.

### 0.2. Validação de stack real (cursorrules §8.3 e §8.8)

```bash
npm ls {{LISTA_STACK_VALIDAR}}
```

Registrar saída literal. Em caso de divergência com `README.md`, **ABORTAR**.

### 0.3. Higiene de ambiente (cursorrules §4)

Verificar `.gitignore` quanto a regras macOS:
- `.DS_Store`, `.AppleDouble`, `.LSOverride`, `Icon?`, `._*`
- `.env`, `.env.local`, `node_modules`, `dist`, `.vite`, `coverage`

Verificar `.gitattributes`:

```text
* text=auto eol=lf
```

Se ausentes, ADICIONAR antes de qualquer outra ação.

### 0.4. Reler regras aplicáveis do `cursorrules.md`

{{#CURSORRULES_REFS}}
- §{{REF}}

{{/CURSORRULES_REFS}}

### 0.5. Fontes de Provenance pré-aprovadas (cursorrules §8.7 + §9.1)

{{#LISTA_PROVENANCE_FONTES}}
- {{ITEM}}
{{/LISTA_PROVENANCE_FONTES}}

> 🤔 **Em caso de incerteza:** declarar honestamente conforme
> `cursorrules §9.1` em vez de fabricar plausibilidade. Marcar com
> `// TODO: provenance-pending` e registrar no Gate humano.

### 0.6. Confirmar disponibilidade de auditoria anterior

Se esta OS deriva de auditoria prévia, confirmar que o arquivo
referenciado existe e está acessível:

```bash
test -f {{META_AUDITORIA}} && echo "Auditoria: OK" || echo "Auditoria: AUSENTE"
```

### 0.7. Sinalizar divergências

Se qualquer item 0.1 a 0.6 apresentar divergência, **PARAR** e

consultar Product Owner antes de prosseguir.

---

## 🛠️ ETAPA 1 — AUDITORIA SISTÊMICA (Security Engineer Mode)

> Antes de qualquer modificação, ativar Security Engineer Mode
> conforme `cursorrules §5` — Comando de Desconfiança.

### 1.1. Escopo de varredura recursiva

Diretórios a auditar:

{{#LISTA_DIRETORIOS_AUDITAR}}
- `{{CAMINHO}}`
{{/LISTA_DIRETORIOS_AUDITAR}}

### 1.2. Regras a varrer

{{#LISTA_REGRAS_VARRER}}
- {{ITEM}}
{{/LISTA_REGRAS_VARRER}}

### 1.3. Formato de entrega da auditoria

Gerar arquivo: `{{META_AUDITORIA}}`

Estrutura obrigatória — tabela única com 5 colunas:

| Caminho:Linha | Severidade | Descrição | Recomendação | Regra Violada |

- **Severidade:** Crítico | Alto | Médio | Baixo
- **Regra Violada:** referenciar seção do cursorrules
  (ex.: §1.1, §2.2, §8.5)

### 1.4. Categorias de auditoria

#### 1.4.a. Layout / Rigid Shell (cursorrules §1)
- Buscar: `100vw`, `100vh`, `vw`, `vh` em CSS/Tailwind/inline.
- Buscar: `h-screen`, `w-screen`, `min-h-screen`.
- Sinalizar onde `dvh`/`svh`/`lvh` deveriam ser usados.
- Containers que quebrarão com a barra dinâmica do Safari iOS.
- Ausência de `env(safe-area-inset-*)` em PWA.
- Falta de `touch-action: pan-x pan-y` em áreas de scroll.
- Classes globais de tema no `body` (violação §1.4).

#### 1.4.b. Tipagem / Zero-Trust TS (cursorrules §3)
- Localizar: `any`, `unknown` sem narrowing, `as any`.

- Localizar: `@ts-ignore`, `@ts-expect-error` sem justificativa.
- Funções exportadas sem tipo de retorno explícito.
- Uso de `Function`, `Object`, `{}` como tipos.

#### 1.4.c. Segurança / Shadow Logic (cursorrules §2.1)
- Verificações de permissão no frontend sem espelho em
  `firestore.rules`.
- Coleções a auditar: `users`, `boards`, `admin`, `sessions`,
  `billing`, `audit_logs`.
- Itens sem espelhamento = severidade Crítico.

#### 1.4.d. Segurança / XSS (cursorrules §2.2)
- Renderizações de dados do usuário sem `DOMPurify`.
- Uso de `innerHTML`, `dangerouslySetInnerHTML` sem sanitização.
- Sanitização via regex manual (proibida).

#### 1.4.e. Segurança / Data Leakage (cursorrules §2.3)
- Selectors retornando objeto User completo onde 1 campo bastaria.
- E-mails/metadados internos visíveis no

frontend.
- Logs do client expondo dados sensíveis.

#### 1.4.f. Segurança / Segredos (cursorrules §2.4)
- `process.env.*` exposto no client bundle.
- Tokens/JWT em `localStorage`/`sessionStorage`.
- Chaves hardcoded em código.

#### 1.4.g. Vibe Coding / Padrões Mortos (cursorrules §8.2)
- `var`, class components React em código novo.
- Lifecycles legados (`componentWillMount`, etc.).
- `moment.js`, `request`, `node-sass`, `enzyme`, jQuery.

#### 1.4.h. Vibe Coding / Antipadrões Memorizados (cursorrules §8.5)
- `try/catch` silencioso (engole erro sem log/throw).
- `JSON.parse` sem guard.
- Regex caseiro para email/URL/CPF.
- `eval`, `Function()` constructor, `setTimeout(string, ...)`.
- `Math.random()` em contexto security-sensitive.
- Comparação de token via `===` (timing attack).

#### 1.4.i. Vibe Coding / Cargo Cult Hooks (cursorrules §8.6)
- `useEffect` para estado derivado calculável.
- `useMemo`/`useCallback` sem profiling que justifique.
- `useState` para constante; `useRef` onde `useState` caberia.

#### 1.4.j. Vibe Coding / Dependências Suspeitas (cursorrules §8.4)
- Pacotes com último release > 12 meses.
- Pacotes com downloads semanais < 10k SEM auditoria documentada.
- Nomes similares a pacotes populares (typosquatting).

#### 1.4.k. Vibe Coding / Falta de Provenance (cursorrules §8.7)
- Decisões arquiteturais não-óbvias sem `// [PROVENANCE] ...`.
- `// TODO: provenance-pending` deixados sem resolução
  (cursorrules §9.2).

{{#IF_REQUER_VALIDACAO_VISUAL}}
#### 1.4.l. Acessibilidade / WCAG 2.2 AA (cursorrules §0.4)
- Auditoria leve: contraste, foco visível, landmarks, navegação por
  teclado, `aria-*` mal usados.
{{/IF_REQUER_VALIDACAO_VISUAL}}

### 1.5. Resumo executivo da auditoria

Ao final da varredura, gerar no topo do `{{META_AUDITORIA}}`:

```text
Resumo Executivo
Total de itens identificados: <N>
Por severidade:
  Crítico: <N>
  Alto: <N>
  Médio: <N>
  Baixo: <N>

Por categoria:
  1.4.a Layout / Rigid Shell: <N>
  1.4.b Tipagem / Zero-Trust TS: <N>
  1.4.c Shadow Logic: <N>
  1.4.d XSS / Sanitização: <N>
  1.4.e Data Leakage: <N>
  1.4.f Segredos: <N>
  1.4.g Padrões Mortos: <N>
  1.4.h Antipadrões Memorizados: <N>
  1.4.i Cargo Cult Hooks: <N>
  1.4.j Dependências Suspeitas: <N>
  1.4.k Falta de Provenance: <N>

Top 5 itens Críticos:
  1. <caminho:linha> — <descrição curta>
  2. ...
```

---

## 🚪 GATE 1 ⛔ — APROVAÇÃO DA AUDITORIA

**Quando:** Etapa 1 concluída, `{{META_AUDITORIA}}` salvo.

**Apresentar no chat:**

1. Resumo executivo completo (item 1.5).
2. Lista dos TOP 5 críticos com caminho:linha e regra violada.
3. **Bloco de incerteza (v1.1):** lista de pontos onde o agente
   declarou `// TODO: provenance-pending` durante a varredura,
   conforme `cursorrules §9.2`. Cada ponto exige decisão do PO:
   - (a) responder a incerteza;
   - (b) aceitar como hipótese e registrar em `LISTA_HIPOTESES`;
   - (c) abortar a OS.
4. Pergunta explícita ao Product Owner:
   - Quais itens Críticos serão corrigidos NESTA OS?
   - Quais itens Críticos serão aceitos como risco e registrados em
     `BACKLOG.md` com `[PRIORIDADE: SEGURANÇA]`?
   - Quais itens Altos/Médios/Baixos serão corrigidos NESTA OS?

**Aguardar:** decisão explícita do Product Owner sobre o escopo de correções da Etapa 2.

> ⚠️ Itens Críticos NÃO endereçados (corrigidos OU aceitos
> formalmente) bloqueiam o release. A OS não pode prosseguir para
> Etapa 3 com Críticos pendentes silenciosamente
> (`cursorrules §7.2`).

---

{{#IF_REQUER_CORRECOES}}
## 🛠️ ETAPA 2 — APLICAÇÃO DE CORREÇÕES (após aprovação do Gate 1)

### 2.1. Princípio cirúrgico

Corrigir APENAS os itens explicitamente aprovados no Gate 1. Cada correção deve:
- Referenciar o item da auditoria pelo identificador.
- Vir acompanhada de comentário inline (v1.1 — placeholders reais):

```ts
// [FIX] AUDIT-{{VERSAO_NOVA}}.md item {{ID_ITEM}} | {{DESCRICAO_CURTA}}
```

> 📌 **Nota v1.1:** o Antigravity DEVE substituir `{{ID_ITEM}}` pelo
> identificador real do item na auditoria

(ex.: `1.4.c-03`) e
> `{{DESCRICAO_CURTA}}` por uma descrição de até 80 caracteres no
> momento da escrita do código. Comentário sem essas substituições
> viola `cursorrules §8.7` e §9.1.

- NÃO refatorar trechos adjacentes não-listados.

### 2.2. Ordem de aplicação

Aplicar correções na seguinte ordem de prioridade:

1. Críticos de Segurança (Shadow Logic, Segredos, XSS).
2. Críticos de Tipagem (`any`, narrowing).
3. Críticos de Layout (`100vh`/`100vw` em raiz).
4. Altos (mesma ordem por categoria).
5. Médios e Baixos (apenas se aprovados explicitamente).

### 2.3. Validação após cada categoria

Após corrigir cada categoria, executar:

```bash
npm run lint
npm run typecheck
npm test
```

```

Não prosseguir para próxima categoria com testes vermelhos.

### 2.4. Atualização do AUDIT

Após cada correção aplicada, atualizar `{{META_AUDITORIA}}` marcando o item como `[CORRIGIDO]` com hash do commit (preenchido após Etapa 3).

### 2.5. Itens aceitos como risco

Para cada item Crítico aceito como risco no Gate 1, registrar em `BACKLOG.md`:

```text
[PRIORIDADE: SEGURANÇA] <descrição>

Origem: AUDIT-{{VERSAO_NOVA}}.md item <ID_ITEM>
Regra: cursorrules §<seção>
Risco aceito em: {{DATA_UTC}} pelo Product Owner
```

{{#IF_TOCA_FIRESTORE_RULES}}
### 2.6. Atualização de `firestore.rules`

Se houve correção de Shadow Logic, atualizar `firestore.rules`
```

correspondente. Validar localmente:

```bash
firebase emulators:start --only firestore
npm run test:rules
```

> ⚠️ Rules NÃO devem ser deployadas nesta etapa. O deploy ocorre na
> Etapa 3, na ordem correta (rules antes do código).
{{/IF_TOCA_FIRESTORE_RULES}}

---

## 🚪 GATE 2 ⛔ — REVISÃO PRÉ-RELEASE

**Quando:** Etapa 2 concluída, todas as correções aplicadas.

**Apresentar no chat:**

1. Lista de itens corrigidos com referência ao AUDIT:

```bash
git diff --stat
```

2. Lista de itens aceitos como risco (registrados em `BACKLOG.md`).

3. Saída de:

```bash
npm run lint
npm run typecheck
npm test
```

Todos verdes.

4. Verificação final anti-vibe-coding:
- [ ] Nenhum `[PROVENANCE]` ausente em decisões não-óbvias.
- [ ] Nenhum `// TODO: provenance-pending` órfão.
- [ ] Nenhum `any` introduzido durante correções.
- [ ] Nenhuma dependência nova adicionada.
- [ ] Nenhum padrão morto introduzido.
- [ ] Todos os comentários `[FIX]` têm `ID_ITEM` e `DESCRICAO_CURTA` preenchidos (v1.1).

5. Status do AUDIT:
- Críticos: `<N corrigidos>` + `<N aceitos como risco>`
- Altos: `<N corrigidos>` + `<N pendentes>`
- Médios/Baixos: `<resumo>`

{{#IF_REQUER_VALIDACAO_VISUAL}}
6. Validação visual em Safari macOS + iOS + Chromium:

- Sem regressões visuais.
- Áreas afetadas pelas correções funcionando.
{{/IF_REQUER_VALIDACAO_VISUAL}}

{{#IF_TOCA_FIRESTORE_RULES}}
7. `firestore.rules`:
- Testes de rules passando no emulator.
- Diff das rules apresentado.
{{/IF_TOCA_FIRESTORE_RULES}}

8. Confirmação de zero Críticos pendentes (não corrigidos E não
   aceitos como risco). Esta é a condição inviolável para prosseguir.

**Aguardar:** aprovação explícita do Product Owner para Etapa 3 (Release).

> ⛔ Se houver QUALQUER Crítico pendente sem decisão formal, abortar.
> Release com Críticos silenciosos viola governança ISO 27001
> (cursorrules §0.1 + §7.2).

---
{{/IF_REQUER_CORRECOES}}

{{^IF_REQUER_CORRECOES}}
## ℹ️ ETAPA 2 — DISPENSADA

Esta OS foi configurada com

`IF_REQUER_CORRECOES = false`. A auditoria da Etapa 1 é puramente documental e não gera alterações de código nesta OS.

Itens identificados na auditoria devem ser registrados em `BACKLOG.md` para endereçamento em OSs futuras.

> ⚠️ Esta configuração só é aceitável se a auditoria não revelar
> itens Críticos. Se houver Críticos, a OS deve ser reconfigurada
> com `IF_REQUER_CORRECOES = true` ou os Críticos formalmente
> aceitos como risco no Gate 1.

---
{{/IF_REQUER_CORRECOES}}

## 🛠️ ETAPA 3 — RELEASE (após aprovação do Gate 2)

### 3.1. Sincronização documental

Atualizar arquivos canônicos com a nova versão:

- `BACKLOG.md`
  - Marcar épico/auditoria como `[ENTREGUE]` com data UTC.
  - Registrar referência à

`{{VERSAO_NOVA}}`.
  - Adicionar itens `[PRIORIDADE: SEGURANÇA]` dos riscos aceitos.

- `README.md`
  - Atualizar versão para `{{VERSAO_NOVA}}`.
  - Stack documentada deve refletir `package.json` real (validar via `npm ls`).

{{#IF_BUMP_MAJOR}}
- `CHANGELOG.md` (e/ou `BREAKING-CHANGES.md`)
  - Documentar cada item de `LISTA_BREAKING_CHANGES`.
  - Incluir guia de migração se aplicável.
{{/IF_BUMP_MAJOR}}

{{#IF_BUMP_MINOR}}
- `CHANGELOG.md` (v1.1)
  - Documentar cada item de `LISTA_FEATURE_NOTES` sob seção "Added" do release `{{VERSAO_NOVA}}`.
{{/IF_BUMP_MINOR}}

- `reportStatus.md`
  - Bloco com data UTC `{{DATA_UTC}}`.
  - Resumo da auditoria (referenciar `{{META_AUDITORIA}}`).
  - Itens corrigidos vs aceitos como risco.

  - Próximos passos.

### 3.2. Stage seletivo

```bash
git add BACKLOG.md README.md
reportStatus.md {{META_AUDITORIA}}
```

Adicionar arquivos de código corrigidos
na Etapa 2 (ver lista do
Gate 2 item 1).

{{#IF_BUMP_MAJOR}}
Adicionar `CHANGELOG.md` (e/ou `BREAKING-
CHANGES.md`):

```bash
git add CHANGELOG.md
```
{{/IF_BUMP_MAJOR}}

{{#IF_BUMP_MINOR}}
Adicionar `CHANGELOG.md`:

```bash
git add CHANGELOG.md
```
{{/IF_BUMP_MINOR}}

> NÃO usar `git add .` cego. Stage apenas
arquivos do escopo
> declarado.

### 3.3. Commit Conventional

```bash
git commit -m "{{MENSAGEM_COMMIT}}

OS: docs/orders/OS-{{DATA_UTC}}-
{{SLUG}}.md
Auditoria: {{META_AUDITORIA}}
Cursorrules: §{{#CURSORRULES_REFS}}
{{REF}}, {{/CURSORRULES_REFS}}
Bump: {{TIPO_BUMP}} ({{VERSAO_ATUAL}} ->
{{VERSAO_NOVA}})"
```

### 3.4. Atualização do AUDIT com hash

Após o commit, recuperar o hash:

```bash
HASH=$(git log -1 --format=%H)
```

Atualizar `{{META_AUDITORIA}}`
substituindo cada placeholder de hash
pelos valores reais. Fazer commit
adicional se necessário:

```bash
git add {{META_AUDITORIA}}
git commit -m "docs(audit): registra
hashes de correção em {{META_AUDITORIA}}"
```

### 3.5. Tag anotada (cursorrules §7.3)

```bash
git tag -a {{TAG_NOME}} -m
"{{TAG_MENSAGEM}}"
```

> Tag leve é proibida. Apenas anotada.

### 3.6. Push com follow-tags

```bash
git push -u origin {{BRANCH_ALVO}}
git push --follow-tags
```

{{#IF_DEPLOY_FIREBASE}}
### 3.7. Deploy Firebase (ordem CRÍTICA)

> ⚠️ Ordem inviolável: rules ANTES do
código.

#### 3.7.a. Backup das rules atuais

```bash
firebase firestore:indexes > backup-
indexes-{{DATA_UTC}}.json
firebase deploy --only firestore:rules --
project <project-id> --dry-run
```

#### 3.7.b. Deploy das rules

```bash
firebase deploy --only firestore:rules --
project <project-id>
```

Validar no console que as novas rules
estão ativas.

#### 3.7.c. Deploy do código (apenas após
rules OK)

```bash
firebase deploy --only hosting --project
<project-id>
```

#### 3.7.d. Smoke test pós-deploy
- Login funciona.
- Boards carregam.
- Permissões respeitam novas rules.
{{/IF_DEPLOY_FIREBASE}}

{{#IF_REQUER_HOTFIX_PLAN}}
### 3.8. Plano de hotfix pré-aprovado

Plano caso problemas críticos sejam
detectados pós-deploy:

{{PLANO_HOTFIX}}
{{/IF_REQUER_HOTFIX_PLAN}}

---

## 🚪 GATE 3 ⛔ — CONFIRMAÇÃO DE RELEASE

**Quando:** Etapa 3 concluída.

**Apresentar no chat:**

1. Saída de:

```bash
git log -2 --stat
```

(commit principal + commit de hashes do AUDIT).

2. Saída de:

```bash
git tag -n {{TAG_NOME}}
```

3. Confirmação de push aceito:

```bash
git ls-remote origin {{TAG_NOME}}
git ls-remote origin {{BRANCH_ALVO}}
```

4. Status do CI (se configurado): build + testes verdes.

{{#IF_DEPLOY_FIREBASE}}

5. Confirmação de deploy Firebase:
- Rules ativas (timestamp do deploy).
- Hosting ativa (URL e timestamp).
- Smoke tests passando.
{{/IF_DEPLOY_FIREBASE}}

6. URL pública de release notes ou referência à tag no remoto
   (GitHub/GitLab/Bitbucket).

7. Confirmação de que `BACKLOG.md`, `README.md` e `reportStatus.md`
   foram atualizados e refletem a nova versão.

**Aguardar:** confirmação humana de fechamento da OS.

> Após aprovação do Gate 3, a OS é considerada CONCLUÍDA. Reabrir
> exige nova OS de hotfix ou continuação.

---

## ✅ DEFINITION OF DONE

### Itens fixos (sempre verificáveis)

- [ ] DoR (Pré-condições) totalmente marcada
- [ ] Etapa 0: branch criada de `{{BRANCH_BASE}}` e stack validada
- [ ] `.gitignore` em conformidade com

`cursorrules §4.1`
- [ ] `.gitattributes` em conformidade com `cursorrules §4.2`
- [ ] Auditoria gerada em `{{META_AUDITORIA}}` com 5 colunas e resumo executivo
- [ ] Gate 1 aprovado: PO decidiu escopo de correções
- [ ] Zero `// TODO: provenance-pending` órfãos
- [ ] Zero Críticos pendentes sem decisão formal
{{#IF_REQUER_CORRECOES}}
- [ ] Correções aplicadas com comentário `[FIX]` rastreável (com `ID_ITEM` e `DESCRICAO_CURTA` preenchidos — v1.1)
- [ ] Riscos aceitos registrados em `BACKLOG.md` com `[PRIORIDADE: SEGURANÇA]`
- [ ] Lint + typecheck + test verdes após Etapa 2
{{/IF_REQUER_CORRECOES}}
- [ ] Gate 2 aprovado: revisão pré-release completa
- [ ] `BACKLOG.md`, `README.md`, `reportStatus.md` atualizados com `{{VERSAO_NOVA}}` e dados REAIS
{{#IF_BUMP_MAJOR}}
- [ ] `CHANGELOG.md` documenta cada item de `LISTA_BREAKING_CHANGES`
{{/IF_BUMP_MAJOR}}
{{#IF_BUMP_MINOR}}

- [ ] `CHANGELOG.md` documenta cada item de `LISTA_FEATURE_NOTES`
{{/IF_BUMP_MINOR}}
- [ ] AUDIT atualizado com hashes de commit
- [ ] Tag anotada `{{TAG_NOME}}` criada (proibida tag leve)
- [ ] Push com `--follow-tags` aceito no remoto
{{#IF_DEPLOY_FIREBASE}}
- [ ] Deploy Firebase executado na ordem correta (rules antes do
      código)
- [ ] Smoke tests pós-deploy aprovados
{{/IF_DEPLOY_FIREBASE}}
- [ ] Gate 3 aprovado: release confirmado publicado
- [ ] Nenhuma dependência nova adicionada (a menos que listada
      explicitamente)
- [ ] Nenhum arquivo fora do escopo declarado foi alterado
- [ ] OS arquivada em `docs/orders/`

### Itens específicos desta OS

{{#DOD_ITEMS}}
- [ ] {{ITEM}}
{{/DOD_ITEMS}}

---

## 🔄 ROLLBACK

> ⚠️ Rollback de release publicada é operação delicada. Em
> ambientes que já consumiram a tag, a melhor prática é **avançar
> com hotfix** em vez de reverter. Use os cenários abaixo apenas em
> janela de tempo curta após publicação.

### Cenário A — Antes do push da tag (Etapa 3.6)

Tudo ainda local:

```bash
git tag -d {{TAG_NOME}}
git reset --hard {{BRANCH_BASE}}
git branch -D {{BRANCH_ALVO}}
```

Impacto: zero (nada chegou ao remoto).

### Cenário B — Após push, antes de qualquer deploy

Tag pública mas código não rodando em produção:

```bash
git tag -d {{TAG_NOME}}
git push --delete origin {{TAG_NOME}}
git push --delete origin {{BRANCH_ALVO}}
```

Impacto colateral conhecido:
- Tag deletada do remoto fica visível no histórico de operações git
  do servidor (auditável, esperado).
- Consumidores que clonaram a tag mantêm cópia local (órfã).
- Comunicar imediatamente ao time/ stakeholders.

{{#IF_DEPLOY_FIREBASE}}
### Cenário C — Após deploy de rules (mas não código)

Reverter rules ANTES de deletar tag:

```bash
firebase deploy --only firestore:rules --project <project-id>
```

Validar que rules antigas voltaram.
Depois aplicar Cenário B.

Impacto:
- Janela curta (segundos) onde rules novas ficaram ativas.
- Auditar logs de acesso nesse intervalo.

### Cenário D — Após deploy completo (rules + código)

Hotfix é preferível a rollback. Se

rollback for inevitável:

1. Deploy do código anterior:

```bash
git checkout {{VERSAO_ATUAL}}
firebase deploy --only hosting --project
<project-id>
```

2. Deploy das rules anteriores (após
código):

```bash
firebase deploy --only firestore:rules --
project <project-id>
```

> Atenção: ordem inversa do deploy
normal. Aqui o código volta
> primeiro, rules depois, para evitar
janela de incompatibilidade.

3. Aplicar Cenário B (delete da tag).

4. Abrir incidente formal e investigar
causa raiz.

Impacto colateral:
- Dados criados no intervalo entre deploy
e rollback podem ficar
  órfãos ou inconsistentes.
- Auditar `audit_logs` cuidadosamente.

```
- Considerar comunicação a usuários
afetados.
{{/IF_DEPLOY_FIREBASE}}

### Após qualquer rollback

- [ ] Abrir OS de incidente em `docs/
orders/` documentando causa.
- [ ] Atualizar `reportStatus.md` com
bloco "Rollback {{DATA_UTC}}".
- [ ] Avaliar se itens da auditoria que
motivaram a OS revertida
      precisam ser re-priorizados.
- [ ] Não reaplicar a mesma `TAG_NOME`
(use `{{TAG_NOME}}-rev1` ou
      similar).

---

{{#OBSERVACOES_LIVRES}}
## 📌 OBSERVAÇÕES

{{OBSERVACOES_LIVRES}}

---
{{/OBSERVACOES_LIVRES}}

## 📎 RASTREABILIDADE

| Campo | Valor |
|---|---|
| OS arquivada em | `docs/orders/OS-
{{DATA_UTC}}-{{SLUG}}.md` |
```

```
| Template usado | `exemplo-
governanca.md` v1.1 |
| Versão antes | `{{VERSAO_ATUAL}}` |
| Versão depois | `{{VERSAO_NOVA}}` |
| Tipo de bump | {{TIPO_BUMP}} |
| Branch base | `{{BRANCH_BASE}}` |
| Branch alvo | `{{BRANCH_ALVO}}` |
| Tag publicada | `{{TAG_NOME}}` |
| Auditoria gerada | `{{META_AUDITORIA}}`
|
| Origem do pedido | {{ORIGEM_OS}} |
| Documento superior | `cursorrules.md`
v1.1 + `prompt.md` v1.1 |
| Engine de renderização | `mustache.js`
^4.2.0 |
| Briefing de pedido | `docs/orders-
requests/REQUEST-{{DATA_UTC}}-
{{SLUG}}.md` (se arquivado) |
{{#IF_BUMP_MAJOR}}
| Breaking Changes | Documentados em
`CHANGELOG.md` |
{{/IF_BUMP_MAJOR}}
{{#IF_BUMP_MINOR}}
| Feature Notes | Documentadas em
`CHANGELOG.md` |
{{/IF_BUMP_MINOR}}
{{#IF_DEPLOY_FIREBASE}}
| Deploy Firebase | rules + hosting
(ordem rules-first) |
{{/IF_DEPLOY_FIREBASE}}

---
```

## 📜 HISTÓRICO DE MUDANÇAS DESTE TEMPLATE

### v1.1 — 2026-10-05
- Frontmatter YAML adicionado.
- Bloco DoR (Pré-condições do Pedido) adicionado.
- `LISTA_HIPOTESES` adicionada como parâmetro opcional renderizável.
- TODOS os blocos shell envelopados em ` ```bash `.
- **C-04 resolvido:** `IF_BUMP_MINOR` ganha bloco condicional próprio
  + `LISTA_FEATURE_NOTES` no inventário.
- **A-03 resolvido:** modelo do comentário `[FIX]` agora usa
  placeholders Mustache reais (`{{ID_ITEM}}`, `{{DESCRICAO_CURTA}}`).
- Notação `§` padronizada (P4 → §0.4).
- Seção "Sinais de que NÃO é caso de governanca" adicionada.
- Gate 1 expandido com bloco de incerteza (`cursorrules §9.2`).
- Item de DoD para `// TODO: provenance-pending` órfão adicionado.

### v1.0
- Versão inicial.

---

### 📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/templates/exemplo-

```
implementacao.md

---
documento: docs/templates/exemplo-
implementacao
versão: 1.1
revisão: 2026-10-05
hash-pendente: <a calcular após
aprovação>
autoridade: template
perfil: 📗 implementação
projeto: Enki
documento-superior: cursorrules.md v1.1 +
prompt.md v1.1
sintaxe: mustache.js v4.x
risco: 🟡 médio
---

{{!
==========================================
==================== }}
{{! TEMPLATE: 📗 IMPLEMENTAÇÃO (exemplo-
implementacao)           }}
{{! Versão: 1.1 | Engine: mustache.js
v4.x                          }}
{{! Uso: codificar 1 feature/fix com
escopo delimitado             }}
{{!
==========================================
==================== }}

{{!
------------------------------------------
```

```
------------------- }}
{{! INVENTÁRIO DE PARÂMETROS
ESPERADOS                          }}
{{!
-----------------------------------------
------------------- }}
{{!
                          }}
{{!
OBRIGATÓRIOS:
                       }}
{{!   TITULO_OS                  string
- título imperativo    }}
{{!   DATA_UTC                   string
- YYYY-MM-DD            }}
{{!   SLUG                       string
- kebab-case            }}
{{!   OBJETIVO                   string
- 1 frase verificável   }}
{{!   CONTEXTO_VERSAO            string
- estado de versão      }}
{{!   ORIGEM_OS                  string
- ex: PLAN-V18.md §2.1  }}
{{!   BRANCH_ALVO                string
- ex: feat/appshell     }}
{{!   BRANCH_BASE                string
- ex: V17.3.0.0         }}
{{!   LISTA_INCLUI               array
- itens no escopo       }}
{{!   LISTA_NAO_INCLUI           array
- itens fora do escopo  }}
{{!   LISTA_ARQUIVOS_CRIAR       array
- novos arquivos        }}
```

```
{{!   LISTA_ARQUIVOS_MODIFICAR   array
- arquivos a alterar    }}
{{!   LISTA_STACK_VALIDAR        string
- pacotes para npm ls   }}
{{!   LISTA_TESTES_OBRIGATORIOS  array
- cenários de teste     }}
{{!   CURSORRULES_REFS           array
- seções aplicáveis     }}
{{!   LISTA_PROVENANCE_FONTES    array
- fontes oficiais       }}
{{!   MENSAGEM_COMMIT            string
- commit Conventional   }}
{{!   DOD_ITEMS                  array
- critérios extras      }}
{{!
                        }}
{{! OPCIONAIS (controlados por flags
IF_):                    }}
{{!   IF_PR_NECESSARIO           bool
- gera PR formal        }}
{{!   IF_REQUER_VALIDACAO_VISUAL bool
- check Safari/iOS      }}
{{!   IF_REQUER_TESTES_E2E       bool
- inclui Playwright/Cyp }}
{{!   IF_TOCA_FIRESTORE_RULES    bool
- revisão de rules      }}
{{!   IF_BLOQUEIA_DEPS           bool
- proibição reforçada   }}
{{!   LISTA_GREP_PROIBICOES      array
- tokens proibidos      }}
{{!   LISTA_LIBS_PERMITIDAS      array
- allowlist de imports  }}
{{!   LISTA_HIPOTESES            array
```

```
- hipóteses (§9.3)      }}
{{!   OBSERVACOES_LIVRES          string
- texto livre           }}
{{!
----------------------------------------
----------------- }}

# ORDEM DE SERVIÇO: {{TITULO_OS}}

> **Perfil:** 📗 Implementação
> **Data UTC:** {{DATA_UTC}}
> **Template:** `exemplo-
implementacao.md` v1.1
> **Risco:** 🟡 Médio

---

## 🎯 OBJETIVO

{{OBJETIVO}}

---

## 📍 CONTEXTO MÍNIMO

| Campo | Valor |
|---|---|
| Estado de versão | {{CONTEXTO_VERSAO}}
|
| Origem desta OS | {{ORIGEM_OS}} |
| Branch base | `{{BRANCH_BASE}}` |
| Branch alvo | `{{BRANCH_ALVO}}` |
{{#IF_TOCA_FIRESTORE_RULES}}
```

```
| ⚠️ Toca `firestore.rules` | **SIM** — exige revisão de Shadow Logic (`cursorrules §2.1`) |
{{/IF_TOCA_FIRESTORE_RULES}}

---

## 🧭 ESCOPO

### ✅ INCLUI

{{#LISTA_INCLUI}}
- {{ITEM}}
{{/LISTA_INCLUI}}

### 🚫 NÃO INCLUI (proibido nesta OS)

{{#LISTA_NAO_INCLUI}}
- {{ITEM}}
{{/LISTA_NAO_INCLUI}}

{{#IF_BLOQUEIA_DEPS}}
> ⛔ **Proibição reforçada:** esta OS NÃO autoriza adição de novas
> dependências. Se uma necessidade surgir durante a execução,
> abortar e abrir OS específica para avaliação da dependência.
{{/IF_BLOQUEIA_DEPS}}

{{#LISTA_LIBS_PERMITIDAS}}
### ✅ Allowlist de imports novos
```

Apenas as bibliotecas abaixo podem ser importadas em arquivos novos
desta OS:

{{#LISTA_LIBS_PERMITIDAS}}
- `{{ITEM}}`
{{/LISTA_LIBS_PERMITIDAS}}
{{/LISTA_LIBS_PERMITIDAS}}

{{#LISTA_GREP_PROIBICOES}}
### 🚫 Tokens proibidos no diff

Rejeitar commit se qualquer um dos tokens abaixo aparecer:

{{#LISTA_GREP_PROIBICOES}}
- `{{ITEM}}`
{{/LISTA_GREP_PROIBICOES}}
{{/LISTA_GREP_PROIBICOES}}

---

## ✅ DoR — Pré-condições do Pedido

Conforme `docs/REQUEST-OS-TEMPLATE.md §3.6`, esta OS só é executável se:

- [ ] `OBJETIVO` cabe em UMA frase verificável.
- [ ] `LISTA_ARQUIVOS_CRIAR` e `LISTA_ARQUIVOS_MODIFICAR` têm
      caminhos plausíveis e existentes

(ou marcados para criação).
- [ ] `LISTA_STACK_VALIDAR` lista os pacotes diretamente tocados
      pela OS (não toda a stack).
- [ ] `LISTA_TESTES_OBRIGATORIOS` define cenários verificáveis
      (não "testar bem").
- [ ] `CURSORRULES_REFS` referencia seções reais e aplicáveis.
- [ ] `LISTA_PROVENANCE_FONTES` aponta para URLs oficiais
      (não blogs/StackOverflow).
- [ ] `BRANCH_BASE` é tag ou branch protegida — não branch volátil.
- [ ] `LISTA_INCLUI` e `LISTA_NAO_INCLUI` sem sobreposição.
{{#IF_TOCA_FIRESTORE_RULES}}
- [ ] Toda nova lógica de permissão tem mapeamento prévio em
      `firestore.rules` (cursorrules §2.1 — Shadow Logic).
{{/IF_TOCA_FIRESTORE_RULES}}

> Se qualquer item da DoR não puder ser marcado, **abortar a
> execução** e devolver a OS ao Oráculo para revisão.

### 🚨 Sinais de que NÃO é caso de implementacao

Se QUALQUER destes for verdadeiro, use outro template:

- A OS apenas atualiza docs/BACKLOG/README → `exemplo-doc-sync.md` (📘).
- A OS faz auditoria sistêmica em múltiplos diretórios + bump de versão + tag de release → `exemplo-governanca.md` (📕).
- A OS toca MAIS DE UM módulo arquitetural simultaneamente → fatie em N OSs de 📗 dedicadas, OU use 📕 se for release.

---

{{#LISTA_HIPOTESES}}
## 🤔 Hipóteses Assumidas (cursorrules §9.3)

Esta OS faz sentido sob as seguintes hipóteses. Se qualquer uma se revelar falsa, **abortar e reportar**:

{{#LISTA_HIPOTESES}}
- {{ITEM}}
{{/LISTA_HIPOTESES}}

---
{{/LISTA_HIPOTESES}}

## 🔍 ETAPA 0 — VERIFICAÇÃO PRÉVIA

### 0.1. Criar branch a partir da base correta

```bash
git checkout -b {{BRANCH_ALVO}}
{{BRANCH_BASE}}
git status
```

Confirmar que a branch foi criada
exatamente a partir de
`{{BRANCH_BASE}}` e que está limpa.

### 0.2. Validar stack real (cursorrules
§8.3 e §8.8)

```bash
npm ls {{LISTA_STACK_VALIDAR}}
```

Registrar a saída literal. Se houver
divergência com versões
esperadas (ex.: README declara Vite 8 mas
instalado é Vite 7),
**ABORTAR** e reportar antes de codar.

### 0.3. Reler regras aplicáveis do
`cursorrules.md`

{{#CURSORRULES_REFS}}
- §{{REF}}
{{/CURSORRULES_REFS}}

### 0.4. Reler bloco de proveniência +
tratamento de incerteza

Esta OS exige `[PROVENANCE]` para decisões não-óbvias. Fontes oficiais disponíveis para citação:

{{#LISTA_PROVENANCE_FONTES}}
- {{ITEM}}
{{/LISTA_PROVENANCE_FONTES}}

> 🤔 **Em caso de incerteza (cursorrules §9.1):** declarar
> honestamente em vez de fabricar. Marcar o ponto no código com
> `// TODO: provenance-pending` e registrar a pergunta no Gate
> humano mais próximo.

### 0.5. Sinalizar divergências

Se qualquer item 0.1 a 0.4 apresentar divergência ou dúvida, **PARAR** e consultar Product Owner antes de prosseguir.

---

## 🛠️ ETAPA 1 — IMPLEMENTAÇÃO

### 1.1. Estrutura de arquivos a criar

{{#LISTA_ARQUIVOS_CRIAR}}
- `{{CAMINHO}}` — {{DESCRICAO}}
{{/LISTA_ARQUIVOS_CRIAR}}

### 1.2. Arquivos a modificar (escopo cirúrgico)

{{#LISTA_ARQUIVOS_MODIFICAR}}
- `{{CAMINHO}}` — {{DESCRICAO}}
{{/LISTA_ARQUIVOS_MODIFICAR}}

> ⚠️ Modificações DEVEM ser cirúrgicas. Não refatorar trechos fora
> do escopo declarado, mesmo que pareçam óbvios. Refatorações
> oportunistas violam o princípio de OS = unidade de mudança
> auditável.

### 1.3. Regras de implementação

#### 1.3.a. Tipagem (cursorrules §3 — Zero-Trust TS)
- Proibido `any`, `unknown` sem narrowing, `@ts-ignore`,
  `@ts-expect-error`.
- Funções exportadas com tipo de retorno explícito.
- Interfaces estritas para props e estados.

#### 1.3.b. Padrões de código (cursorrules §8.2 — Anti-Padrões)
- Function components only (proibido class components).
- `const`/`let` (proibido `var`).

- `async/await` (preferir sobre callbacks aninhados).
- Imports apenas de pacotes já em `package.json` (ou da allowlist).

#### 1.3.c. Provenance obrigatório (cursorrules §8.7)

Toda decisão não-óbvia DEVE vir com comentário inline:

```ts
// [PROVENANCE] <razão> | <fonte oficial> | {{DATA_UTC}}
```

Exemplo concreto:

```ts
// [PROVENANCE] React 19 use() preferred over useEffect for data fetching
//              | https://react.dev/reference/react/use | {{DATA_UTC}}
```

#### 1.3.d. Acessibilidade (cursorrules §0.4 — WCAG 2.2 AA)
- Landmarks semânticos (`<main>`, `<nav>`, `<header>`).
- Foco visível em elementos interativos.
- Contraste de cor ≥ 4.5:1 para texto normal.
- Navegação por teclado funcional.

{{#IF_TOCA_FIRESTORE_RULES}}
#### 1.3.e. Espelhamento de permissões (cursorrules §2.1 — Shadow Logic)
- Toda lógica de permissão no frontend DEVE ter regra correspondente
  em `firestore.rules`.
- Auditar: `users`, `boards`, `admin`, `sessions`, `billing`,
  `audit_logs`.
- Itens sem espelhamento = falha crítica, abortar Gate 1.
{{/IF_TOCA_FIRESTORE_RULES}}

### 1.4. Testes obrigatórios

{{#LISTA_TESTES_OBRIGATORIOS}}
- {{ITEM}}
{{/LISTA_TESTES_OBRIGATORIOS}}

Os testes devem rodar localmente antes do Gate 1:

```bash
npm run lint
npm run typecheck
npm test
```

{{#IF_REQUER_TESTES_E2E}}
Testes E2E também são obrigatórios:

```bash
```

```
npm run test:e2e
```
{{/IF_REQUER_TESTES_E2E}}

{{#IF_REQUER_VALIDACAO_VISUAL}}
### 1.5. Validação visual obrigatória

- Testar em **Safari macOS** (versão atual).
- Testar em **iOS Safari** (real device ou simulador).
- Testar em **Chromium** (Chrome/Edge atual).
- Verificar:
  - Sem scroll lateral em viewport mobile e desktop.
  - Sem salto na barra dinâmica do Safari iOS.
  - Notch / safe-area respeitados em PWA.
  - Áreas de scroll local funcionam isoladamente.
{{/IF_REQUER_VALIDACAO_VISUAL}}

---

## 🚪 GATE 1 ⛔ — REVISÃO PRÉ-COMMIT

**Quando:** Etapa 1 concluída, todos os testes verdes localmente.

**Apresentar no chat:**

1. Lista de arquivos criados/modificados:

```bash
git status
git diff --stat
```

2. Saída do test runner (todos verdes).

3. Saída de lint + typecheck (zero erros, zero warnings novos).

4. **Verificação de tokens proibidos:**

{{#LISTA_GREP_PROIBICOES}}
```bash
grep -RIn "{{ITEM}}"
{{#LISTA_ARQUIVOS_CRIAR}}{{CAMINHO}} {{/
LISTA_ARQUIVOS_CRIAR}}
{{#LISTA_ARQUIVOS_MODIFICAR}}{{CAMINHO}}
{{/LISTA_ARQUIVOS_MODIFICAR}}
```
{{/LISTA_GREP_PROIBICOES}}

Cada grep DEVE retornar zero ocorrências. Caso contrário, abortar.

5. Confirmação de presença de `[PROVENANCE]` em decisões não-óbvias.

6. **Auto-validação anti-vibe-coding** (cursorrules §8):

- [ ] Nenhum `any` introduzido sem ticket

de débito.
- [ ] Nenhum padrão morto (var, class component, moment.js, etc.).
- [ ] Nenhuma dependência nova fora da allowlist.
- [ ] Nenhum `useEffect` para estado derivado calculável.
- [ ] Nenhum cargo cult (`useMemo`/`useCallback` sem profiling).
- [ ] Nenhum `// TODO: provenance-pending` deixado sem decisão.

{{#IF_REQUER_VALIDACAO_VISUAL}}
7. Validação visual em Safari + Chromium aprovada.
{{/IF_REQUER_VALIDACAO_VISUAL}}

{{#IF_TOCA_FIRESTORE_RULES}}
8. Confirmação de espelhamento Shadow Logic ↔ `firestore.rules`.
{{/IF_TOCA_FIRESTORE_RULES}}

**Aguardar:** aprovação explícita do Product Owner antes da Etapa 2.

---

## 🛠️ ETAPA 2 — VALIDAÇÃO LOCAL E COMMIT (após Gate 1)

### 2.1. Validações finais antes do commit

```bash
npm run lint
npm run typecheck
npm test
```

{{#IF_REQUER_TESTES_E2E}}
```bash
npm run test:e2e
```
{{/IF_REQUER_TESTES_E2E}}

Todas devem retornar zero erros e zero warnings novos.

### 2.2. Stage seletivo (não usar `git add .` cego)

```bash
git add {{#LISTA_ARQUIVOS_CRIAR}}{{CAMINHO}} {{/LISTA_ARQUIVOS_CRIAR}}{{#LISTA_ARQUIVOS_MODIFICAR}}{{CAMINHO}}{{/LISTA_ARQUIVOS_MODIFICAR}}
```

> ⚠️ NÃO commitar arquivos fora do escopo declarado, mesmo que
> `git status` mostre alterações adicionais. Se houver arquivos
> extras modificados, abortar e investigar a origem antes de
> prosseguir.

### 2.3. Commit Conventional

```bash
git commit -m "{{MENSAGEM_COMMIT}}

OS: docs/orders/OS-{{DATA_UTC}}-
{{SLUG}}.md
Plano: {{ORIGEM_OS}}
Cursorrules: §{{#CURSORRULES_REFS}}
{{REF}}, {{/CURSORRULES_REFS}}
Provenance: {{#LISTA_PROVENANCE_FONTES}}
{{ITEM}}; {{/LISTA_PROVENANCE_FONTES}}"
```

### 2.4. Push para remoto

```bash
git push -u origin {{BRANCH_ALVO}}
```

{{#IF_PR_NECESSARIO}}
### 2.5. Abertura de Pull Request

Abrir PR de `{{BRANCH_ALVO}}` para
`{{BRANCH_BASE}}` (ou branch
principal conforme política do projeto)
com:

- **Título:** {{MENSAGEM_COMMIT}}
- **Descrição:** referenciar OS, plano e
cursorrules.
- **Reviewers:** Product Owner.
- **Labels:** conforme convenção do

repositório.
{{/IF_PR_NECESSARIO}}

---

## 🚪 GATE 2 ⛔ — CONFIRMAÇÃO DE PUSH

**Quando:** Etapa 2 concluída.

**Apresentar no chat:**

1. Saída de:

```bash
git log -1 --stat
```

2. Confirmação de push aceito no remoto:

```bash
git ls-remote origin {{BRANCH_ALVO}}
```

3. Saída do CI (se configurado): build verde, testes verdes.

{{#IF_PR_NECESSARIO}}
4. URL do Pull Request aberto.
5. Status dos checks automatizados do PR.
{{/IF_PR_NECESSARIO}}

**Aguardar:** aprovação humana para:
- merge (se PR), OU

- continuação para próxima OS (próximo sub-épico).

---

## ✅ DEFINITION OF DONE

### Itens fixos (sempre verificáveis)

- [ ] DoR (Pré-condições) totalmente marcada
- [ ] Etapa 0: branch criada de `{{BRANCH_BASE}}` e stack validada
- [ ] Arquivos criados conforme `LISTA_ARQUIVOS_CRIAR`
- [ ] Arquivos modificados conforme `LISTA_ARQUIVOS_MODIFICAR`
- [ ] Nenhum arquivo fora do escopo foi alterado
- [ ] Zero ocorrências de tokens proibidos (`LISTA_GREP_PROIBICOES`)
- [ ] Zero `any`/`unknown` sem narrowing (`cursorrules §3`)
- [ ] Toda decisão não-óbvia com `[PROVENANCE]` (`cursorrules §8.7`)
- [ ] Zero `// TODO: provenance-pending` órfãos (`cursorrules §9.2`)
- [ ] Lint + typecheck + test verdes
{{#IF_REQUER_TESTES_E2E}}
- [ ] Testes E2E verdes
{{/IF_REQUER_TESTES_E2E}}
{{#IF_REQUER_VALIDACAO_VISUAL}}
- [ ] Validação visual em Safari macOS +

```
iOS + Chromium aprovada
{{/IF_REQUER_VALIDACAO_VISUAL}}
{{#IF_TOCA_FIRESTORE_RULES}}
- [ ] Shadow Logic espelhada em
`firestore.rules` (`cursorrules §2.1`)
{{/IF_TOCA_FIRESTORE_RULES}}
- [ ] Gates 1 e 2 aprovados
- [ ] Commit Conventional com refs à OS,
plano e cursorrules
- [ ] Branch publicada no remoto
{{#IF_PR_NECESSARIO}}
- [ ] Pull Request aberto e com checks
verdes
{{/IF_PR_NECESSARIO}}
- [ ] Nenhuma dependência nova adicionada
- [ ] OS arquivada em `docs/orders/`

### Itens específicos desta OS

{{#DOD_ITEMS}}
- [ ] {{ITEM}}
{{/DOD_ITEMS}}

---

## 🔄 ROLLBACK

### Cenário A — antes do merge (branch
ainda isolada)

```bash
git checkout {{BRANCH_BASE}}
git branch -D {{BRANCH_ALVO}}
```
```

```
git push origin --delete {{BRANCH_ALVO}}
```

Impacto: zero (branch nunca chegou ao código principal).

### Cenário B — após merge (já em branch principal)

```bash
git revert -m 1 <hash-do-merge>
git push
```

Impactos colaterais conhecidos:
- Revert visível no histórico (esperado e auditável).
- Componentes/hooks que dependiam do código revertido podem requerer
  ajuste — abrir OS de remediação se necessário.

{{#IF_TOCA_FIRESTORE_RULES}}
### Cenário C — `firestore.rules` em produção

Se as rules foram deployadas:

```bash
firebase deploy --only firestore:rules --project <project-id>
```

```
Restaurar versão anterior das rules ANTES
de reverter o código, para
evitar janela de inconsistência
permission ↔ código.
{{/IF_TOCA_FIRESTORE_RULES}}

### Após qualquer rollback

- [ ] Abrir OS de incidente em `docs/
orders/` documentando causa.
- [ ] Atualizar `reportStatus.md` com
bloco "Rollback {{DATA_UTC}}".
- [ ] Avaliar se itens da OS revertida
precisam ser re-priorizados no
      `BACKLOG.md`.

---

{{#OBSERVACOES_LIVRES}}
## 📌 OBSERVAÇÕES

{{OBSERVACOES_LIVRES}}

---
{{/OBSERVACOES_LIVRES}}

## 📎 RASTREABILIDADE

| Campo | Valor |
|---|---|
| OS arquivada em | `docs/orders/OS-
{{DATA_UTC}}-{{SLUG}}.md` |
| Template usado | `exemplo-
```

implementacao.md` v1.1 |
| Branch base | `{{BRANCH_BASE}}` |
| Branch alvo | `{{BRANCH_ALVO}}` |
| Origem do pedido | {{ORIGEM_OS}} |
| Documento superior | `cursorrules.md` v1.1 + `prompt.md` v1.1 |
| Engine de renderização | `mustache.js` ^4.2.0 |
| Briefing de pedido | `docs/orders-requests/REQUEST-{{DATA_UTC}}-{{SLUG}}.md` (se arquivado) |

---

## 📜 HISTÓRICO DE MUDANÇAS DESTE TEMPLATE

### v1.1 — 2026-10-05
- Frontmatter YAML adicionado.
- Bloco DoR (Pré-condições do Pedido) adicionado.
- `LISTA_HIPOTESES` adicionada como parâmetro opcional renderizável.
- TODOS os blocos shell envelopados em ```bash ` (havia muitos
  soltos no v1.0).
- Seções Mustache `LISTA_LIBS_PERMITIDAS` e `LISTA_GREP_PROIBICOES`
  corrigidas (aninhamento ambíguo).
- Notação `§` padronizada (P4 → §0.4).
- Etapa 0.4 expandida com bloco de tratamento de incerteza
  (`cursorrules §9`).

- Item de DoD para `// TODO: provenance-pending` órfão adicionado.
- Seção "Sinais de que NÃO é caso de implementacao" adicionada.

### v1.0
- Versão inicial.

---

### 📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/templates/prompt-retomada.md

---
documento: docs/templates/prompt-retomada
versão: 1.1
revisão: 2026-10-05
hash-pendente: <a calcular após aprovação>
autoridade: template
perfil: 📐 molde-livre
projeto: Enki
documento-superior: cursorrules.md v1.1 + prompt.md v1.1
sintaxe: mustache.js v4.x
risco: variável
---

{{!
========================================
==================== }}
{{! TEMPLATE: 📐 MOLDE LIVRE (prompt-

```
retomada)                          }}
{{! Versão: 1.1 | Engine: mustache.js
v4.x                               }}
{{! Uso: tarefas atípicas que não se
encaixam nos 3 estruturados }}
{{!
========================================
================== }}

{{!
----------------------------------------
---------------- }}
{{! INVENTÁRIO DE PARÂMETROS
ESPERADOS                          }}
{{!
----------------------------------------
---------------- }}
{{!
                          }}
{{!
OBRIGATÓRIOS:
                        }}
{{!   TITULO_OS              string  -
título imperativo curto   }}
{{!   DATA_UTC               string  -
YYYY-MM-DD                }}
{{!   SLUG                   string  -
kebab-case nome arquivo   }}
{{!   OBJETIVO               string  - 1
frase verificável         }}
{{!   CONTEXTO_ESTADO        string  -
estado atual do projeto   }}
{{!   ORIGEM_OS              string  - de
```

```
onde nasceu o pedido   }}
{{!   LISTA_INCLUI           array   -
itens dentro do escopo    }}
{{!   LISTA_NAO_INCLUI       array   -
itens fora do escopo      }}
{{!   ETAPAS                 array   -
passos da execução        }}
{{!   GATES                  array   -
pontos de aprovação (≥1)  }}
{{!   DOD_ITEMS              array   -
critérios verificáveis    }}
{{!   CURSORRULES_REFS       array   -
seções aplicáveis (NOVO   }}
{{!
v1.1: obrigatório aqui)  }}
{{!
                          }}
{{! OPCIONAIS (controlados por flags
IF_):                      }}
{{!   IF_TOCA_CODIGO         bool    -
inclui Etapa 0 stack      }}
{{!   IF_TOCA_PRODUCAO       bool    -
inclui ROLLBACK           }}
{{!   IF_GERA_COMMIT         bool    -
inclui MENSAGEM_COMMIT    }}
{{!   IF_GERA_TAG            bool    -
inclui TAG_NOME + msg     }}
{{!   IF_REQUER_PROVENANCE   bool    -
exige bloco PROVENANCE    }}
{{!   BRANCH_ALVO            string  - se
IF_TOCA_CODIGO         }}
{{!   LISTA_STACK_VALIDAR    string  -
pacotes para npm ls       }}
```

```
{{!   LISTA_ARQUIVOS_COMMIT  array   - se
IF_GERA_COMMIT          }}
{{!   MENSAGEM_COMMIT        string  - se
IF_GERA_COMMIT          }}
{{!   TAG_NOME               string  - se
IF_GERA_TAG             }}
{{!   TAG_MENSAGEM           string  - se
IF_GERA_TAG             }}
{{!   ROLLBACK_COMANDOS      string  - se
IF_TOCA_PRODUCAO        }}
{{!   ROLLBACK_IMPACTOS      string  - se
IF_TOCA_PRODUCAO        }}
{{!   LISTA_HIPOTESES        array   -
hipóteses não-óbvias      }}
{{!
(cursorrules §9.3)        }}
{{!   OBSERVACOES_LIVRES     string  -
texto livre adicional     }}
{{!
------------------------------------------
------------------ }}

# ORDEM DE SERVIÇO: {{TITULO_OS}}

> **Perfil:** 📐 Molde Livre
> **Data UTC:** {{DATA_UTC}}
> **Template:** `prompt-retomada.md` v1.1
> **Risco:** variável (declarar nas
etapas)

---

## 🎯 OBJETIVO
```

{{OBJETIVO}}

---

## 📍 CONTEXTO MÍNIMO

**Estado atual do projeto:**
{{CONTEXTO_ESTADO}}

**Origem desta OS:**
{{ORIGEM_OS}}

{{#IF_TOCA_CODIGO}}
**Branch alvo:** `{{BRANCH_ALVO}}`
{{/IF_TOCA_CODIGO}}

---

## 🧭 ESCOPO

### ✅ INCLUI

{{#LISTA_INCLUI}}
- {{ITEM}}
{{/LISTA_INCLUI}}

### 🚫 NÃO INCLUI (proibido nesta OS)

{{#LISTA_NAO_INCLUI}}
- {{ITEM}}
{{/LISTA_NAO_INCLUI}}

---

## ✅ DoR — Pré-condições do Pedido

Conforme `docs/REQUEST-OS-TEMPLATE.md §3.6`, esta OS só é executável se as pré-condições abaixo estiverem atendidas:

- [ ] Objetivo (`OBJETIVO`) cabe em UMA frase verificável.
- [ ] `LISTA_INCLUI` e `LISTA_NAO_INCLUI` preenchidas (sem sobreposição entre elas).
- [ ] Origem da OS (`ORIGEM_OS`) referenciada com caminho/§ exato.
- [ ] Pelo menos UM Gate humano definido em `GATES`.
- [ ] Aplicabilidade do 📐 Molde Livre justificada (esta OS de fato não se encaixa em 📘 / 📗 / 📕 — ver "Sinais" abaixo).

{{#IF_TOCA_CODIGO}}
- [ ] Stack relevante listada em `LISTA_STACK_VALIDAR`.
- [ ] Branch alvo (`BRANCH_ALVO`) segue convenção do perfil.
{{/IF_TOCA_CODIGO}}

{{#IF_TOCA_PRODUCAO}}
- [ ] `ROLLBACK_COMANDOS` e `ROLLBACK_IMPACTOS` preenchidos.
{{/IF_TOCA_PRODUCAO}}

> Se qualquer item da DoR não puder ser marcado, **abortar a
> execução** e devolver a OS ao Oráculo para revisão.

### 🚨 Sinais de que NÃO é caso de Molde Livre

Se QUALQUER destes for verdadeiro, esta OS deveria usar template estruturado em vez de `prompt-retomada.md`:

- A OS só atualiza BACKLOG / README / reportStatus → usar
  `exemplo-doc-sync.md` (📘).
- A OS cria/altera arquivos `.tsx`/`.ts`/`.css` com escopo
  conhecido → usar `exemplo-implementacao.md` (📗).
- A OS faz bump de versão, gera AUDIT-*.md ou tag de release →
  usar `exemplo-governanca.md` (📕).

> Em caso de dúvida, escolha o template de **maior rigor** (📕 > 📗 >
> 📘 > 📐), conforme `docs/templates/README.md`.

---

{{#LISTA_HIPOTESES}}
## 🤔 Hipóteses Assumidas (cursorrules

§9.3)

Esta OS faz sentido sob as seguintes hipóteses. Se qualquer uma se revelar falsa durante a execução, **abortar e reportar**:

{{#LISTA_HIPOTESES}}
- {{ITEM}}
{{/LISTA_HIPOTESES}}

---
{{/LISTA_HIPOTESES}}

{{#IF_TOCA_CODIGO}}
## 🔍 ETAPA 0 — VERIFICAÇÃO PRÉVIA

### 0.1. Confirmar branch limpa

```bash
git status
```

### 0.2. Confirmar versões reais da stack relevante

```bash
npm ls {{LISTA_STACK_VALIDAR}}
```

Em caso de divergência com versões esperadas, **ABORTAR** e reportar antes de prosseguir (cursorrules §8.3 e

§8.8).

### 0.3. Reler regras aplicáveis do `cursorrules.md`

{{#CURSORRULES_REFS}}
- §{{REF}}
{{/CURSORRULES_REFS}}

### 0.4. Sinalizar QUALQUER divergência antes de prosseguir.

---
{{/IF_TOCA_CODIGO}}

{{^IF_TOCA_CODIGO}}
## 🔍 ETAPA 0 — VERIFICAÇÃO PRÉVIA (não-código)

### 0.1. Confirmar branch limpa

```bash
git status
```

### 0.2. Reler regras aplicáveis do `cursorrules.md`

{{#CURSORRULES_REFS}}
- §{{REF}}
{{/CURSORRULES_REFS}}

### 0.3. Sinalizar QUALQUER divergência

antes de prosseguir.

---
{{/IF_TOCA_CODIGO}}

## 🛠️ ETAPAS DE EXECUÇÃO

{{#ETAPAS}}
### {{NUMERO}}. {{NOME}}

{{DESCRICAO}}

{{#ENTREGAVEL}}
**Entregável:** `{{CAMINHO}}`
{{/ENTREGAVEL}}

{{#INSTRUCOES}}
- {{ITEM}}
{{/INSTRUCOES}}

{{/ETAPAS}}

---

{{#IF_REQUER_PROVENANCE}}
## 🧠 PROVENANCE OBRIGATÓRIO

Toda decisão arquitetural não-óbvia introduzida por esta OS DEVE vir acompanhada de comentário inline no formato:

```ts
```

```
// [PROVENANCE] <razão> | <fonte oficial>
| <data>
```

Conforme `cursorrules.md §8.7` (Anti-Vibe-Coding).

Em caso de incerteza, declarar honestamente conforme `cursorrules.md §9.1` em vez de fabricar plausibilidade.

---
{{/IF_REQUER_PROVENANCE}}

## 🚪 GATES DE APROVAÇÃO HUMANA

{{#GATES}}
### Gate {{NUMERO}} ⛔ — {{NOME}}

**Quando:** {{QUANDO}}

**Apresentar no chat:**
{{#APRESENTAR}}
- {{ITEM}}
{{/APRESENTAR}}

**Aguardar:** aprovação explícita do Product Owner antes de prosseguir.

{{/GATES}}

---

{{#IF_GERA_COMMIT}}
## 📝 COMMIT (após aprovação do último Gate)

```bash
git add {{#LISTA_ARQUIVOS_COMMIT}} {{ITEM}} {{/LISTA_ARQUIVOS_COMMIT}}
git commit -m "{{MENSAGEM_COMMIT}}

OS: docs/orders/OS-{{DATA_UTC}}-{{SLUG}}.md
Cursorrules: §{{#CURSORRULES_REFS}} {{REF}}, {{/CURSORRULES_REFS}}"
```

A mensagem deve seguir Conventional Commits e referenciar:
- OS arquivada em `docs/orders/`.
- Seções aplicáveis do `cursorrules.md`.

---
{{/IF_GERA_COMMIT}}

{{#IF_GERA_TAG}}
## 🏷️ TAG ANOTADA

```bash
git tag -a {{TAG_NOME}} -m "{{TAG_MENSAGEM}}"
git push --follow-tags
```

Conforme `cursorrules.md §7.3` — tag leve

é proibida.

---
{{/IF_GERA_TAG}}

## ✅ DEFINITION OF DONE

{{#DOD_ITEMS}}
- [ ] {{ITEM}}
{{/DOD_ITEMS}}

### Itens fixos do perfil 📐

- [ ] DoR (Pré-condições do Pedido) totalmente marcada antes da Etapa 0.
- [ ] Todos os Gates definidos foram aprovados pelo Product Owner.
- [ ] `CURSORRULES_REFS` lido e respeitado.
{{#IF_TOCA_CODIGO}}
- [ ] Stack real validada via `npm ls` (cursorrules §8.8).
- [ ] Nenhum padrão morto introduzido (cursorrules §8.2).
- [ ] `[PROVENANCE]` presente em decisões não-óbvias (cursorrules §8.7).
{{/IF_TOCA_CODIGO}}
{{#IF_GERA_COMMIT}}
- [ ] Commit Conventional realizado com referência à OS e cursorrules.
{{/IF_GERA_COMMIT}}
{{#IF_GERA_TAG}}
- [ ] Tag anotada `{{TAG_NOME}}` criada

(proibida tag leve).
{{/IF_GERA_TAG}}
- [ ] OS arquivada em `docs/orders/`.

---

{{#IF_TOCA_PRODUCAO}}
## 🔄 ROLLBACK

**Como reverter em caso de problema:**

```bash
{{ROLLBACK_COMANDOS}}
```

**Impactos colaterais conhecidos:**
{{ROLLBACK_IMPACTOS}}

### Após qualquer rollback

- [ ] Abrir OS de incidente em `docs/orders/` documentando causa.
- [ ] Atualizar `reportStatus.md` com bloco "Rollback {{DATA_UTC}}".
- [ ] Avaliar se itens originais da OS revertida precisam ser re-priorizados no `BACKLOG.md`.

---
{{/IF_TOCA_PRODUCAO}}

{{#OBSERVACOES_LIVRES}}
## 📌 OBSERVAÇÕES

{{OBSERVACOES_LIVRES}}

---
{{/OBSERVACOES_LIVRES}}

## 📎 RASTREABILIDADE

| Campo | Valor |
|---|---|
| OS arquivada em | `docs/orders/OS-{{DATA_UTC}}-{{SLUG}}.md` |
| Template usado | `prompt-retomada.md` v1.1 |
| Documento superior | `cursorrules.md` v1.1 + `prompt.md` v1.1 |
| Engine de renderização | `mustache.js` ^4.2.0 |
| Briefing de pedido | `docs/orders-requests/REQUEST-{{DATA_UTC}}-{{SLUG}}.md` (se arquivado) |

---

## 📜 HISTÓRICO DE MUDANÇAS DESTE TEMPLATE

### v1.1 — 2026-10-05
- Frontmatter YAML adicionado.
- `CURSORRULES_REFS` promovido a OBRIGATÓRIO no inventário.
- Bloco DoR (Pré-condições do Pedido) adicionado.

- `LISTA_HIPOTESES` adicionada como parâmetro opcional renderizável.
- Blocos shell envelopados em ` ```bash `.
- Tag HTML órfã removida.
- Mensagem de commit corrigida para usar `CURSORRULES_REFS` real.
- Seção "Sinais de que NÃO é caso de Molde Livre" adicionada.

### v1.0
- Versão inicial.

---

### 📂 Caminho: /Users/darwin/Projetos/Enki-mvp/docs/templates/README.md

---
documento: docs/templates/README
versão: 1.1
revisão: 2026-10-05
hash-pendente: <a calcular após aprovação>
autoridade: orientação
projeto: Enki
documento-superior: cursorrules.md v1.1 + prompt.md v1.1
sintaxe-templates: mustache.js v4.x
---

# 📚 Templates de Ordens de Serviço (OS) — Projeto Enki

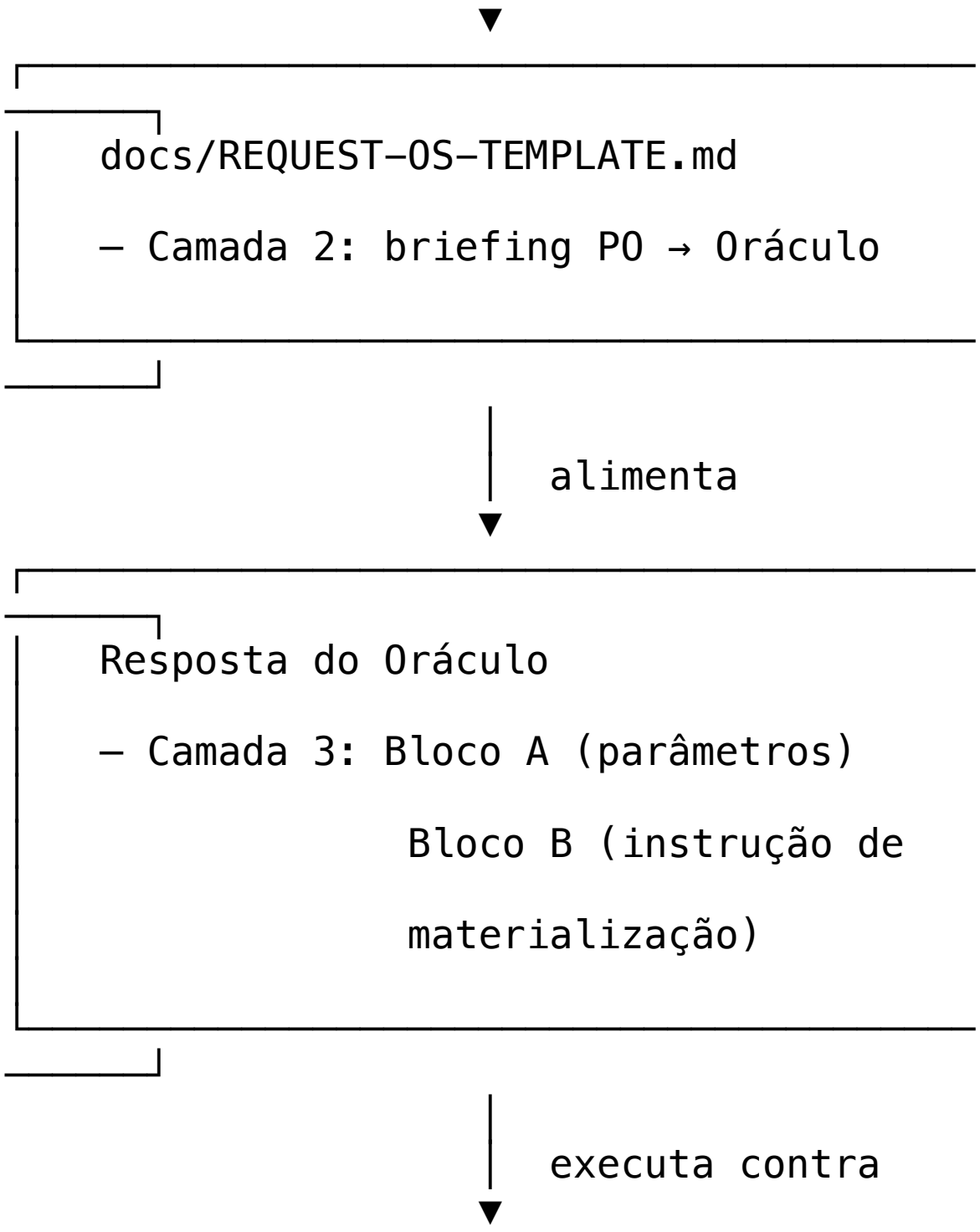
> **Camada:** Templates parametrizados (Camada 4 da arquitetura).
> **Autoridade superior:** `cursorrules.md` (v1.1) + `prompt.md` (v1.1).
> **Engine oficial:** `mustache.js` ^4.2.0.
> **Consumidores:** Antigravity (executa) e Oráculo (referencia).

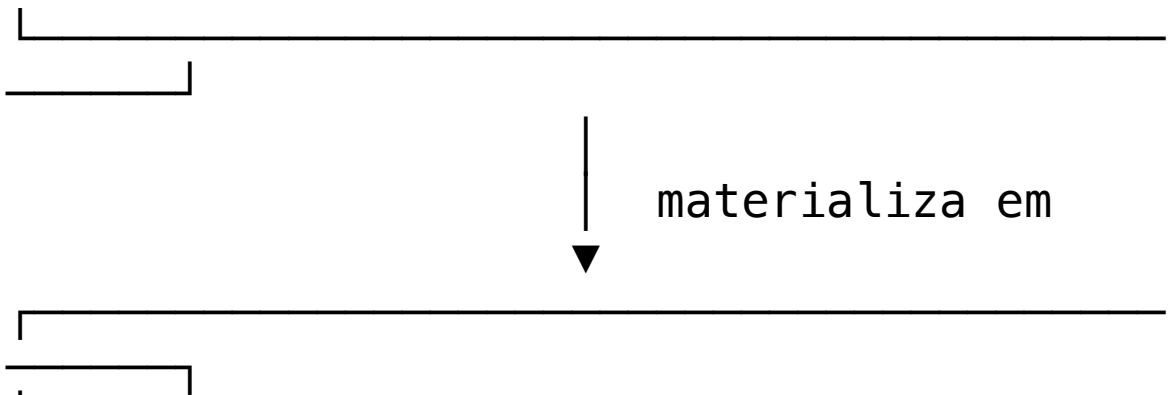
Este diretório contém os **moldes parametrizados** que o Antigravity materializa em OSs concretas, conforme parâmetros fornecidos pelo Oráculo de Delfos a partir de briefings do Product Owner.

---

## 🧭 Visão Geral do Sistema

```
┌──────────────────────────────────────────────────────────┐
│   SYSTEM PROMPT  (cursorrules.md + prompt.md)            │
│   — Camada 1: autoridade constitucional                  │
└──────────────────────────────────────────────────────────┘
                          │
                          │  governa
```


docs/REQUEST-OS-TEMPLATE.md
— Camada 2: briefing PO → Oráculo
alimenta
Resposta do Oráculo
— Camada 3: Bloco A (parâmetros)
Bloco B (instrução de
materialização)
executa contra
docs/templates/ ← VOCÊ ESTÁ AQUI
— Camada 4: esqueletos {{Mustache}}




---

## 📦 Catálogo de Templates

| Arquivo | Perfil | Uso | Risco |
|---|---|---|---|
| `prompt-retomada.md` | 📐 Molde Livre | Casos atípicos sem template específico | Variável |
| `exemplo-doc-sync.md` | 📘 Documentação | Atualizar BACKLOG / README / reportStatus + commit + tag | 🟢 Baixo |
| `exemplo-implementacao.md` | 📗 Implementação | Codificar 1 feature/fix com escopo delimitado | 🟡 Médio |
| `exemplo-governanca.md` | 📕 Governança / Release | Auditoria sistêmica + correções + bump de versão + |

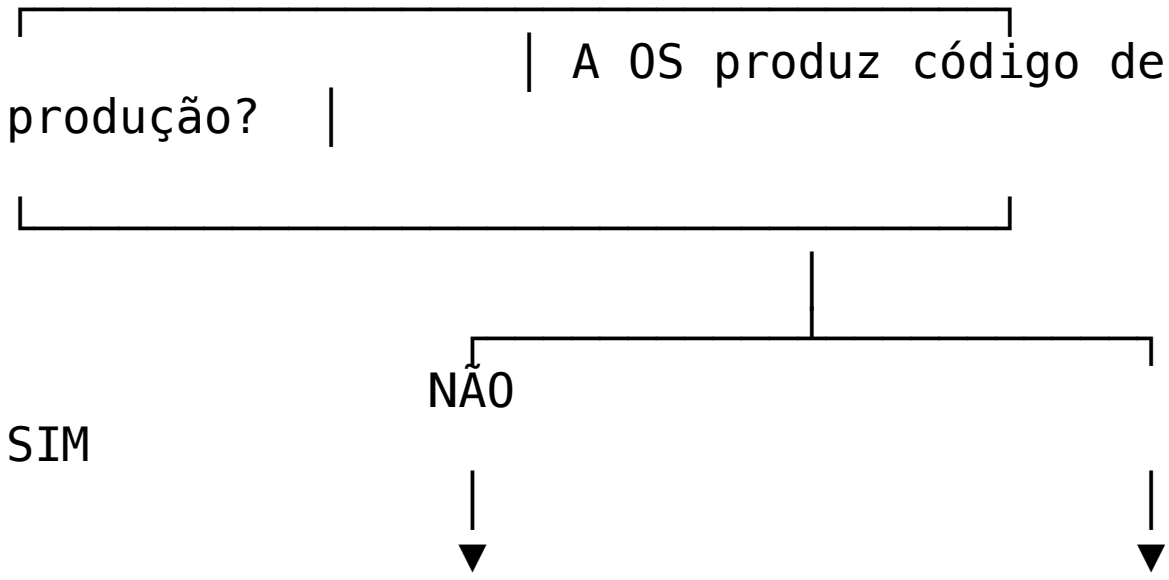
tag | 🔴 Alto |

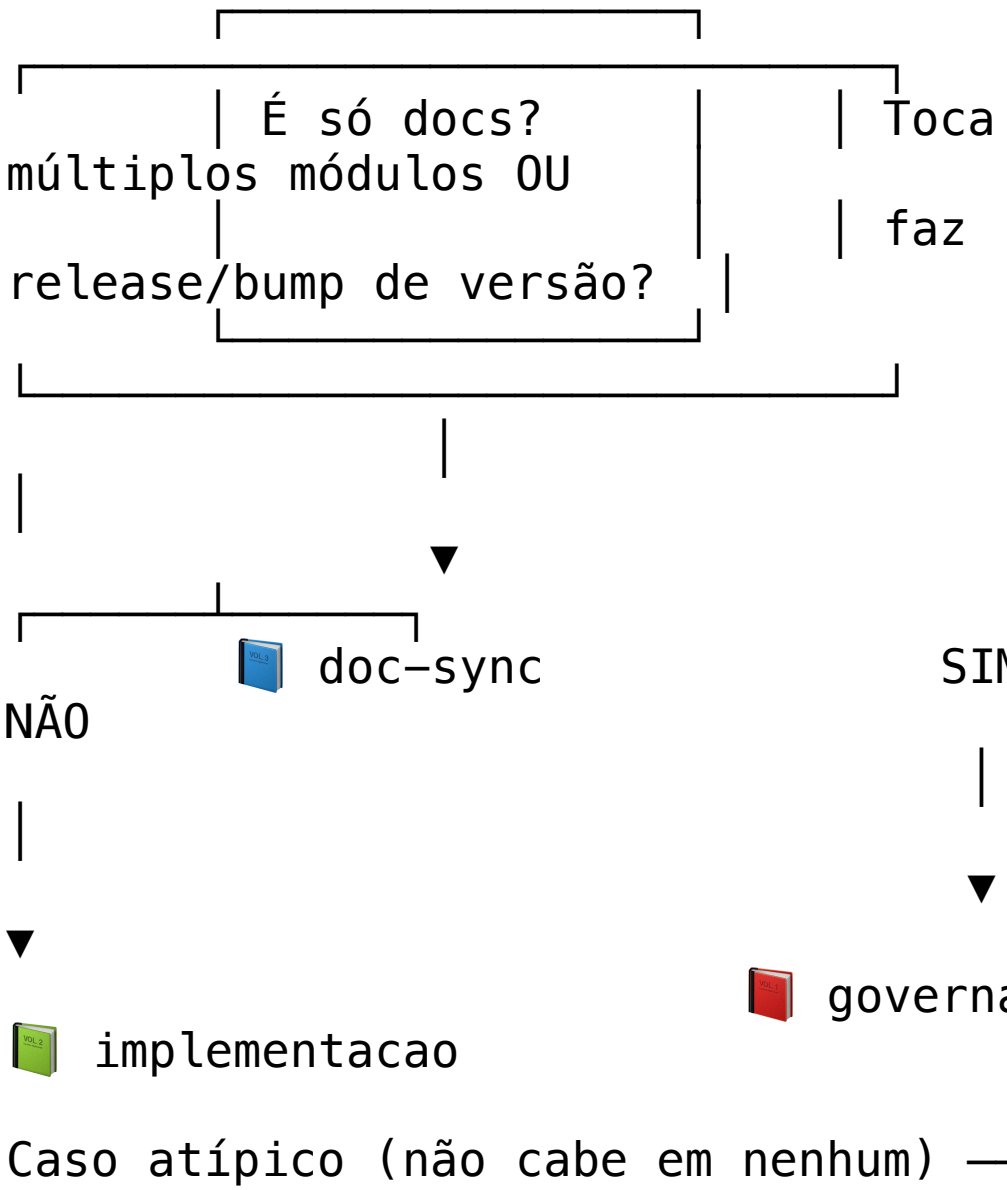
> ⚠️ **Sobre o prefixo `exemplo-`:** apesar do nome, **estes não são
> exemplos** — são os templates oficiais. A renomeação para
> `template-doc-sync.md`, etc. está **diferida** aguardando ADR
> dedicado em `docs/governance/`. Mudar o nome implica atualizar
> todas as OSs já materializadas que referenciam o template — por
> isso a decisão é tratada como migração planejada.

---

## 🌳 Árvore de Decisão — Qual Template Usar?

```


```




Caso atípico (não cabe em nenhum) → prompt-retomada
```

---

## 🎯 Sintaxe de Placeholders (Mustache, conforme `prompt.md §3.2`)

### Placeholder simples
```mustache
{{NOME_PARAMETRO}}

```
- Sempre em `UPPER_SNAKE_CASE`.
- Substituído por valor único (string, número, data).

### Lista (seção positiva)
```mustache
{{#LISTA_NOME}}
- {{ITEM}}
{{/LISTA_NOME}}
```
- Usado quando o parâmetro é um array.
- Bloco entre `{{#X}}` e `{{/X}}` é repetido para cada item.

### Condicional positiva
```mustache
{{#IF_TOCA_PRODUCAO}}
ROLLBACK obrigatório
{{/IF_TOCA_PRODUCAO}}
```
- Bloco incluído apenas se a flag for `true`.

### Condicional negativa
```mustache
{{^IF_REQUER_CORRECOES}}
Etapa 2 dispensada nesta OS.
{{/IF_REQUER_CORRECOES}}
```
- Bloco incluído apenas se a flag for `false`.
```

### Comentário interno
```mustache
{{! este texto não aparece no arquivo final }}
```
- Renderizado como vazio.
- Útil para documentar campos ao revisor humano.

---

## 🛡️ Regra Inviolável de Validação

Conforme `prompt.md §4`, antes de salvar qualquer OS materializada em `docs/orders/`, o Antigravity DEVE executar **dois passos** de validação:

### Passo 1 — Parâmetros não-resolvidos
```regex
\{\{(?![!#/^])[^}]+\}\}
```
Se houver QUALQUER ocorrência → ABORTAR (parâmetro faltando).

### Passo 2 — Seções remanescentes
```regex
\{\{[#/^][^}]+\}\}
```
Se houver QUALQUER ocorrência → ABORTAR (engine Mustache falhou).

> Esta regra impede que arquivos com `{{XYZ}}` literal vazem para o
> repositório, o que constituiria falha de governança ISO 27001/42001.

---

## 📋 Inventário Resumido de Parâmetros por Template

> A lista **completa e canônica** está no início de cada arquivo
> `.md` correspondente. O resumo abaixo serve para orientação
> rápida.

### 📐 `prompt-retomada.md` (molde livre)
- `TITULO_OS`, `OBJETIVO`, `DATA_UTC`, `SLUG`
- `LISTA_INCLUI`, `LISTA_NAO_INCLUI`
- `ETAPAS` (estrutura livre)
- `GATES` (mínimo 1)
- `CURSORRULES_REFS` (**obrigatório desde v1.1**)

### 📘 `exemplo-doc-sync.md`
- Identificação: `TITULO_OS`, `DATA_UTC`, `SLUG`
- Versão: `VERSAO_ATUAL`, `VERSAO_NOVA`
- Contexto: `ENTREGA_RECENTE`, `BRANCH_ALVO`
- Escopo: `LISTA_ARQUIVOS_DOCS`, `LISTA_INCLUI`, `LISTA_NAO_INCLUI`

- Opcionais: `IF_TAG_ANOTADA`,
`TAG_NOME`, `TAG_MENSAGEM`,
`CHANGELOG_ENTRY`
- Trilha: `MENSAGEM_COMMIT`,
`CURSORRULES_REFS`

### 📗 `exemplo-implementacao.md`
- Identificação + Versão (idem acima)
- Escopo cirúrgico:
`LISTA_ARQUIVOS_CRIAR`,
`LISTA_ARQUIVOS_MODIFICAR`
- Stack: `LISTA_STACK_VALIDAR`,
`LISTA_TESTES_OBRIGATORIOS`
- Provenance: `CURSORRULES_REFS`,
`LISTA_PROVENANCE_FONTES`
- Opcionais: `IF_PR_NECESSARIO`,
`IF_REQUER_VALIDACAO_VISUAL`,
  `IF_REQUER_TESTES_E2E`,
`IF_TOCA_FIRESTORE_RULES`,
  `LISTA_LIBS_PERMITIDAS`,
`LISTA_GREP_PROIBICOES`

### 📕 `exemplo-governanca.md`
- Todos os parâmetros do 📗 +
- Auditoria: `LISTA_DIRETORIOS_AUDITAR`,
`LISTA_REGRAS_VARRER`,
  `META_AUDITORIA`
- Release: `TIPO_BUMP`, `TAG_NOME`,
`TAG_MENSAGEM`
- Risco: `LISTA_RISCOS_DECLARADOS`,
`LISTA_BREAKING_CHANGES`,
  `LISTA_MIGRATIONS`
- Opcionais: `IF_BUMP_MAJOR`,

`IF_BUMP_MINOR`, `IF_REQUER_CORRECOES`,
  `IF_REQUER_HOTFIX_PLAN`,
`IF_DEPLOY_FIREBASE`, `PLANO_HOTFIX`

---

## ✅ Checklist de Sanidade da OS Gerada

Antes de aprovar uma OS gerada pelo Oráculo, valide:

- [ ] **Objetivo** cabe em UMA frase verificável
- [ ] **Contexto mínimo** informa estado atual + origem da OS
- [ ] **DoR** (Definition of Ready) preenchida
- [ ] **NÃO INCLUI** preenchido (essencial — evita escopo inflado)
- [ ] **Etapa 0** (verificação prévia) está presente
- [ ] Há pelo menos **UM Gate humano** definido
- [ ] **Entregáveis** têm caminho definido
- [ ] **`[PROVENANCE]`** exigido para decisões não-óbvias
- [ ] **`LISTA_HIPOTESES`** declarada (ou explicitamente "nenhuma")
- [ ] **DoD** com itens binários (verificáveis)
- [ ] **ROLLBACK** preenchido se há mudança em produção

- [ ] **Referências ao `cursorrules.md`** por `§` (não copy/paste)
- [ ] **Todos os `{{PLACEHOLDERS}}`** foram substituídos por valores reais
- [ ] **Branch alvo** segue convenção do perfil

---

## 📐 Convenções de Nomenclatura

### OSs em `docs/orders/`
```
OS-AAAA-MM-DD-slug-curto.md
```

Exemplos:
- `OS-2026-06-10-appshell.md`
- `OS-2026-06-15-doc-sync-V18-rc1.md`
- `OS-2026-07-01-release-V18.md`

### Branches por perfil

| Perfil | Prefixo de branch |
|---|---|
| 📘 Documentação | `docs/<slug>` |
| 📗 Implementação | `feat/<slug>` ou `fix/<slug>` |
| 📕 Governança / Release | `chore/release-V<x.y.z>` |
| 📐 Livre | a critério da OS |

### Mensagem de commit (Conventional

Commits)
```
<tipo>(<escopo>): <título curto>

<corpo opcional, explicando o "porquê">

OS: docs/orders/OS-AAAA-MM-DD-slug.md
Cursorrules: §<seções>
Refs: <AUDIT/PLAN/DIAGNOSIS relacionado>
```

Tipos canônicos: `feat`, `fix`, `chore`, `docs`, `refactor`, `test`, `perf`, `build`, `ci`.

---

## 🔄 Ciclo de Vida de uma OS

```
Pedido          → você preenche docs/REQUEST-OS-TEMPLATE.md
                  e envia ao Oráculo
                        │
                        ▼
Geração         → Oráculo responde com Bloco A + Bloco B
                        │
                        ▼
Revisão         → você valida parâmetros (Bloco A)
                  [aqui você pode pedir ajustes ao Oráculo]
```

```
                          |
                          ▼
Materialização → Antigravity substitui {{...}} no template
                 e VALIDA via 2 regex (prompt.md §4)
                          |
                          ▼
Arquivamento   → Antigravity salva em docs/orders/
                          |
                          ▼
Execução       → Antigravity executa as etapas, parando nos Gates
                          |
                          ▼
Conclusão      → DoD verificada, OS marcada como [EXECUTADA]
                 commit/tag conforme aplicável
```

---

## 📝 Manutenção dos Templates

- **Revisão trimestral:** validar que templates refletem boas
  práticas vigentes.
- **Revisão imediata:** após qualquer auditoria que detecte falha de
  governança originada em template inadequado.

- **Mudança em template:** exige OS própria do perfil 📘 Documentação,
  com justificativa registrada em ADR (`docs/governance/ADR-NNNN-*.md`).
- **Versionamento:** mudanças significativas marcam o template com
  nova versão no frontmatter YAML.

---

## 🚫 Anti-Padrões a Evitar

### ❌ Não faça
- Editar manualmente uma OS já materializada para "consertar"
  placeholder esquecido.
- Pular o `docs/REQUEST-OS-TEMPLATE.md` por achar que "é uma tarefa
  simples".
- Usar `prompt-retomada.md` quando um dos 3 estruturados se aplica.
- Aprovar OS sem ler o Bloco A inteiro.
- Materializar OS sem rodar as **duas** validações regex.
- Misturar perfis (ex.: 📗 Implementação que também faz release →
  fatie em duas OSs).
- Reaproveitar literalmente OS antiga sem revalidar contra
  `cursorrules.md` vigente.

### ✅ Faça
- Resista a OSs grandes — fatie em sub-

épicos.
- Use `[PROVENANCE]` mesmo quando parecer óbvio (auditoria futura
  agradece).
- Arquive briefings em `docs/orders-requests/` para rastreabilidade
  do processo decisório.
- Em dúvida sobre o template, escolha o de **maior rigor**
  (📕 > 📗 > 📘 > 📐).
- Declare `LISTA_HIPOTESES` quando houver pressuposto não-óbvio
  (cursorrules §9.3).

---

## 🆘 FAQ

**P: Posso editar um template diretamente sem passar por OS?**
R: Não. Mudanças em template exigem OS de perfil 📘 + ADR.

**P: O que faço se nenhum dos 3 templates estruturados serve?**
R: Use o `prompt-retomada.md` (📐 Livre). Mas reflita: talvez você
esteja diante de um padrão novo que merece um 5º template.

**P: O Oráculo pode propor um template novo?**
R: Pode propor, mas a criação exige sua

aprovação + ADR + OS 📘.

**P: E se o Antigravity detectar inconsistência entre OS e cursorrules?**
R: Ele DEVE abortar e reportar antes de executar (ver `cursorrules.md §10` — Hierarquia de Autoridade).

**P: Como reaproveito uma OS antiga similar?**
R: Não reaproveite o arquivo materializado — refaça o pedido ao Oráculo apontando a OS antiga como referência. Isso garante que mudanças no template ou cursorrules sejam incorporadas.

**P: O que faço se o Antigravity declarar incerteza durante uma OS?**
R: Conforme `cursorrules.md §9` e `prompt.md §5`, o Antigravity DEVE parar e registrar a incerteza no Gate humano mais próximo. Sua função como PO é decidir: (a) responder a incerteza; (b) abortar a OS; ou (c) aceitar como hipótese e prosseguir, registrando em `LISTA_HIPOTESES`.

**P: Por que existe um bloco "Definition of Ready" agora?**
R: Para evitar que OSs entrem em execução

com pedido mal-formado. A
DoR força reflexão sobre pré-condições
antes da Etapa 0 técnica.
Isso reduz drasticamente o risco de
retrabalho.

---

## 🔗 Mapeamento Pilar → Seção (migração v1.0 → v1.1)

Quem lembra dos antigos `P1, P2, P3, P4` do `cursorrules.md` v1.0,
use a tabela abaixo para localizar o equivalente na v1.1:

| v1.0 | v1.1 | Tema |
|---|---|---|
| P1 | §0.1 | Privacidade por Padrão (ISO 27001) |
| P2 | §0.2 | Transparência e Ética Algorítmica (ISO 42001) |
| P3 | §0.3 | Arquitetura "Rigid Shell" |
| P4 | §0.4 | Acessibilidade por Padrão (WCAG 2.2 AA) |

---

## 📎 Arquivos Relacionados

- **Camada 1 (constitucional):** `cursorrules.md`, `prompt.md` (raiz)
- **Camada 2 (briefing):** `docs/REQUEST-

OS-TEMPLATE.md`
- **Camada 4 (templates):** este diretório
- **Camada 5 (OSs executadas):** `docs/orders/`
- **Briefings históricos:** `docs/orders-requests/` (opcional)
- **ADRs:** `docs/governance/`
- **Auditorias:** `docs/audits/`
- **Diagnósticos:** `docs/diagnostics/`
- **Planos:** `docs/plans/`

---

## 📜 HISTÓRICO DE MUDANÇAS

### v1.1 — 2026-10-05
- Frontmatter YAML adicionado.
- Diagrama ASCII recriado (estava corrompido na v1.0).
- Tags HTML órfãs removidas.
- Engine Mustache referenciado oficialmente como `mustache.js v4.x`.
- Regex de validação atualizada (2 passos, conforme `prompt.md §4`).
- Notação `§` padronizada.
- Tabela de mapeamento Pilar → Seção adicionada.
- FAQ expandida (incerteza + DoR).

### v1.0
- Versão inicial.

---

### 📂 Caminho: /Users/darwin/Projetos/Enki-mvp/ref/enki.md

qual o artefato primitivo  de gerenciamento de tarefas tipo trello, mas analógico,  mais antigo conhecido

26 de abr.
Claude respondeu: O artefato mais antigo conhecido de gerenciamento de tarefas analógico no estilo Trello é o Kanban Board (看板), desenvolvido por Taiichi Ohno na Toyota no final…
O artefato mais antigo conhecido de gerenciamento de tarefas analógico no estilo Trello é o Kanban Board (看板), desenvolvido por Taiichi Ohno na Toyota no final dos anos 1940.

Kanban da Toyota (1948–1950)
Ohno criou o sistema para controlar o fluxo de produção nas fábricas da Toyota, inspirado no funcionamento dos supermercados americanos — onde os itens nas prateleiras eram repostos conforme eram consumidos.

O sistema físico original usava cartões de papel (kan = visual, ban = cartão) movidos entre colunas que representavam estágios do trabalho:

[ A FAZER ] → [ EM PROGRESSO ] →
[ CONCLUÍDO ]
Cada cartão continha informações sobre a peça ou tarefa, e só era "puxado" para a próxima etapa quando havia capacidade disponível — exatamente a lógica dos cards no Trello.

Existe algo ainda mais antigo?
Sim, de forma mais rudimentar:

Listas de tarefas em argila (Mesopotâmia, ~2000 a.C.) — escribas registravam tarefas e obrigações em tábuas, mas sem o conceito de fluxo entre colunas.
Quadros de produção militares (Primeira Guerra Mundial) — o diagrama de Gantt (1910–1915), de Henry Gantt, é um primo direto: barras horizontais em papel representando tarefas ao longo do tempo.
Mas nenhum deles tinha a mecânica de colunas com cartões móveis que define o estilo Trello. O Kanban da Toyota é o ancestral direto e mais fiel desse modelo.

Você disse: Listas de tarefas em argila . Listas de tarefas em argila . Tem algum nome?

26 de abr.
Claude respondeu: Boa pergunta!

Boa pergunta! Essas tábuas de argila não tinham um nome único e universal, mas eram chamadas genericamente de:

Tábuas de Argila (Clay Tablets)
Na Suméria e Babilônia, os escribas as usavam para registrar praticamente tudo — e algumas categorias se aproximam de "listas de tarefas":

Tipos mais relevantes encontrados:
Listas administrativas (Administrative Lists) — registros de trabalhadores, rações, tarefas a distribuir e obrigações a cumprir. Algumas funcionavam como ordens de serviço.
Tábuas de dívidas e obrigações — registravam o que ainda precisava ser feito ou pago, funcionando como um "pendente".
Cartas cuneiformes — algumas eram literalmente mensagens como "envie 10 homens para colher cevada amanhã", funcionando como delegação de tarefas.
Um exemplo famoso:
A "Lista de Reis Sumérios" (Sumerian King List, ~2100 a.C.) é uma das tábuas mais conhecidas — mas há também listas de trabalhadores de Ur III (~2100–2000 a.C.) que registravam quem devia fazer o quê, para quem e quando.

Curiosidade:

A escrita cuneiforme em si nasceu (~3200 a.C.) não para poesia ou religião, mas para contabilidade e gestão de tarefas — registrar grãos, trabalhadores e obrigações. Ou seja, o primeiro uso da escrita foi essencialmente um sistema de produtividade.

Você disse: listas de trabalhadores de Ur III .
listas de trabalhadores de Ur III .
Gostei disso. Dê um nome geográfico-histórico para elas.

26 de abr.
Claude respondeu: Ótima intuição!
Ótima intuição! Propondo um nome geográfico-histórico preciso:

Tábuas Administrativas de Ur III
(Ur III Administrative Tablets)

Ou, com mais sabor histórico:

Tábuas de Puzriš-Dagān
Puzriš-Dagān (atual Drehem, no Iraque) era o grande centro administrativo do Império de Ur III, e é de lá que vem a maior concentração dessas listas de trabalhadores e tarefas já encontradas — dezenas de milhares de tábuas escavadas.

Por que esse nome é preciso:

Elemento	Significado
Ur III	Terceira Dinastia de Ur (~2112–2004 a.C.)
Puzriš-Dagān	Centro redistribuidor, onde trabalhadores eram registrados e alocados
Drehem	Nome árabe moderno do sítio arqueológico

Como eram chamadas tecnicamente pelos assiriólogos:
"Balanced Accounts" – prestações de contas de trabalhadores
"Labor Allocation Tablets" – alocação de mão de obra
"Ration Lists" – listas de rações distribuídas por tarefa cumprida
A mais direta seria:

"Tábuas de Alocação de Puzriš-Dagān"

Um Trello de argila, com 4.000 anos de idade, enterrado no Iraque.

Você disse: procuro um nome simples para um aplicativo
procuro um nome simples para um aplicativo

26 de abr.
Claude respondeu: Que tipo de aplicativo é esse?
Que tipo de aplicativo é esse? Para eu sugerir um nome que realmente faça

sentido, me conta:

Você disse: P: O que o app faz?
P: O que o app faz?
R: Gerenciamento de tarefas / produtividade

P: Qual o tom do nome?
R: Curto e moderno (tipo Slack, Notion)

26 de abr.
Claude respondeu: Partindo da nossa conversa — argila, Suméria, tábuas, cuneiforme — mas com som moderno e curto:
Partindo da nossa conversa — argila, Suméria, tábuas, cuneiforme — mas com som moderno e curto:

Favoritos

| Nome | Raiz | Por quê funciona |
|---|---|---|
| Urr | Ur (cidade) | Ultra curto, soa tech |
| Kish | Kish (cidade suméria) | Uma sílaba, forte, memorável |
| Dagi | Puzriš-Dagān | Curto, amigável, original |
| Cunn | Cuneiforme | Soa moderno, remete à escrita |
| Slab | Tábua/slab de argila | Direto, visual, em inglês universal |

Outros candidatos

| Nome | Raiz |
|---|---|
| Tabi | Tábua |

Drehem  Nome árabe do sítio arqueológico
Nibru  Nome sumério de Nippur
Enki  Deus sumério da sabedoria
Meu favorito pessoal:
Kish
Uma sílaba. Sem significado óbvio em português ou inglês. Soa como um app de 2025. E carrega 5.000 anos de história administrativa suméria por baixo.

Quer explorar algum desses em mais profundidade?

Você disse: entre kish , enki e urr
entre kish , enki e urr

26 de abr.
Claude respondeu: Os três são bons.
Os três são bons. Deixa eu comparar com frieza:

| Kish | Enki | Urr |
| --- | --- | --- |
| Sílabas | 1 | 2 | 1 |
| Pronúncia | Universal | Universal |
| | Ambígua (ur? urr?) | |
| Busca no Google | Competida (nome comum) | |
| Competida (mitologia) | Limpa | |
| Som | Seco, forte | Fluido, amigável |
| | Estranho, difícil | |
| App Store | Provavelmente livre | |
| Provavelmente livre | Provavelmente livre | |

Meu ranking:

🥇 Enki
Dois sílabas que soam como uma. Fácil em qualquer idioma. O deus sumério da sabedoria e da organização — perfeito para produtividade. E tem uma leveza que Kish não tem.

🥈 Kish
Forte e seco. Mas "kish" tem competição — é sobrenome, cidade moderna no Irã, e pode soar brusco.

🥉 Urr
O mais original, mas a pronúncia gera dúvida. Ninguém sabe se fala "ur" ou "urr".

Enki é o nome.

---

### 📂 Caminho: /Users/darwin/Projetos/Enki-mvp/ref/PO-GUIDE.MD

---
documento: docs/PO-GUIDE
versão: 1.0

```
revisão: 2026-10-05
hash-pendente: <a calcular após
aprovação>
autoridade: guia operacional
público: Product Owner + Antigravity
(leitor secundário)
projeto: Enki
documento-superior: cursorrules.md v1.1 +
prompt.md v1.1
modo: híbrido (manual atual + marcas [com
enki] para automação futura)
---

# 📖 Guia do Product Owner — Projeto Enki

> Este é seu **manual operacional** como
PO do Enki. Não é doc de
> referência — é guia de **uso diário**,
denso e enciclopédico.
> Leia uma vez do início ao fim. Depois
consulte por seção.
>
> **Leitor primário:** Aristóteles Soares
Benício (PO).
> **Leitor secundário:** Antigravity
(consulta quando recebe OS).
>
> **Pressuposto sobre automação:** o CLI
`enki` (Sprint B) está
> especificado mas não implementado. As
seções marcadas com
> `[com enki: ...]` indicam o que ficará
automático quando a Fase 1
```

> do CLI for implementada.

---

## 🗺️ COMO USAR ESTE GUIA

### Por contexto de uso

| Situação | Vá direto para |
|---|---|
| "Não entendo quem é quem nesse sistema" | Camada 1 |
| "Preciso abrir uma OS agora" | Camada 2 + Camada 3 |
| "Antigravity me pediu decisão em um Gate" | Camada 2 §2.4 |
| "Algo deu errado, e agora?" | Camada 4 |
| "Quero parar de fazer trabalho manual" | Camada 5 |
| "Acho que algo está estranho no projeto" | Camada 6 |
| "Não conheço esse termo" | Glossário (Parte 4) |

### Convenções tipográficas

- `código` = comando shell ou nome de arquivo/regra
- **negrito** = ação que você executa
- _itálico_ = conceito sendo definido
- ⚠️ = atenção
- ⛔ = nunca faça

- ✅ = sempre faça
- 🤖 = trabalho do Antigravity
- 🏛️ = trabalho do Oráculo
- 👤 = seu trabalho como PO
- `[com enki: ...]` = ficará automático com o CLI

---

# 📐 CAMADA 1 — MAPA MENTAL

## 1.1. Os três atores

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│      👤 PO                  🏛️ Oráculo                🤖 Antigravity │
│      (você)                 (Claude)                  (agente IA)    │
│                                                                      │
│      Decide                 Valida                    Executa        │
│      o quê e por quê        forma e contexto          tecnicamente   │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

### 1.1.a. Você (👤 PO)

| Faz | Não Faz |
|---|---|
| Decide objetivo de cada OS | Escreve código |
| Preenche briefing | Gera parâmetros (Bloco A) |
| Aprova/rejeita Gates | Materializa templates |
| Aceita ou rejeita riscos | Faz commits ou tags |
| Define DoD específica | Roda testes |
| Audita coerência periódica | Operações git/firebase |

> **Princípio:** seu valor é **decisão**, não **execução**. Toda vez
> que você se vê fazendo trabalho mecânico, é sintoma de processo
> mal-desenhado.

### 1.1.b. Oráculo (🏛️ Claude)

| Faz | Não Faz |
|---|---|
| Lê briefing + cursorrules + templates | Executa código |
| Escolhe template apropriado | Acessa repositório direto |
| Calcula parâmetros (Bloco A) | Faz commits/tags |

| Gera instrução ao Antigravity (Bloco B)
| Decide aceitar risco |
| Sugere fatiamento se OS está grande |
Aprova Gates |
| Aponta inconsistências com cursorrules
| Toma ações de produção |

> **Princípio:** o Oráculo é
**assessor**, não **executor**. Você o
> consulta antes da OS começar. Depois
ele sai de cena até a
> próxima OS.

### 1.1.c. Antigravity (🤖 agente dev)

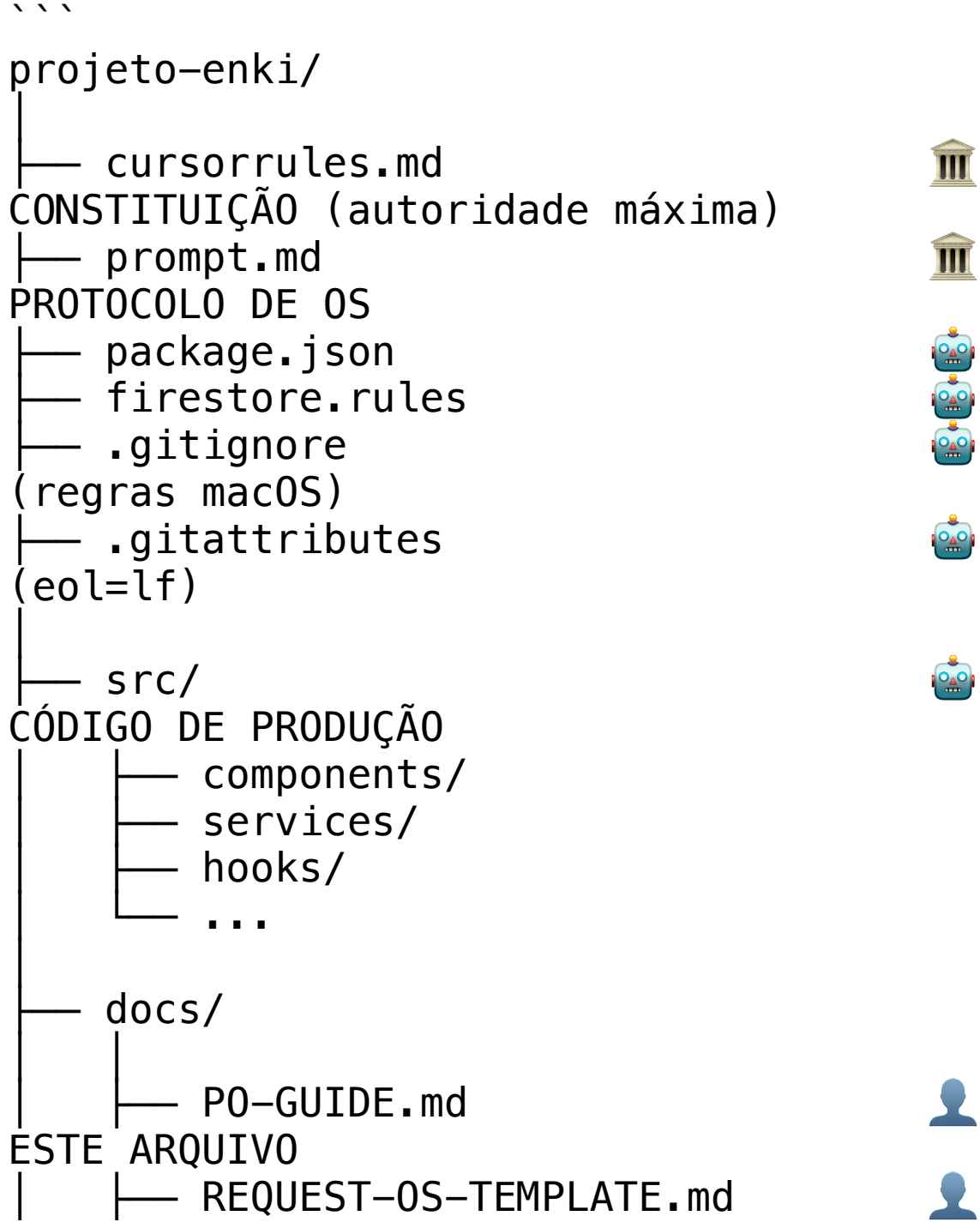
| Faz | Não Faz |
|---|---|
| Localiza template no repo | Pula
validação Mustache |
| Renderiza Mustache | Pula Gates |
| Valida (dupla regex) | Aceita risco
sozinho |
| Salva OS materializada | Modifica
`cursorrules.md` |
| Executa etapas técnicas | Adiciona
dependência fora de allowlist |
| Para em todo Gate | Refatora fora de
escopo |
| Reporta incertezas honestamente |
Esconde falhas |

> **Princípio:** o Antigravity é
**executor disciplinado**. Toda

> liberdade que ele tem está enquadrada no template + cursorrules.

---

## 1.2. Onde mora cada coisa



```
BRIEFING (você preenche)
│   │
│   ├── templates/                    🏛️
MOLDES MUSTACHE
│   │   ├── README.md
(referência)
│   │   ├── prompt-retomada.md        📐
livre
│   │   ├── exemplo-doc-sync.md       📘
docs
│   │   ├── exemplo-implementacao.md  📗
código
│   │   └── exemplo-governanca.md     📕
release
│   │
│   ├── orders/                       🤖
OSs MATERIALIZADAS
│   │   ├── OS-2026-10-05-appshell.md
│   │   └── ...
│   │
│   ├── orders-requests/              👤
BRIEFINGS ARQUIVADOS
│   │   ├── REQUEST-2026-10-05-
appshell.md
│   │   └── ...
│   │
│   ├── audits/                       🤖
AUDITORIAS GERADAS
│   │   └── AUDIT-V17.3.md
│   │
│   ├── plans/                        👤
PLANOS MACRO
│   │   └── PLAN-V18.md
```

```
│   │
│   ├── diagnostics/                    👤
🤖 DIAGNÓSTICOS
│   │
│   └── governance/                     👤
ADRs E SPECs
│       ├── SPEC-enki-cli.md
│       └── ADR-NNNN-*.md
│
├── reportStatus.md                     🤖
STATUS DO PROJETO
├── BACKLOG.md                          👤
🤖 BACKLOG VIVO
├── README.md                           🤖
README DO REPO
└── CHANGELOG.md                        🤖
HISTÓRICO DE RELEASES
```

### Quem lê, quem escreve

| Arquivo | Quem lê | Quem escreve |
|---|---|---|
| `cursorrules.md` | 🤖🏛️ sempre | 👤 raramente (com ADR) |
| `prompt.md` | 🤖🏛️ sempre | 👤 raramente (com ADR) |
| `REQUEST-OS-TEMPLATE.md` | 👤 sempre | 👤 raramente (atualiza estrutura) |
| `docs/templates/*.md` | 🤖 sob demanda | 👤 raramente (com ADR) |
| `docs/orders/OS-*.md` | 🤖👤 | 🤖 |
| `docs/orders-requests/*.md` | 👤 | 👤 |

| `docs/audits/AUDIT-*.md` | 🤖👤 | 🤖 |
| `BACKLOG.md` | 👤🤖 | 🤖 (via OS) + 👤 (anotações) |
| `reportStatus.md` | 👤 | 🤖 (via OS) |
| `README.md` | público | 🤖 (via OS) |
| `CHANGELOG.md` | público | 🤖 (via OS de governança) |

---

## 1.3. O ciclo de vida de uma OS

```
                                📝                              🏛️
🤖
   PREPARO ──────► BRIEFING ──────► OS GERADA ──────► EXECUÇÃO
   (você)        (Oráculo)                          (Antigravity)
     │              │              │              │
     ▼              ▼              ▼              ▼
   /tmp/          resposta       docs/          docs/
   briefing       do chat        orders/        audits/

+ commits

+ tags

|
```

▼

🚪 GATES ⛔

|

▼

✅ CONCLUÍDA

(arquivada)
```

### 1.3.a. Tempo médio típico (manual atual)

| Etapa | Tempo |
|---|---|
| 1. Preparar briefing | ~15 min |
| 2. Oráculo gera OS | ~2 min |
| 3. PO revisa Bloco A | ~3 min |
| 4. Antigravity materializa | ~1 min |
| 5. Execução (varia por perfil) | 10 min – várias horas |
| 6. Arquivamento | ~2 min |

**Total mínimo:** ~33 min para OS pequena de doc-sync.
**[com enki: ~7 min]** — economia de ~80% no preparo + arquivamento.

---
```

## 1.4. Os quatro perfis de OS

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│    📐 LIVRE                    Casos atípicos                        │
│    prompt-retomada.md          Quando nenhum outro serve             │
│    Risco: variável                                                   │
│                                                                      │
│    📘 DOC-SYNC                 Atualizar BACKLOG/README/etc.         │
│    exemplo-doc-sync.md         SEM tocar código de produção          │
│    Risco: 🟢 baixo                                                   │
│                                                                      │
│    📗 IMPLEMENTAÇÃO            Codar 1 feature/fix                   │
│    exemplo-implementacao.md    Escopo cirúrgico em código            │
│    Risco: 🟡 médio                                                   │
│                                                                      │
│    📕 GOVERNANÇA / RELEASE     Auditoria +
```

```
correções + release |
|   exemplo-governanca.md     Bump de
versão + tag + deploy   |
|   Risco: 🔴 alto
|
|
|
└──────────────────────────────────────────────────
──────────────────────┘
```

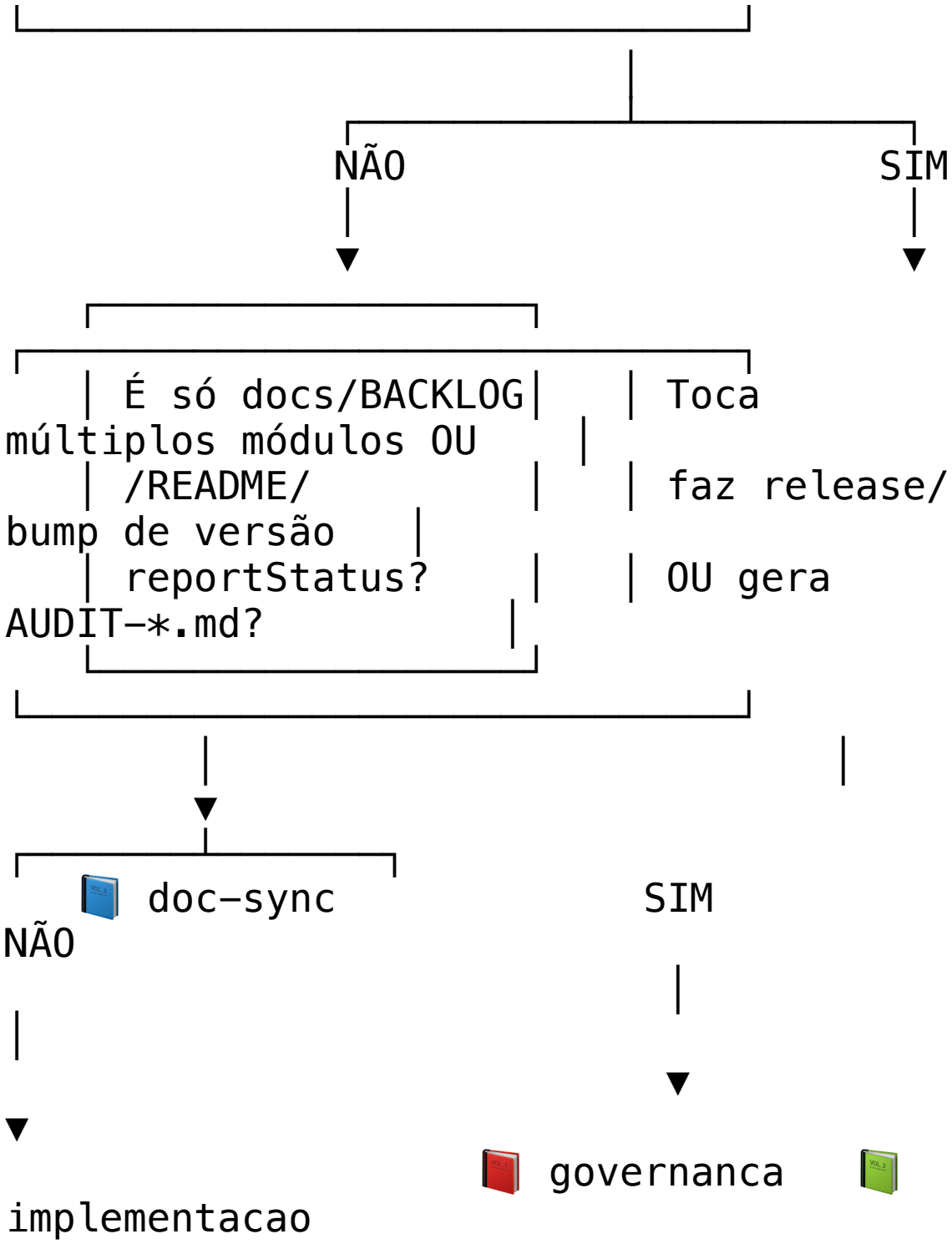
> **Regra mnemônica:** em caso de dúvida, escolha o de **maior
> rigor** (📕 > 📗 > 📘 > 📐). É mais fácil "abafar" rigor extra do
> que descobrir que faltou no meio da execução.

---

# 🎯 CAMADA 2 — DECISÕES DIÁRIAS DO PO

## 2.1. Decision tree — Qual template usar?

```
┌──────────────────────────────────────┐
                    | A OS produz código
de        |
                    | produção
(.ts/.tsx/.css/etc)?|
```




Caso atípico que não cabe em nenhum dos 3 → prompt-retomada

```

### 2.1.a. Tabela de decisão rápida
```

| Sua tarefa | Template |
|---|---|
| "Atualizar BACKLOG.md com itens da última auditoria" | 📘 |
| "Atualizar README.md com nova versão" | 📘 |
| "Criar `AppShell.tsx` em `src/components/`" | 📗 |
| "Refatorar hook `useUser` para Zero-Trust TS" | 📗 |
| "Auditar Shadow Logic + corrigir críticos + fechar V18" | 📕 |
| "Bump de MAJOR com breaking changes documentados" | 📕 |
| "Investigar por que CI quebrou (sem commit ainda)" | 📐 |
| "Discutir arquitetura de novo módulo (ADR)" | 📐 |

### 2.1.b. Sinais de que você está usando o template errado

| Sintoma | Template provável | Trocar para |
|---|---|---|
| OS 📘 termina criando `.tsx` | escolha errada | 📗 |
| OS 📗 inclui `git tag -a` | escolha errada | 📕 |
| OS 📗 toca 3 módulos não-relacionados | escopo inflado | fatiar em 3 OSs 📗 |
| OS 📐 com Gates idênticos a 📗 padrão |

uso indevido do livre | 📗 |
| OS 📕 sem `AUDIT-*.md` gerado | escolha errada | 📗 (se só fix) |

---

## 2.2. Pré-DoR — Como saber se o briefing está pronto

Antes de enviar ao Oráculo, verifique **mentalmente** (ou via `enki briefing validate`):

```
┌──────────────────────────────────────────────────────────────────────┐
│  Pré-DoR Checklist                                                   │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  [ ] Objetivo cabe em UMA frase verificável?                         │
│                                                                      │
│  [ ] LISTA_NAO_INCLUI está preenchida?                               │
│      (sem ela, escopo será inflado)                                  │
│                                                                      │
│  [ ] LISTA_HIPOTESES declarada (ou
```

```
"nenhuma")?          |
|
|
|  [ ] Critérios de sucesso são
MENSURÁVEIS?              |
|      (ruim: "ficar bom"
|
|       bom: "lint+typecheck verdes, CLS
< 0.05")          |
|
|
|  [ ] Dependências externas listadas?
|
|
|
|  [ ] OS NÃO acumula múltiplos perfis?
|
|      (se acumula, fatie ANTES)
|
|
└──────────────────────────────────────
──────────────────┘
```

> **Regra:** se você não consegue marcar TODOS os checkboxes acima
> com confiança, o briefing **NÃO** está pronto. Refine antes de
> chamar o Oráculo.

`[com enki: enki briefing validate <path>]`

---

## 2.3. Como revisar o Bloco A do Oráculo

Quando o Oráculo responder com Bloco A + B, **não** aprove no
automático. Faça esta checagem rápida (~3 min):

```
┌──────────────────────────────────────────────────────────────────────┐
│  Checklist de Revisão do Bloco A                                     │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  [ ] Template escolhido faz sentido para o objetivo?                 │
│                                                                      │
│  [ ] BRANCH_BASE é tag ou branch protegida?                          │
│                                                                      │
│  [ ] LISTA_ARQUIVOS_CRIAR / MODIFICAR estão no escopo?               │
│                                                                      │
│  [ ] CURSORRULES_REFS cita seções RELEVANTES?                        │
```

```
|      (não exaustivas — só as que se
aplicam)                |
|
|
|   [ ] LISTA_PROVENANCE_FONTES tem URLs
OFICIAIS?               |
|      (não Stack Overflow, Medium, blog)
|
|
|
|   [ ] DOD_ITEMS específicos são
VERIFICÁVEIS?                |
|      (não "fazer bem feito")
|
|
|
|   [ ] Se IF_TOCA_FIRESTORE_RULES = true,
OS reconhece    |
|      isso explicitamente?
|
|
|
└──────────────────────────────────────────
──────────────────┘
```

### 2.3.a. O que dizer ao Oráculo

| Situação | Comando |
|---|---|
| Tudo OK | "Aprovado" |
| Pequeno ajuste pontual | "Ajusta X em Y" |

| Muitos ajustes | "Refaz com foco em Z" |
| Escopo está grande demais | "Sugere fatiamento" |
| Template parece errado | "Por que escolheu 📗 e não 📕?" |

---

## 2.4. Como decidir em um Gate

Antigravity vai parar em pontos pré-definidos da OS pedindo decisão.
Existem 3 tipos de Gate por perfil:

### 2.4.a. Gates de implementação (📗)

| Gate | O que Antigravity apresenta | Sua decisão |
|---|---|---|
| Gate 1 | git diff + testes + verificações anti-vibe-coding | ✅ commit OU 🔧 ajustar OU ❌ abortar |
| Gate 2 | git log + push aceito + status CI | ✅ encerrar OS OU ➡️ próxima OS |

### 2.4.b. Gates de governança (📕)

| Gate | O que Antigravity apresenta | Sua decisão |
|---|---|---|
| Gate 1 | Resumo executivo da auditoria + TOP 5 críticos | Decidir quais

Críticos: corrigir agora vs aceitar como risco |
| Gate 2 | Correções aplicadas + zero críticos pendentes | ✅ release OU 🔧 ajustar |
| Gate 3 | Commit + tag + push + deploy + smoke tests | ✅ encerrar OS |

### 2.4.c. Gates de doc-sync (📘)

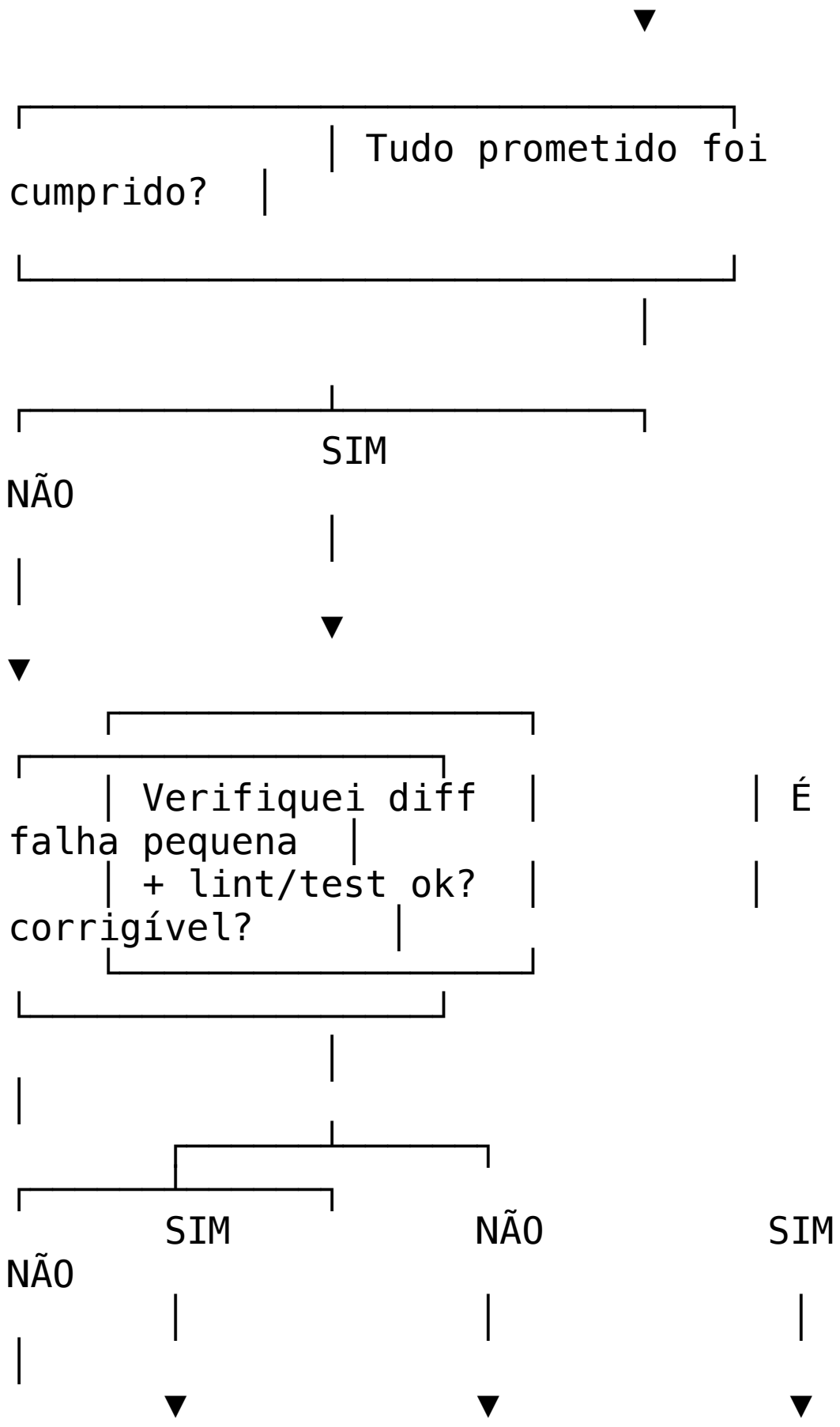
| Gate | O que Antigravity apresenta | Sua decisão |
|---|---|---|
| Gate 1 | git diff dos arquivos docs | ✅ commitar OU 🔧 ajustar |
| Gate 2 | commit + push (+ tag se aplicável) | ✅ encerrar OS |

---

## 2.5. Quando aprovar vs rejeitar um Gate

```

┌──────────────────────────────────────┐
                    | Antigravity apresentou   |
                    | status no Gate |
└──────────────────────────────────────┘
                                    |
```


Tudo prometido foi cumprido?
SIM
NÃO
Verifiquei diff + lint/test ok?
É falha pequena corrigível?
SIM
NÃO
SIM
NÃO

▼

✅ APROVAR 🔧 PEDIR 🔧 PEDIR ❌ ABORTAR
PROVA AJUSTE a OS
EXPLÍCITA
```

### 2.5.a. Sinais de "aprovar com confiança"

- ✅ Diff é exatamente o esperado (não há arquivos extras).
- ✅ Lint + typecheck + test verdes (saída literal mostrada).
- ✅ Antigravity declarou as incertezas honestamente
  (`// TODO: provenance-pending`) e você decidiu sobre cada uma.
- ✅ Zero tokens proibidos no diff.
- ✅ `[PROVENANCE]` presente em decisões não-óbvias.

### 2.5.b. Sinais de "pedir mais prova antes de aprovar"

- ⚠️ Antigravity diz "testes verdes" mas não cola a saída literal.
- ⚠️ Diff mostra arquivos fora da `LISTA_ARQUIVOS_MODIFICAR`.
- ⚠️ Há `// TODO` ou `// FIXME` que não estavam declarados.
- ⚠️ Aparece dependência nova não
```

autorizada.

### 2.5.c. Sinais de "abortar a OS"

- 🚫 Antigravity diz "vou pular validação" ou similar.
- 🚫 Há `any`, `@ts-ignore` introduzidos.
- 🚫 Houve refatoração fora de escopo "porque parecia óbvio".
- 🚫 Auditoria revelou Crítico não-declarado em DoR.
- 🚫 Antigravity ignorou Gate anterior e seguiu sozinho.

> **Princípio:** abortar uma OS **não é fracasso seu**. É evidência
> de que o sistema de governança funcionou. Releases ruins entregues
> são piores do que OSs abortadas.

---

## 2.6. Quando aceitar risco vs corrigir agora

Em OSs 📕 (governança), o Gate 1 te pede decisão sobre cada item
Crítico encontrado na auditoria. Use a matriz:

```
                    IMPACTO SE FALHAR
                    ┌──────────┬──────────┐
```

| | | ALTO | BAIXO |
|---|---|---|---|
| EFORÇO DA CORR. | ALTO | CORRIGE AGORA | ACEITA COM PRAZO |
| | BAIXO | CORRIGE AGORA | CORRIGE AGORA |

```

### 2.6.a. Critérios para "aceitar como risco"

Aceite formalmente como risco APENAS se:

1. ✅ O item NÃO é de segurança crítica (XSS, Shadow Logic, segredos).
2. ✅ O risco está documentado em `BACKLOG.md` com tag `[PRIORIDADE: SEGURANÇA]`.
3. ✅ Há prazo declarado para endereçamento (próximo épico, p.ex.).
4. ✅ Você (PO) registra a decisão **por escrito** na OS.

### 2.6.b. NUNCA aceite como risco

- 🚫 Itens de **segurança crítica** (cursorrules §2 inteiro).
- 🚫 Shadow Logic ausente em coleção sensível (`users`, `boards`).
- 🚫 Segredos no client bundle.
```

- 🚫 XSS em campo de input livre.
- 🚫 `any` em código que toca autenticação/autorização.

> **Regra de ouro:** se a hesitação te dá frio na barriga, **não
> aceite**. Vá para correção agora ou aborte a OS.

---

⬅️ END **Fim da Parte 1/4.**

**📍 Próxima:** Parte 2/4 — Camada 3 (Cheat Sheet Shell completo).

Aguardo *"continua"* (ou *"ajusta X na Parte 1"*).

---